

Software Secondary **Development Manual**

Software Development Kit (SDK) Document

V2.0

Disclaimer

Due to the diversity and complexity of software development processes, control objects, control devices, and software operating environments, despite rigorous testing and validation of documentation and secondary development interfaces, defects (**bugs**) and omissions may inevitably exist. Our company will make every effort to correct and update these. Should you encounter any issues or have questions during use, please contact us promptly for feedback. We will verify and address them in a timely manner, providing you with the latest version.

We solemnly **remind** you that secondary development is a process for which you bear full responsibility. Please carefully read this document, rigorously debug, test, and validate the software, and ensure adequate **safety** precautions are in place to prevent personal injury and damage to equipment or property. Only after verifying the software is correct and error-free should you deploy it in actual production environments for software and control purposes. During the development process, please contact us promptly if you encounter any issues so we can provide relevant assistance

and suggestions.

Thank you for choosing our product. We wish you a satisfying experience!

Table of Contents

1	Terminology, Concepts, Definitions	10
1.1	Conventions	10
1.1.1	Action/Parameter Settings	10
1.1.2	Status/Parameter Read	11
1.1.3	Object ID	11
1.1.4	Multi-Controller Device Secondary Development	11
1.1.5	Function Naming	12
1.1.6	Function Call Timing	12
1.1.7	Compatibility Functions	12
1.1.8	Parameter Memory Storage	12
1.2	Units of Measurement and Conversion	13
1.2.1	Conversion Principles	13
1.2.2	Coefficient Definition	13
1.2.3	Software sends commands to the controller	13
1.2.4	Software reads controller position and speed	13
1.2.5	Example	13
1.2.6	Acceleration and deceleration	16
1.2.7	Stepper Microstepping	17
1.2.8	Servo Electronic Ratio	17
1.2.9	Conversion Function	17
1.3	Coordinate System	17
1.3.1	Definition of Coordinate Systems	17
1.3.2	Logical Coordinate Memory and Calibration	17
1.3.3	Coordinate System Adjustment	17
1.4	Motion Control Concepts	18
1.4.1	Types of Motion	18
1.4.2	Components of a Motion Control System	18
1.4.3	Limit Switches, Zero Position Switches, and Software Limits	19
1.4.4	Acceleration and Deceleration	20
1.4.5	Stepper Microstepping	20
1.4.6	Lead Screw Pitch	20
1.4.7	Gear Ratio/Reduction Ratio	20
1.4.8	Encoder	20
1.4.9	Glass Scale	20
1.4.10	Open-loop control	20
1.4.11	Closed-loop control	21
1.4.12	Relative and Absolute	21
1.5	Factors Affecting System Accuracy	21
1.5.1	Mechanical Precision	21
1.5.2	Motor/Drive Accuracy	21
1.5.3	Controller Pulse Output	21

1.6	Function Mode	21
1.6.1	Open-Loop Zero-Point Search Mode	21
1.6.2	Closed-Loop Zero-Point Search Mode	24
1.6.3	Constant Speed Mode	24
1.6.4	Open-Loop Positioning Mode (Open-Loop Positioning Mode)	25
1.6.5	Closed-Loop Positioning Mode (Closed-Loop Positioning Mode) ..	27
1.6.6	Multi-axis Interpolation Mode	27
1.6.7	Arc Interpolation Mode	28
1.6.8	Stream Command Mode	28
1.6.9	PLC Function.....	28
1.6.10	Software Limit Function	29
1.6.11	Synchronous Trigger Function	29
2	Development Methods and Processes	30
2.1	Overview	30
2.2	Software Design Recommendations	30
2.2.1	State Machine (Timer-Driven).....	30
2.2.2	Thread	33
2.3	Zero-Position Mode	34
2.4	Constant Speed Mode	35
2.5	Open-Loop Positioning Mode	36
2.6	Closed-loop Positioning Mode	37
2.7	Multi-axis Interpolation Mode (Including Linear Interpolation).....	38
2.8	Arc Interpolation Mode	39
2.9	Flow Command Mode	40
2.10	PLC Mode.....	41
2.11	Thread Safety	42
2.12	Calling Conventions	42
2.13	Return Value Evaluation	42
3	Function Description	43
3.1	Initialization	43
3.1.1	MT_Init.....	43
3.1.2	MT_DeInit	43
3.1.3	MT_Get_DLL_Version	43
3.2	Communication Port	44
3.2.1	MT_Close_UART	44
3.2.2	MT_Open_UART	44
3.2.3	MT_Open_UART_Unicode.....	45
3.2.4	MT_Open_UART_ID.....	45
3.2.5	MT_Close_USB.....	46
3.2.6	MT_Open_USB.....	46
3.2.7	MT_Open_USB_SN	46
3.2.8	MT_Close_Net	47
3.2.9	MT_Open_Network.....	47

3.2.10	MT_Open_Net_Name.....	48
3.3	Communication Handshake	48
3.3.1	MT_Check	48
3.4	Hardware Information	49
3.4.1	MT_Get_Product_Resource	49
3.4.2	MT_Get_Product_ID.....	50
3.4.3	MT_Get_Product_Serial_Number	50
3.4.4	MT_Get_Product_Version[Compatible].....	50
3.4.5	MT_Get_Product_Version2.....	51
3.5	Zero Mode (Open-Loop and Closed-Loop).....	51
3.5.1	MT_Set_Axis_Mode_Home[Compatible]	51
3.5.2	MT_Set_Axis_Mode_Home_Home_Switch	52
3.5.3	MT_Set_Axis_Mode_Home_Home_Zero.....	52
3.5.4	MT_Set_Axis_Mode_Home_Encoder_Index.....	53
3.5.5	MT_Set_Axis_Mode_Home_Encoder_Home_Switch	53
3.5.6	MT_Set_Axis_Home_V	54
3.5.7	MT_Set_Axis_Home_Stop	54
3.5.8	MT_Set_Axis_Home_V_Start	54
3.5.9	MT_Set_Axis_Home_Acceleration	55
3.5.10	MT_Set_Axis_Home_Dec	55
3.5.11	MT_Get_Axis_Home_Acc	56
3.5.12	MT_Get_Axis_Home_Dec.....	56
3.6	Constant Speed Mode	56
3.6.1	MT_Set_Axis_Mode_Velocity.....	56
3.6.2	MT_Get_Axis_Velocity_V_Target.....	57
3.6.3	MT_Set_Axis_Velocity_V_Target_Absolute	57
3.6.4	MT_Set_Axis_Velocity_V_Target_Relative	58
3.6.5	MT_Set_Axis_Velocity_Stop.....	58
3.6.6	MT_Set_Axis_Velocity_V_Start	59
3.6.7	MT_Set_Axis_Velocity_Acceleration.....	59
3.6.8	MT_Set_Axis_Velocity_Dec	59
3.6.9	MT_Get_Axis_Velocity_Acc.....	60
3.6.10	MT_Get_Axis_Velocity_Dec	60
3.7	Positioning Mode (Open-Loop and Closed-Loop).....	61
3.7.1	MT_Set_Axis_Mode_Position[Compatible]	61
3.7.2	MT_Set_Axis_Mode_Position_Open.....	61
3.7.3	MT_Set_Axis_Mode_Position_Close.....	61
3.7.4	MT_Get_Axis_Position_V_Max	62
3.7.5	MT_Set_Axis_Position_V_Max.....	62
3.7.6	MT_Get_Axis_Position_P_Target	63
3.7.7	MT_Set_Axis_Position_P_Target_Abs.....	63
3.7.8	MT_Set_Axis_Position_P_Target_Relative.....	64
3.7.9	MT_Set_Axis_Position_Stop.....	64
3.7.10	MT_Set_Axis_Position_V_Start	65

3.7.11	MT_Set_Axis_Position_Acc	65
3.7.12	MT_Set_Axis_Position_Dec	65
3.7.13	MT_Get_Axis_Position_Acc	66
3.7.14	MT_Get_Axis_Position_Dec	66
3.7.15	MT_Set_Axis_Position_Close_Dec_Factor	67
3.7.16	MT_Set_Encoder_Over_Enable	67
3.7.17	MT_Set_Encoder_Over_Max	67
3.7.18	MT_Set_Encoder_Over_Stable	68
3.7.19	MT_Set_Encoder_Z_Polarity	68
3.7.20	MT_Set_Encoder_Direction_Polarity	69
3.8	Software Limit	69
3.8.1	MT_Set_Axis_Software_Limit_Neg_Value	69
3.8.2	MT_Set_Axis_Software_Limit_Pos_Value	70
3.8.3	MT_Set_Axis_Software_Limit_Enable	70
3.8.4	MT_Set_Axis_Software_Limit_Disable	71
3.9	Status-Related Functions	71
3.9.1	MT_Get_Axis_Num	71
3.9.2	MT_Get_Encoder_Num	72
3.9.3	MT_Get_Axis_Mode	72
3.9.4	MT_Get_Axis_Acc[Compatible]	72
3.9.5	MT_Set_Axis_Acc[Compatible]	73
3.9.6	MT_Get_Axis_Dec[Compatible]	73
3.9.7	MT_Set_Axis_Dec[Compatible]	74
3.9.8	MT_Get_Axis_V_Now	74
3.9.9	MT_Get_Axis_P_Now	74
3.9.10	MT_Get_Axis_Software_P_Now[Compatible]	75
3.9.11	MT_Get_Axis_Software_P[Compatible]	75
3.9.12	MT_Set_Axis_P_Now	76
3.9.13	MT_Set_Axis_Software_P[Compatible]	76
3.9.14	MT_Set_Axis_Software_P_Now[Compatible]	76
3.9.15	MT_Get_Axis_Status[Compatible]	77
3.9.16	MT_Get_Axis_Status2	78
3.9.17	MT_Get_Axis_Status_Run	79
3.9.18	MT_Get_Axis_Status_Dir	79
3.9.19	MT_Get_Axis_Status_Neg	79
3.9.20	MT_Get_Axis_Status_Pos	80
3.9.21	MT_Get_Axis_Status_Zero	80
3.9.22	MT_Get_Axis_Status_Mode	81
3.9.23	MT_Get_Encoder_Position	81
3.9.24	MT_Set_Encoder_Position	82
3.9.25	MT_Set_Axis_Halt	82
3.9.26	MT_Set_Axis_Halt_All	82
3.10	Memory Operations	83

3.10.1	MT_Get_Param_Mem_Len	83
3.10.2	MT_Get_Param_Mem_Data	83
3.10.3	MT_Set_Param_Mem_Data	84
3.11	General Digital Input	84
3.11.1	MT_Get_Optic_In_Num	84
3.11.2	MT_Get_Optic_In_Single.....	85
3.11.3	MT_Get_Optic_In_All	85
3.12	General Digital Output	85
3.12.1	MT_Get_Optic_Out_Num	86
3.12.2	MT_Set_Optic_Out_Single	86
3.12.3	MT_Set_Optic_Out_All.....	86
3.13	2-8 Multi-Axis Interpolation.....	87
3.13.1	MT_Set_Axis_Line_Axis[Compatible].....	87
3.13.2	MT_Set_Axis_Line_Acc.....	87
3.13.3	MT_Set_Axis_Line_Dec	88
3.13.4	MT_Set_Axis_Line_V	88
3.13.5	MT_Set_Axis_Line_Run[Compatible].....	88
3.13.6	MT_Set_Axis_Line_Stop	89
3.13.7	MT_Set_Axis_Line_Halt.....	89
3.13.8	MT_Set_Axis_Line_Rel[Compatible].....	89
3.13.9	MT_Set_Axis_Line_Abs[Compatible]	90
3.13.10	MT_Set_Axis_Line_Run_Rel[Compatible].....	90
3.13.11	MT_Set_Axis_Line_Run_Abs[Compatible]	91
3.13.12	MT_Get_Axis_Line_Num	91
3.13.13	MT_Get_Axis_Line_Status.....	92
3.13.14	MT_Get_Axis_Line_Acc	92
3.13.15	MT_Get_Axis_Line_Dec.....	92
3.13.16	MT_Get_Axis_Line_V.....	93
3.13.17	MT_Get_Axis_Line_Axis[Compatible].....	93
3.13.18	MT_Set_Axis_Line_V_Start.....	94
3.13.19	MT_Set_Axis_Line_X_Count[Compatible]	94
3.13.20	MT_Set_Axis_Line_X_Axis[Compatible]	94
3.13.21	MT_Set_Axis_Line_X_Target_Rel[Compatible].....	95
3.13.22	MT_Set_Axis_Line_X_Target_Abs[Compatible].....	95
3.13.23	MT_Set_Axis_Line_X_Run_Rel	96
3.13.24	MT_Set_Axis_Line_X_Run_Abs	96
3.14	Arc Interpolation	97
3.14.1	MT_Set_Axis_Circle_Axis.....	97
3.14.2	MT_Set_Axis_Circle_Acc.....	98
3.14.3	MT_Set_Axis_Circle_Dec	98
3.14.4	MT_Set_Axis_Circle_V	98
3.14.5	MT_Set_Axis_Circle_Stop.....	99
3.14.6	MT_Set_Axis_Circle_Halt.....	99

3.14.7	MT_Set_Axis_Circle_R_CW_Run_Rel	99
3.14.8	MT_Set_Axis_Circle_R_CW_Run_Abs	100
3.14.9	MT_Set_Axis_Circle_R_CCW_Relative_Run	100
3.14.10	MT_Set_Axis_Circle_R_CCW_Run_Abs	101
3.14.11	MT_Get_Axis_Circle_Num	101
3.14.12	MT_Get_Axis_Circle_Status	102
3.14.13	MT_Get_Axis_Circle_Acc	102
3.14.14	MT_Get_Axis_Circle_Dec	103
3.14.15	MT_Get_Axis_Circle_V	103
3.14.16	MT_Get_Axis_Circle_Axis	103
3.14.17	MT_Set_Axis_Circle_V_Start	104
3.15	PLC Mode/Offline Mode	104
3.15.1	MT_Get_PLC_Var_Num	104
3.15.2	MT_Get_PLC_Var_Data	105
3.15.3	MT_Set_PLC_Var_Data	105
3.15.4	MT_Set_PLC_Pause	105
3.15.5	MT_Set_PLC_Stop	106
3.15.6	MT_Set_PLC_Run	106
3.15.7	MT_Get_PLC_Status	106
3.16	Stream Command Mode	107
3.16.1	MT_Get_Stream_Space	107
3.16.2	MT_Set_Stream_Pause	107
3.16.3	MT_Set_Stream_Stop	108
3.16.4	MT_Set_Stream_Run	108
3.16.5	MT_Set_Stream_Clear	108
3.16.6	MT_Set_Stream_Run_ID	109
3.16.7	MT_Get_Stream_Run_ID	109
3.16.8	MT_Set_Stream_Delay	110
3.16.9	MT_Set_Stream_Line_Acc	110
3.16.10	MT_Set_Stream_Line_Dec	110
3.16.11	MT_Set_Stream_Line_V_Max	111
3.16.12	MT_Set_Stream_Line_V_Start	111
3.16.13	MT_Set_Stream_Line_Stop	111
3.16.14	MT_Set_Stream_Line_Halt	112
3.16.15	MT_Set_Stream_Line_X_Run_Rel	112
3.16.16	MT_Set_Stream_Line_X_Run_Abs	113
3.16.17	MT_Set_Stream_Wait_Line	114
3.16.18	MT_Set_Stream_Circle_Axis	114
3.16.19	MT_Set_Stream_Circle_Acc	115
3.16.20	MT_Set_Stream_Circle_Dec	115
3.16.21	MT_Set_Stream_Circle_V_Max	115
3.16.22	MT_Set_Stream_Circle_V_Start	116
3.16.23	MT_Set_Stream_Circle_Stop	116
3.16.24	MT_Set_Stream_Circle_Halt	116

3.16.25	MT_Set_Stream_Circle_R_CW_Run_Rel.....	117
3.16.26	MT_Set_Stream_Circle_R_CW_Run_Abs.....	117
3.16.27	MT_Set_Stream_Circle_R_CCW_Relative_Run.....	118
3.16.28	MT_Set_Stream_Circle_R_CCW_Run_Abs.....	118
3.16.29	MT_Set_Stream_Wait_Circle	119
3.16.30	MT_Set_Stream_Optic_Out_Single.....	119
3.16.31	MT_Set_Stream_Optic_Out_All	119
3.16.32	MT_Set_Stream_Dec_Enable	120
3.16.33	MT_Set_Stream_Dec_Disable	120
3.17	Synchronous Trigger	121
3.17.1	MT_Set_Trigger_Enable	121
3.17.2	MT_Get_Trigger_Enable.....	122
3.17.3	MT_Set_Trigger_Start.....	122
3.17.4	MT_Get_Trigger_Start.....	122
3.17.5	MT_Set_Trigger_End	123
3.17.6	MT_Get_Trigger_End.....	123
3.17.7	MT_Set_Trigger_Step	123
3.17.8	MT_Get_Trigger_Step	124
3.17.9	MT_Set_Trigger_Width	124
3.17.10	MT_Get_Trigger_Width.....	125
3.17.11	MT_Set_Trigger_Polarity	125
3.17.12	MT_Get_Trigger_Polarity	125
3.17.13	MT_Set_Trigger_Channel	126
3.17.14	MT_Get_Trigger_Channel	126
3.18	Auxiliary Calculation [Compatible].....	127
3.18.1	MT_Help_Step_Line_Real_To_Steps.....	127
3.18.2	MT_Help_Step_Circle_Real_To_Steps	127
3.18.3	MT_Help_Step_Line_Steps_To_Real.....	128
3.18.4	MT_Help_Step_Circle_Steps_To_Real	128
3.18.5	MT_Help_Encoder_Line_Real_To_Steps.....	129
3.18.6	MT_Help_Encoder_Circle_Real_To_Steps	129
3.18.7	MT_Help_Encoder_Line_Steps_To_Real.....	130
3.18.8	MT_Help_Encoder_Circle_Steps_To_Real	130
3.18.9	MT_Help_Grating_Line_Real_To_Steps.....	131
3.18.10	MT_Help_Grating_Circle_Real_To_Steps.....	131
3.18.11	MT_Help_Grating_Line_Steps_To_Real.....	132
3.18.12	MT_Help_Grating_Circle_Steps_To_Real.....	132
3.18.13	MT_Help_Encoder_Factor	132
3.18.14	MT_Help_Grating_Line_Factor.....	133
3.18.15	MT_Help_Grating_Circle_Factor	133
4	Document Version History	134

1 Terminology, Concepts, Definitions

Through a concise, unified dynamic link library (**dll on Windows, so on Linux**) as the interface, control of our company's products is achieved, abstracting away complex underlying protocols so you can focus on application development. Currently supported development languages include C, VC, VB, C#/VB.NET, QT, Matlab, Labview, Python, Delphi, C++ Builder, GCC, Eclipse,

CodeBlocks, Lazarus, and other mainstream languages and development tools, facilitating secondary development and integration into your software systems.

Additionally, we provide ASCII string commands (via serial and network ports; USB port not supported), enabling secondary development without dynamic link libraries. Support is also extended to various embedded systems such as microcontrollers (MCUs) FPGAs, Raspberry Pi, and single-board computers as controllers.

For ASCII string commands, refer to the separate command documentation. The development approach is similar to this manual, differing only in the interface. Please select based on your requirements.

1.1 Conventions

DLL functions follow a unified, systematic, and WYSIWYG naming convention for easy mastery.

Important Note: Functions and controls operate asynchronously. Functions execute **immediately without waiting for actions to complete**, returning promptly. We provide comprehensive status information; you must use relevant read functions to retrieve status and determine execution results or the current object state. Unlike synchronous methods, you need not worry about process deadlocks or similar issues.

1.1.1 Action/Parameter Settings

All configuration parameters and action execution functions follow the format:

int32_t MT_Set_xxxx(yyyy)

The return value is a 32-bit signed integer. **0** indicates successful communication and receipt of the controller's response (note: execution may not be complete) **A non-zero value** (e.g., -1) indicates communication failure. The parameters enclosed in parentheses represent a set of configuration parameters. The number of parameters varies depending on the function, but the format remains consistent.

For example:

MT_Set_Axis_Position_P_Target_Abs(0,3000);

Controls the first axis (index 0) to

position (3000) in the coordinate system.

MT_Set_Axis_Velocity_Stop(1);

Controls the second axis to decelerate and stop

1.1.2 Status/Parameter Reading

All functions for reading parameters or status follow this format:

int32_t MT_Get_xxxx(yyyy)

Returns a 32-bit signed integer. **0** indicates successful communication and a valid return value. **Non-zero** (e.g., **-1**) indicates communication failure, with the return value being an invalid random number. Values within parentheses are returned via pointer or reference.

Example:

MT_Get_Axis_Status_Run(0, pRun);

Reads the current motion status of the first axis. ***pRun == 1** indicates motion in progress; ***pRun == 0** indicates motion has stopped.

MT_Get_Axis_Software_P(2, pPos);

Reads the current open-loop motion position of the third axis. ***pPos** indicates the current coordinate system position of the third axis, readable both during motion and when stopped.

1.1.3 Object ID

Axis numbers for control objects start at **0**. For example, the first axis corresponds to axis number **0**, the second axis to axis number **1**, and so on.

The device number for the first controller is **0**, and the

device number for the second controller is **1**. This pattern

also applies when developing interface functions for multi-

controller systems.

1.1.4 Multi-Controller Device Secondary Development

If you need to control multiple independent controller devices, simply modify the functions in the subsequent sections of this document from

MT_xxx_xxx(axis_id...) to ***MT_M_xxx_xxx(device_id, axis_id...)***

For example:

MT_Get_Axis_Status_Run(0, pRun);

Reads the current motion status of the first axis. *pRun==1 indicates motion in progress, while *pRun==0 indicates stopped. This is the default behavior when only one controller is present.

MT_M_Get_Axis_Status_Run(2, 0, pRun);

Reads the open-loop position of the first axis on the third controller.

The first parameter of **MT_M** is fixed as the device serial number.

Note:

- The remainder of this document describes scenarios involving a single device only. For multiple devices, simply modify the code according to the established pattern.
- For multiple devices combined via hardware or software combiners, they are treated as **a single device** for secondary development.

1.1.5 Function Naming

- Generally, function names clearly express their functionality.
- To aid understanding or prevent misunderstanding, some functions with identical functionality may have multiple names. Choose the one easiest for you to remember.
- **Functions marked as "compatibility" are not recommended for new software development. These are functions maintained for backward compatibility with older interfaces and may be deprecated with hardware upgrades. Use the newly named functions instead.**

1.1.6 Function Call Duration

Depends on the communication port and hardware configuration. Typical call times (for reference only; actual times depend on hardware configuration and are influenced by computer and network conditions):

Serial Number	Interface Type	Fastest Speed per Call	Notes
1	Standard USB	Approx. 10ms	Compatible with most computers
2	Custom USB	Approx. 2 or 4ms	Depends on the computer's USB port
3	RS232 Serial Port	Approximately 10ms	
4	Network module type	Approx. 30ms	
5	Network integrated	Approx. 2ms	Direct connection

1.1.7 Compatibility Functions

Functions marked with "[Compatibility]" are provided for backward compatibility with older software versions. These functions **may** be **deprecated or unsupported** in future software updates. Alternative functions or more convenient replacements are available, such as those with appended numeric suffixes like 2, 3, etc.

Newly developed software should not use functions marked with "[Compatibility]".

1.1.8 Parameter Memory Storage

The controller **only retains coordinate position information**. Other parameters set via functions **will automatically revert to default values after power loss**. All parameters must be reconfigured before use.

1.2 Measurement Units and Conversion

All measurement units used by functions are based on the smallest unit of motor movement: the pulse (step) Position is measured in steps, speed in steps per second (Hz/s) and acceleration in steps per second squared (Hz/s²)

A proportional conversion factor is required to relate motor steps to actual physical quantities.

1.2.1 Conversion Principle

Control systems, motors, and drivers are universally designed based on microsteps. The control system only recognizes step counts. When driving different mechanical structures, the corresponding physical quantities of the actual motion differ and require conversion. This conversion is linear. Once the mechanical structure and motor drive are fixed, this conversion becomes a fixed proportional coefficient. The conversion fact

mechanical structures, the corresponding physical quantities of actual motion differ and require conversion. This conversion is linear. Once the mechanical structure and motor drive are determined, this conversion becomes a fixed proportional coefficient. Acceleration, deceleration, and velocity are simply calculated by multiplying or dividing by this coefficient.

1.2.2 Coefficient Definitions

Unit Physical Quantity: Commonly 1 mm for translation stages, 1° for rotation stages
Open-Loop Pulse Count: Motor steps corresponding to the unit physical quantity

Closed-loop pulses: Counts from encoders/optical scales/magnetic scales per unit physical unit

Physical units may also be selected from other options (e.g., 1 mrad, 1 cm) with the pulse count adjusted accordingly.

Tip: Setting all three parameters as integers—rather than using a single proportional coefficient that may involve decimals—prevents cumulative calculation errors from non-integer divisibility. If divisibility is possible, a single proportional coefficient may suffice for conversion.

Open-loop	proportional
coefficient:	Open-loop pulses/unit
physical quantity	Closed-loop
proportional coefficient:	Closed-

loop pulses/unit physical quantity
Closed-loop deceleration
coefficient: Open-loop
pulses/closed-loop pulses

1.2.3 Software sends commands to the controller

Round to nearest integer (physical quantity * coefficient) => Controller

1.2.4 Software reads controller position and velocity

Controller value / Coefficient => Physical quantity

1.2.5 Example

- Example 1

The pitch of the translation stage is **1mm**, corresponding to a motor driver microstepping of **32** (requiring $32 \times 200 = 6400$ pulses per motor revolution). Without a sensor connected

, the unit physical quantity is 1, the open-loop pulse count is 6400, and the proportional coefficient is 6400

In positioning mode, if acceleration is set to 2mm/s, deceleration to 2mm/s, and velocity to 5mm/s, moving to 2.5mm

C/C++ code example:

```
// Open-Loop positioning
MT_Set_Axis_Mode_Position_Open(0);

// Motion parameters are consistent
for both open-loop and closed-loop
MT_Set_Axis_Position_Acc(0, 2 * 6400);
MT_Set_Axis_Position_Dec(0, 2 * 6400);
// Positioning movement at 5mm/s speed
MT_Set_Axis_Position_V_Max(0, 5 * 6400);

// Absolute positioning
MT_Set_Axis_Position_P_Target_Abs(0, 2.5 * 6400)
```

Read current position and velocity:

```
int iV, iP;
double fV, fP;

// Current open-loop position
MT_Get_Axis_Software_P_Now(0, &iP);
// Current velocity
(common for open-
Loop/closed-loop)
MT_Get_Axis_V_Now(0, &iV);
// Convert to physical units,
scaling factor 6400 fV = iV /
6400;
fP = iP / 6400;
```

● Example 2

The turntable reducer has a gear ratio of 180, corresponding to a motor drive microstepping resolution of 32 (requiring $32 \times 200 = 6400$ pulses per motor revolution) A circular optical encoder (with 16384 lines, not divisible by 360) is mounted on the table surface.

Thus, the physical unit 360 (one full rotation of the stage)

Open-loop pulses: $6400 \times 180 = 1,152,000$ (The motor must rotate 180 revolutions through the reducer for the table to complete one revolution)

Closed-loop pulses: $16384 \times 4 = 65536$ (The controller circuit automatically quadruples the count for each full rotation of the encoder to enhance resolution)

Open-loop coefficient

= $1,152,000 / 360$

Closed-loop

coefficient = $65,536 /$

360

Since these coefficients are not exact divisors, do not directly substitute them into calculations. Instead, perform calculations using physical quantities, open-loop pulse count, and closed-loop pulse count.

In positioning mode, if acceleration is set to 2 mm/s, deceleration to 2 mm/s, and velocity to 5 mm/s, the target position is 2.5 mm.

C/C++ code example for open-loop mode:

```
// Open-loop positioning
MT_Set_Axis_Mode_Position_Open(0);

// Motion parameters are consistent for both
open-loop and closed-loop
MT_Set_Axis_Position_Acc(0, 2 * 1152000 / 360);
MT_Set_Axis_Position_Dec(0, 2 * 1152000 / 360);
// Positioning movement at 5mm/s
MT_Set_Axis_Position_V_Max(0, 5 * 1152000 / 360);

//Absolute positioning
MT_Set_Axis_Position_P_Target_Abs(0, 2.5 * 1152000 / 360);
```

Read current position and velocity:

```
int iV, iP;
double fV, fP;

//Current open-loop position
MT_Get_Axis_Software_P_Now(0, &iP);
// Current velocity
MT_Get_Axis_V_Now(0, &iV);

// Convert to physical units
fV = iV / (1152000 / 360);
fP = iP / (1152000 / 360);
```

Closed-loop mode C/C++ code example:

```
// Enable closed-loop operation when
using a linear encoder
MT_Set_Axis_Mode_Position_Close(0);
// Set the closed-loop deceleration factor to enhance automatic closed-loop control performance
// Deceleration factor = open-loop pulses / closed-loop pulses
MT_Set_Axis_Position_Close_Dec_Factor(0, 1152000 / 65536);
// Motion parameters are consistent for both
open-loop and closed-loop
MT_Set_Axis_Position_Acc(0, 2 * 1152000 / 360);
MT_Set_Axis_Position_Dec(0, 2 * 1152000 / 360);
// 5mm/s velocity positioning movement
MT_Set_Axis_Position_V_Max(0, 5 * 1152000 / 360);
```

//Absolute positioning (differs from open-loop operation)

*MT_Set_Axis_Position_P_Target_Abs(0, 2.5 * 65536 / 360);*

Read current position and velocity:

```
int iV, iP, iP_Close;
double fV, fP, fP_Close;

// Current open-loop position
MT_Get_Axis_Software_P_Now(0, &iP);
// Current closed-loop position
MT_Get_Encoder_Pos(0, &iP_Close);

// Current speed
MT_Get_Axis_V_Now(0, &iV);
//Convert to physical units
fV = iV / (1152000 / 360);
fP = iP / (1152000 / 360);

fP_Close = iP_Close / (65536 / 360);
```

- Example 3

The turntable reducer has a gear ratio of 180, corresponding to a motor driver microstepping of 20 (requiring $20 \times 200 = 4000$ pulses per motor revolution) An encoder (400 lines) is mounted on the motor shaft.

Thus, the open-loop pulse count for the physical unit of 360 (one full rotation of the platform) is $4000 \times 180 = 720000$,

Closed-loop pulses: $400 \times 180 \times 4 = 288,000$ (the controller circuit quadruples the grating line count to enhance resolution)

Another configuration method:

Physical unit: 2 (one full rotation of the motor) Open-loop pulses: 4000 (one full rotation of the motor)

Closed-loop pulse count: $400 \times 4 = 1600$ (The controller circuit performs quadruple frequency counting on the grating lines to enhance resolution)

- Example 4

The translation stage pitch is 1mm, corresponding to a servo motor electronic ratio of 10000. Without a sensor connected, the unit physical quantity is 1,

Open-loop pulses: 10000

1.2.6 Acceleration/Deceleration

The acceleration/deceleration rate from stop to target speed. Higher values result in faster acceleration and deceleration; lower values yield slower acceleration and deceleration. Units are mm/s^2 for translation stages and $^\circ/\text{s}^2$ for rotation stages.

1.2.7 Stepper Microstepping

Stepper drivers marked as 1, 2, 4, 8, 16, 32, 64, 128, 256, 5, 10, 20, 50, 100, 200, etc., indicate microstepping. For a motor operating in full-step mode, the pulses per revolution correspond to **Microstepping Value** $\times$ 200 (2-phase stepper full-step count is 200).

Another type is labeled as microsteps. The number of microsteps per revolution is the value obtained by multiplying the microstep value by 200 (e.g., 200, 400, 800, 1600, 3200, etc.). In this case, the value has already been multiplied by 200, so do not multiply it again. When entering the microstep value in the Motion Control Assistant, divide it by 200.

1.2.8 Servo Electronic Ratio

The servo's electronic ratio represents the number of steps per full rotation of the motor. When entering the microstepping value in the Motion Control Assistant, divide this value by 200.

1.2.9 Conversion Functions

A set of auxiliary calculation functions (see Section 3.20 of this document) is provided to assist users in converting between physical units and pulse units when using translation stages, rotation stages, encoders, and linear scales.

We recommend developing using the approach demonstrated in this section for more concise code.

1.3 Coordinate Systems

This manual defines two types of coordinate systems: the physical coordinate system and the logical coordinate system. By default, these two coordinate systems are aligned.

1.3.1 Coordinate System Definition

- **Physical Coordinate System**
The physical coordinate system refers to the coordinate system marked on our machinery, such as the scale lines on the machine table, or travel (0-100mm), angle (0-30°), etc. The values in this coordinate system are generally fixed.
- **Logical Coordinate System**
The logical coordinate system is the system used by the controller for operation. This system may differ from the physical coordinate system and can be **adjusted** via **offset** functions. All parameters and example values within functions are based on this logical coordinate system.
- **Open-Loop Logical Coordinate System and Closed-Loop Logical Coordinate System**
Controllers operating in open-loop mode utilize an **open-loop logical**

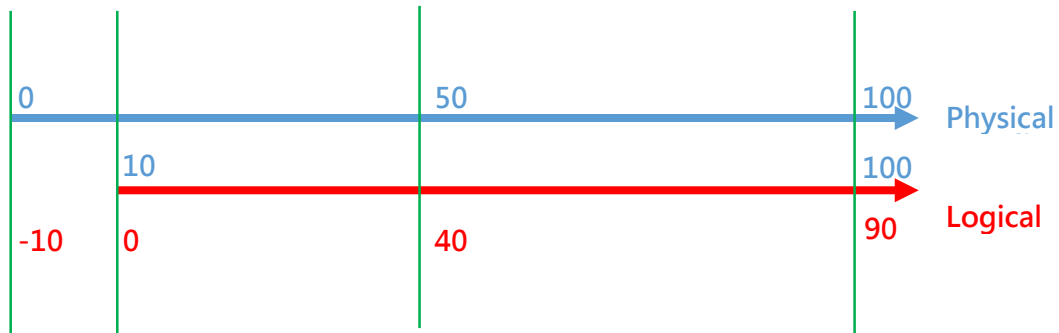
coordinate system, while those in closed-loop mode employ a **closed-loop logical coordinate system**. These two coordinate systems are independent and can be freely adjusted.

Operating in

1.3.2 Logic Coordinate Memory and Calibration

The controller features logical coordinate memory. Upon power loss, it preserves the current logical coordinate systems (separately for open-loop and closed-loop) These are automatically restored at the next power-up. If the machinery remains stationary after power loss, coordinate recalibration can be performed at extended intervals. However, if the machinery has been moved—either manually or due to gravitational forces—recalibration is required after each power-up.

1.3.3 Coordinate System Adjustment



In practical applications, it may be necessary to adjust the logical coordinate system for convenience. The example above illustrates how to modify the logical coordinate system.

Blue represents physical coordinates, while red denotes logical coordinates. For convenience, we wish to offset the logical coordinate system by **10** units.

Method 1: Set the position in the logical coordinate system to **0** where the physical coordinate system is at **10**. Method 2:

set the logical coordinate position to **-10** at the physical coordinate position **0**. Function for adjusting the coordinate system:

```
MT_Set_Axis_Software_P(axis_number,  
target_logical_coordinate_position);           // Open-  
loop MT_Set_Encoder_Pos(axis_index,  
target_logical_coordinate_position);           //  
Closed-loop
```

After coordinate system adjustment, the position read via the read function at the physical coordinate **50** is **40**. All subsequent actions are calculated and executed based on the new logical coordinate system.

1.4 Motion Control Concepts

1.4.1 Types of Motion

Common mechanical motions can be simply decomposed into linear and rotational movements. Combining these two produces various motion forms. For example:

- Two perpendicular linear motions:
 - Fixed proportional velocity yields planar linear motion.
 - Variable proportional velocity yields two-dimensional curved motion.
- Three mutually perpendicular linear motions:
 - Fixed velocity ratio yields spatial linear motion.

– Variable velocity ratio yields spatial curved motion. The combination of a linear motion and a rotational motion results in a helical motion.

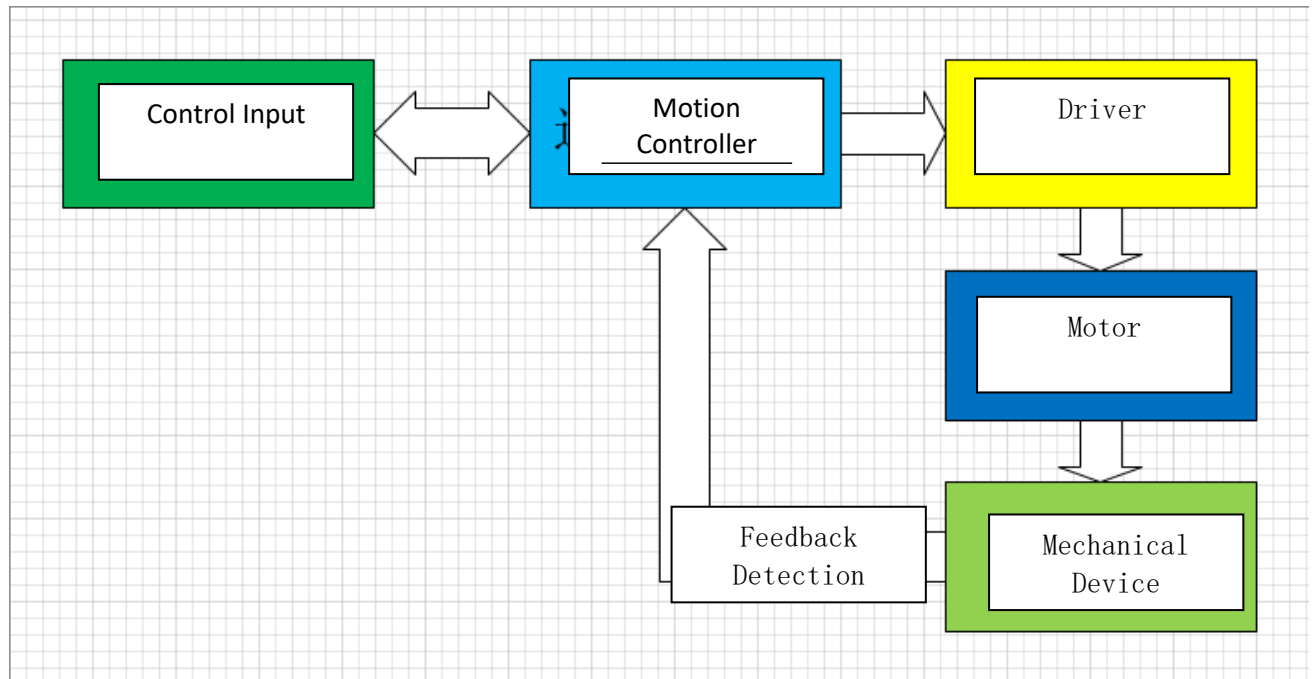
For motion control cards, the function is to drive motor rotation, which is then converted into other forms of motion by mechanical structures. The smallest rotational unit of a motor is one **step**, which translates into displacement or rotation in the mechanical system. Typically, this conversion ratio is fixed, determined jointly by the mechanical design and the motor driver.

Since control systems must adapt to diverse mechanical structures with varying parameters, secondary development begins by establishing this ratio coefficient. Subsequent development focuses solely on the motor's rotational step count.

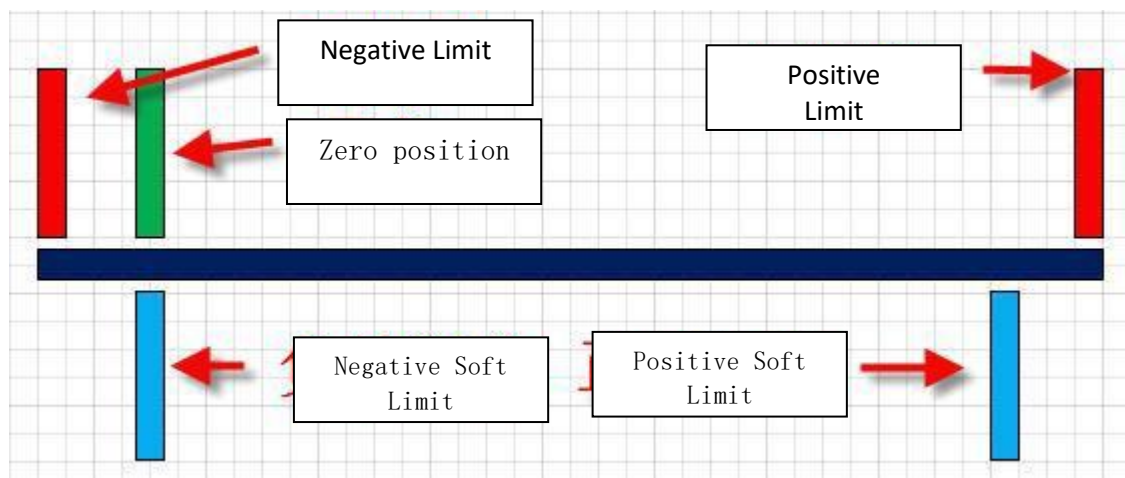
1.4.2 Components of a Motion Control System

A typical motion system consists of: Control Input (computer, HMI, MCU, switches, cameras, etc.) + Motion Controller (card) + Motor

drive (power amplifier) + **motor** + mechanical assembly + feedback detection (limit switches, zero position switches, proximity switches, encoders, linear encoders).



1.4.3 Limit Switches, Zero Position Switches, and Software Limits



Negative limit, positive limit, and zero position are typically implemented in hardware using mechanical switches, photoelectric switches, proximity switches, etc.

Negative and positive limits refer to the physical maximum positions. When the system reaches these limits, the controller automatically triggers an emergency stop to protect the mechanical device. Both limits are not necessarily required; they can be added based on the direction requiring protection.

Zero position represents the mechanical coordinate origin, serving as an absolute reference point for coordinate calibration.

Software limits are implemented via controller software to manually define a movable range within the current logical coordinate system for protection. **Software limits are typically used after zero-point calibration and are generally smaller than the physically movable range.**

1.4.4 Acceleration and Deceleration

Most mechanical devices possess significant inertia. To achieve better control, acceleration during startup and deceleration during stopping are essential. This improves control precision, reduces inertia-induced damage to workpieces, and protects the electrical control system. Stepper systems, in particular, can provide greater starting torque through low-speed initiation.

1.4.5 Stepper Microstepping

Stepper motors typically advance one step angle per full step. For a standard 2-phase 4-wire stepper motor, the step angle is 1.8° , meaning 200 pulses per revolution. The driver can electrically subdivide each full step, delivering the required current incrementally rather than all at once. This enables subdivided motion, enhancing precision and smoothness.

1.4.6 Lead Screw Pitch

The motor converts rotational motion into linear motion of the stage on the lead screw. The lead screw pitch determines the linear distance traveled per motor revolution.

1.4.7 Gear Ratio/Reduction Ratio

Mechanically, precision and torque can be enhanced through gearboxes or reduction gears. The reduction ratio/transmission ratio is defined as the number of mechanical rotations produced by the motor after reduction, corresponding to the number of motor rotations.

1.4.8 Encoder

Based on detection principles, encoders can be classified as optical, magnetic, inductive, or capacitive. According to their scaling methods and signal output formats, they are categorized into incremental, absolute, and hybrid types.

Incremental encoders measure rotational angle and output data via pulses. The critical parameter is line count—the number of pulses generated per full rotation.

The **controller quadruples** the line count of encoders with long-line outputs. For example, a 1000-line encoder rotating one full revolution results in 4000 counts on the controller. Note whether the encoder parameter specifies line count (1000 in this example) or resolution (4000 in this example).

1.4.9 Glass Scale

A linear encoder measures displacement using optical methods. When mounted linearly, it measures linear displacement; when mounted circularly, it measures angular

displacement. The key parameter is the minimum scale value, which represents the displacement corresponding to a single output pulse.

The controller performs quadruple frequency conversion on the line count of the linear encoder for long-range output. For example, when a linear encoder marked with 10,000 lines completes one full travel cycle, the controller counts 40,000 pulses. Note that the encoder parameter specification indicates either the line count (10,000 in this example) or the resolution (40,000 in this example).

1.4.10 Open-Loop Control

Open-loop control refers to a system that does not monitor the state of the controlled object or perform feedback adjustments. It assumes the target moves according to the preset goal.

1.4.11 Closed-loop control

Closed-loop control involves monitoring the state of the controlled object and adjusting control based on feedback from the object to ultimately achieve the set target. Closed-loop control requires additional monitoring sensors, such as encoders or linear encoders, to perform feedback control based on sensor status.

1.4.12 Relative and Absolute

Relative refers to offsets relative to the state at the instant the command is executed. Examples include offsets adjusted relative to the current speed or relative to the current position. Absolute, however, refers to the desired position in the logical coordinate system, independent of the current speed or position.

Relative is an offset concept.

1.5 Factors Affecting System Accuracy

The overall accuracy of a system is determined by multiple factors, with the system's accuracy being limited by its weakest link. The primary factors include:

1.5.1 Mechanical Accuracy

This is the most fundamental factor, setting the upper limit for system accuracy.

1.5.2 Motor/Driver Accuracy

- Stepper Motors

The full step of a two-phase stepper motor is 1.8° , and full-step accuracy forms the foundation.

The full step can be subdivided through the driver's microstepping capability, where uniformity of subdivision is crucial. Generally, microstepping error in stepper motors does not accumulate, resulting in high open-loop accuracy.

As the speed of a stepper motor increases, its load capacity decreases. When the load exceeds the motor's capacity, it may lose steps (slippage)

- Servo Motor

Closed-loop control is achieved via an encoder mounted on the motor. Encoder accuracy is fundamental, and feedback control capability is critical.

1.5.3 Controller Pulse Output

If the controller calculates an incorrect number of pulses or outputs a poor waveform, system accuracy will be compromised. Under normal circumstances, these parameters are typically guaranteed. Rapid verification can be achieved using an oscilloscope and logic analyzer.

1.6 Functional Modes

1.6.1 Open-Loop Zero-Point Search Mode

The coordinate system of the control system requires calibration during use. Calibration typically involves using a physical position as a coordinate reference point to adjust the logical coordinate system.

Typically, mechanical limit switches (negative, positive, or zero position switches) mounted on the structure serve as reference points, or their positions are directly designated as the origin of the logical coordinate system.

- Standard Zero-Point Search Mode

MT_Set_Axis_Mode_Home_Home_Switch *//(Same-name function*

MT_Set_Axis_Mode_Home)

MT_Set_Axis_Home_V(axis_number, home_speed) *//Initiate homing*

Moves in the positive or negative direction at the user-defined speed. When encountering a zero-position switch or limit switch, stops movement and sets the current position as **the origin** (coordinate = 0) of the motion controller's **logical coordinate system**.

This mode is recommended in the following situations:

- (1) Linear stages without zero switches installed, where the negative limit switch (or positive limit switch for axes with negative coordinate ranges) is used as the zero position.
- (2) Rotary stages equipped with only a zero switch and no limit switches
- (3) Rotary stages equipped with limit switches but without zero switches

If the zero position mode speed is set to positive, the motor moves forward and stops upon reaching the zero position or positive limit, establishing it as the origin (0 point) of the logical coordinate system. If the zero position mode speed is set to negative, the motor moves backward and stops upon reaching the zero position or negative limit, establishing it as the origin (0 point) of the logical coordinate system.

A slower zero-position search speed yields higher precision in locating the zero position. Select an appropriate speed that avoids overshoot (this depends on factors such as the signal width of the zero switch)

In special cases (not recommended, as controllers with position memory do not require frequent zero-point searches for coordinate system calibration), if rapid zero-point detection is needed, perform a coarse zero-point search (with overshoot) followed by a fine zero-point search, as shown below:



- Zero-Priority Search Mode

MT_Set_Axis_Mode_Home_Home_Zero**MT_Set_Axis_Home_V(axis_number, zero_speed) //Initiate zeroing**

Move in the positive or negative direction at the user-set speed. When the first switch encountered is the zero position switch, stop movement and set the current position as **the origin of the** motion controller's **logical coordinate system** (coordinate = 0). If the first switch encountered is the limit switch, **automatically reverse direction** and move once more. When the zero position switch or limit switch is encountered, stop movement and set the current position as **the origin of the** motion controller's **logical coordinate system** (coordinate = 0).

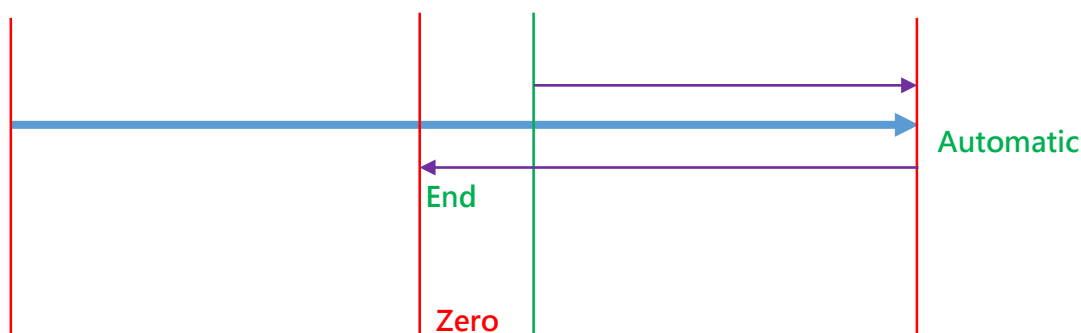
If the zero-position mode speed is set positive, the motor moves forward. Upon first encountering the zero-position switch, it stops and sets the current position as the origin (0 point) of the logical coordinate system. If the first switch encountered is the positive limit switch, it immediately reverses direction. Upon encountering either the zero-position switch or the negative limit switch, it stops and sets the current position as the origin (0 point) of the logical coordinate system.

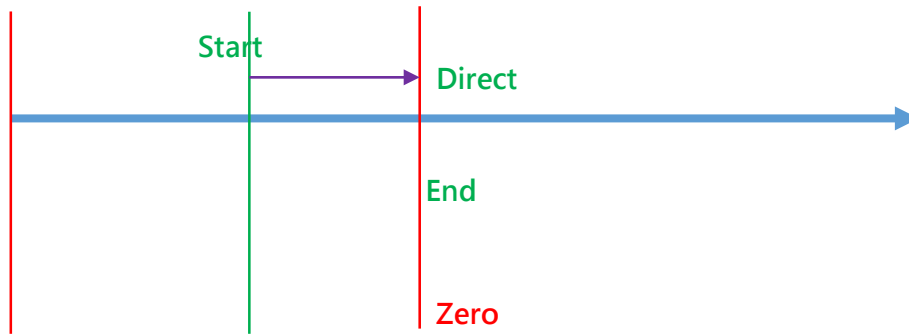
If the zero position mode speed is set to negative, the motor moves in the negative direction. Upon first encountering the zero position switch, it stops and sets the logical coordinate system origin (0 point). If the first encounter is the negative limit switch, it immediately reverses direction, stops upon encountering either the zero position or positive limit switch, and sets the logical coordinate system origin (0 point).

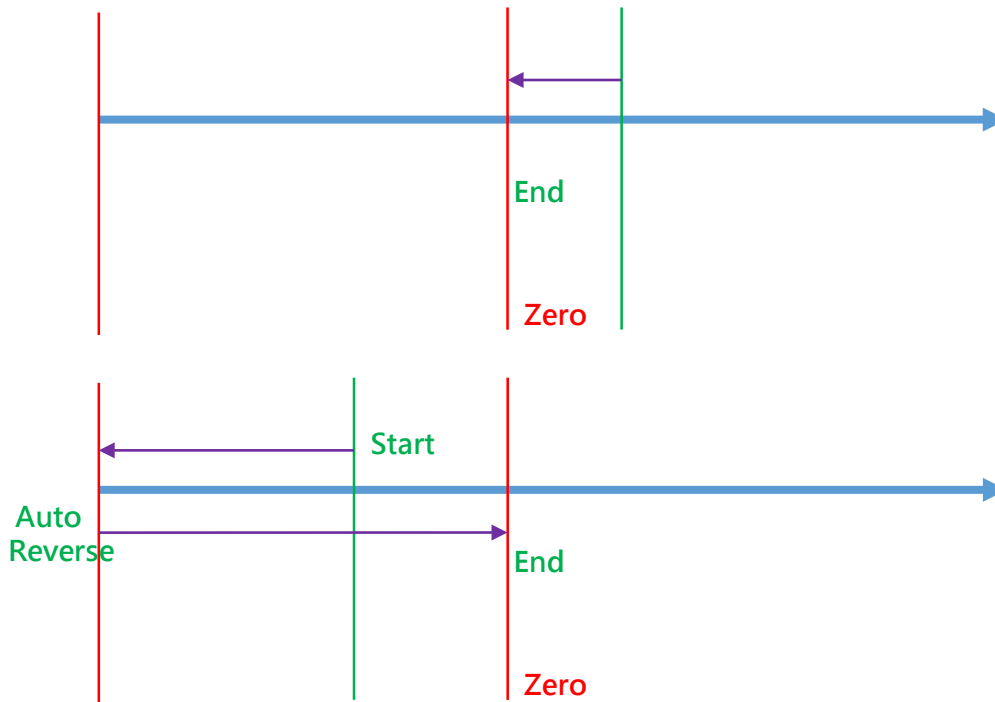
This mode is recommended in the following situations:

- (1) Linear stages equipped with negative limit switches, positive limit switches, and zero position switches
- (2) Rotary stages equipped with negative limit switches, positive limit switches, and zero switches, where 360° rotation is not possible

Using a linear stage as an example, the diagram illustrates various possible scenarios. The situation for rotary stages is identical, equivalent to converting spherical coordinates into linear coordinates.







Note: Zero position can also be searched using Positioning Mode or Constant Speed Mode. In Positioning Mode and Constant Speed Mode, the system stops upon encountering a limit switch but does not automatically adjust the coordinate system. Manual adjustment via the adjustment function is required.

In zero-position mode, once the stop condition is met, the current position is automatically adjusted to 0.

In zero-position mode, commands from Constant Speed Mode, Positioning Mode, Multi-Axis Interpolation Mode, and Arc Interpolation Mode may cause conflicts and will be discarded without execution.

Always switch the operating mode before calling functions related to that mode.

1.6.2 Closed-loop zero-position search mode

Closed-loop zero mode is similar to open-loop zero mode, except that the signal determining zero position stop is provided by sensors, including linear encoders and rotary encoders. Stopping is typically achieved by detecting the Z-axis signal or the zero switch signal from the sensor.

In zero mode, commands issued during constant speed mode, positioning mode, multi-axis interpolation mode, or circular interpolation mode may conflict and will be discarded without

execution.

Always switch the operating mode before calling functions related to that mode.

1.6.3 Constant Speed Mode

Specifies the axis to operate at a designated speed. Speed adjustments can be made at any time during this mode. Acceleration and deceleration occur at the specified acceleration rate.

When the speed is positive, the motor rotates in the positive direction; when the speed is negative, it rotates in the negative direction.

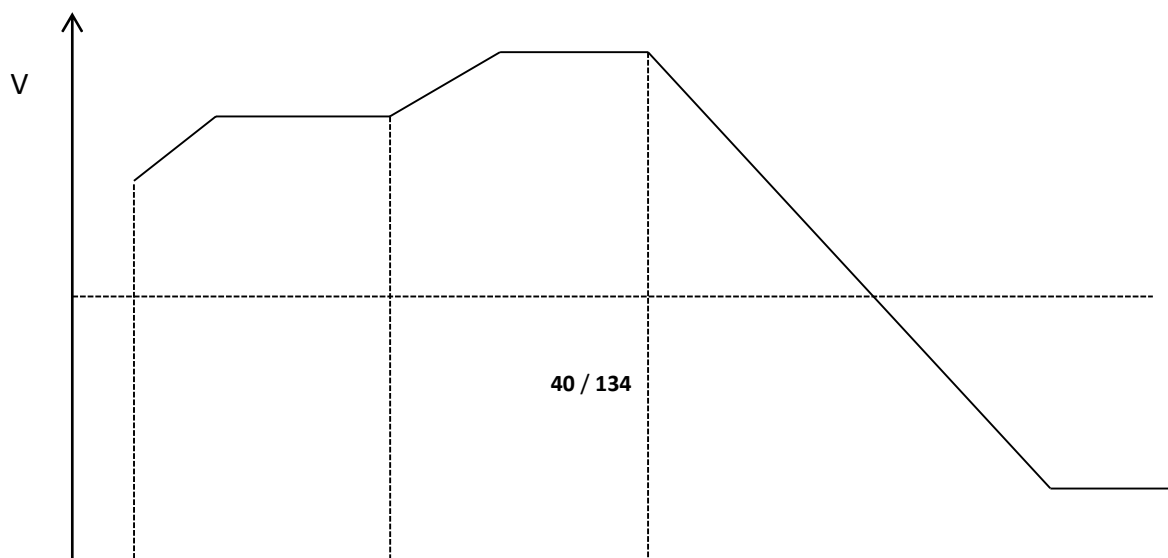
When reversing direction, decelerate first, then accelerate to the specified reverse speed.

During forward motion, if the forward limit switch is active, the motor stops immediately.

During reverse motion, if the reverse limit switch is active, the motor stops immediately.

The minimum speed in constant speed mode is $\pm 50 \text{ Hz/s}$ (adjustable via function in special cases, not recommended).

The following diagram illustrates the workflow for setting three new speeds. Users can set new operating speeds at any time, with the controller automatically performing acceleration/deceleration. For reverse directions, it automatically executes deceleration followed by acceleration. **V1, V2, V3** represent new target speeds.



In constant speed mode, commands issued in zero-point search mode, positioning mode, multi-axis interpolation mode, or circular interpolation mode may conflict and will be discarded without execution.

Ensure the operating mode is changed before calling functions associated with that mode.

1.6.4 Open-Loop Positioning Mode (Open-Loop Positioning Mode)

The specified axis moves according to the designated number of steps and stops upon reaching the target position. Acceleration and deceleration occur during movement. If a constant speed phase is present, the axis moves at the set maximum speed.

If the specified position is in the positive direction relative to the current position, the motor moves in the positive direction. **If the motor is currently moving in the negative direction, it decelerates in the negative direction before accelerating to the positive direction.**

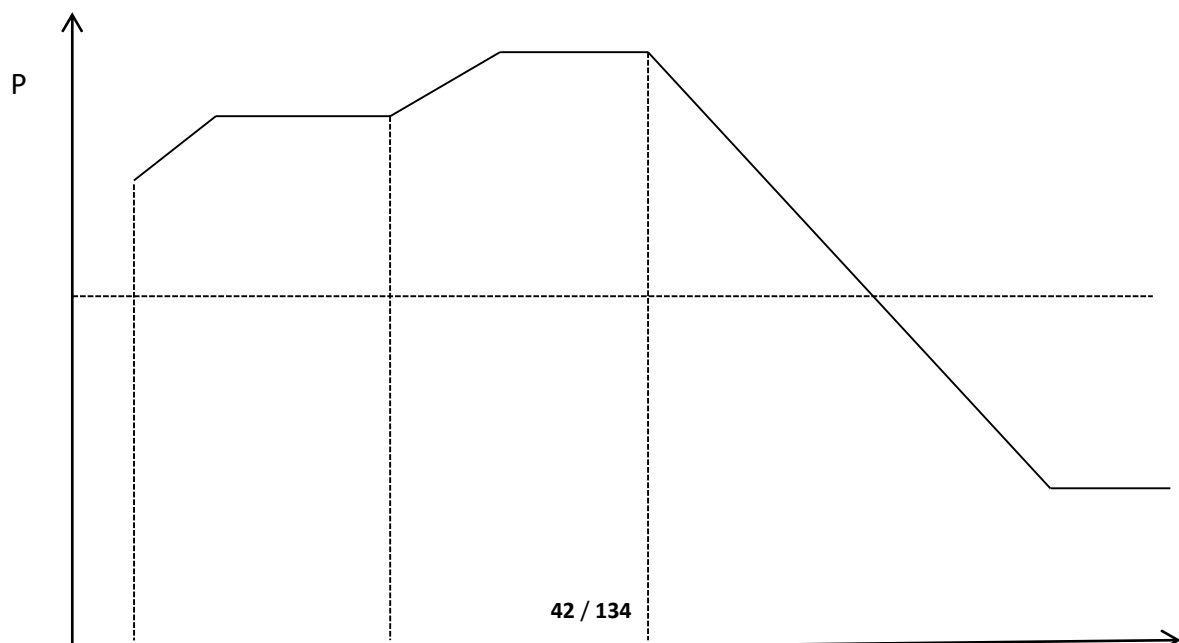
If the specified position is in the negative direction relative to the current position, the motor moves in the negative direction. **If the motor is currently moving in the positive direction, it decelerates in the positive direction before accelerating to the negative direction.**

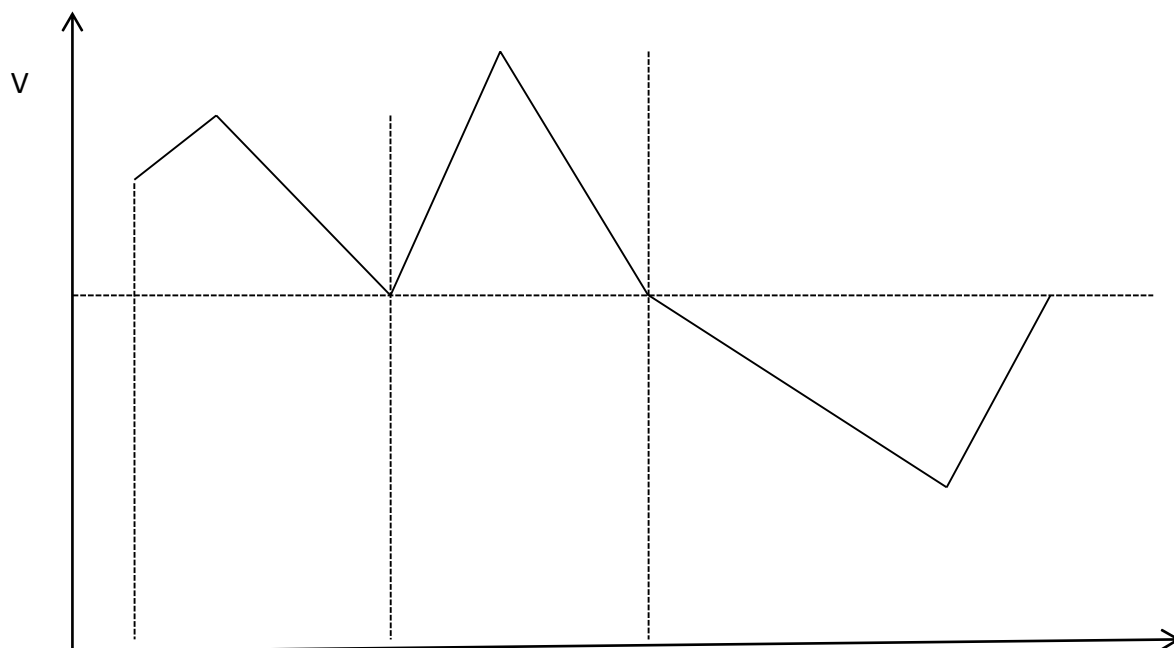
If the specified position is the same as the current position, the motor remains stationary.

During forward motion, if the forward limit switch is active, the motor stops immediately.

During reverse motion, if the reverse limit switch is active, the motor stops immediately.

The diagram below illustrates the speed magnitude, direction, and position target during positioning movement: When the position remains unchanged, it indicates a speed of 0, meaning the movement has stopped. **P1**, **P2**, and **P3** represent the set target positions.





In positioning mode, commands issued in zero-point search mode, constant-speed mode, multi-axis interpolation mode, or circular interpolation mode may be discarded and not executed due to potential conflicts.

Ensure the operating mode is changed before calling functions associated with that mode.

1.6.5 Closed-Loop Positioning Mode (Closed-Loop Positioning Mode)

Closed-loop positioning mode is similar to open-loop positioning mode, except that position determination relies on sensor counts rather than pulse counts.

In positioning mode, commands in zero-point search mode, constant speed mode, multi-axis interpolation mode, and circular interpolation mode may cause conflicts and will be discarded without execution.

Always switch the operating mode before calling functions related to that mode.

1.6.6 Multi-Axis Interpolation Mode

Interpolation refers to the simultaneous start (synchronization time $< 10 \mu s$) and stop (synchronization time $< 10 \mu s$) of multiple axes, suitable for applications requiring strict timing coordination among multiple axes. Operation is similar to Positioning Mode, except multiple specified axes must simultaneously move to designated target positions. The motion controller automatically determines movement

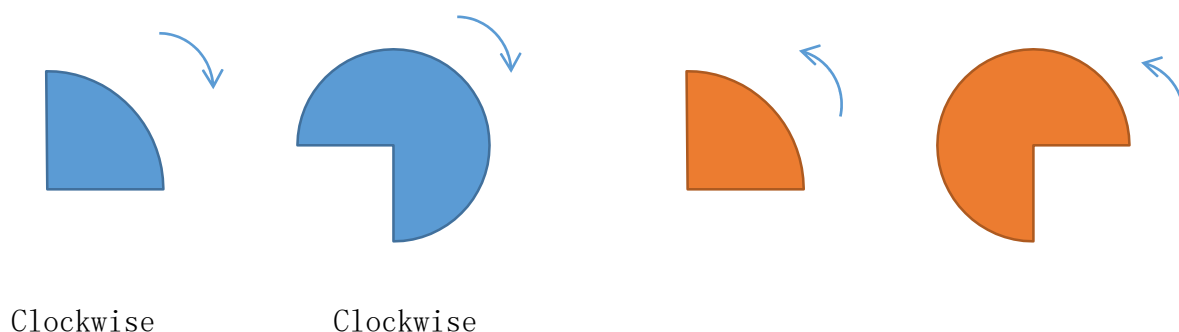
direction. The set speed is the fastest axis's movement speed, with the fastest axis automatically identified by the controller. If any axis encounters a limit switch, the current interpolation movement is immediately terminated. Any combination of axes can participate in interpolation, with a minimum of 2 axes and support for all axes of the motion controller simultaneously.

Note: Unlike positioning mode, new target values cannot be set during motion in linked interpolation mode. Operations must either stop

or wait until the current interpolation movement is complete before initiating the next one.

1.6.7 Circular Interpolation Mode

Arc interpolation mode involves two axes performing interpolation along an arc path. Any two axes of the motion controller can be designated. This mode requires specifying the target coordinate point and interpolation radius. Interpolation halts immediately if any axis encounters a limit switch. For a given target position and radius, arc interpolation divides into **four** scenarios: clockwise inferior arc, clockwise superior arc, counterclockwise inferior arc, and counterclockwise superior arc.



Note: In arc interpolation mode, new series target values cannot be set during motion. Only stop or wait for the current interpolation to complete before initiating the next interpolation.

1.6.8 Stream Command Mode

Stream Command Mode refers to an operational mode where commands can be sent continuously for execution by the motion controller, suitable for continuous actions requiring rapid response. This mode is equivalent to the **FIFO** command mode of other motion controllers (cards), but surpasses typical

FIFO command mode. While standard **FIFO** modes on other motion controllers (cards) typically support only sequential positioning functions (e.g., continuous linear interpolation), Stream Command Mode additionally supports continuous positioning commands (multi-axis interpolation, circular arc interpolation, single-axis positioning, etc.) along with general-purpose **IO** output and delay functions (**1ms** real-time precision)

In stream command mode, the motion controller allocates a command buffer. Commands prefixed with "**Stream**" enter this buffer for continuous execution by the controller. The application only needs to check the buffer's available space. If space

is available, commands can be sent continuously, significantly reducing the burden on the user application. This is particularly advantageous in scenarios requiring rapid response or

requires 1ms precision delay (**which cannot be achieved by the application running on Windows**)

This mode is generally not recommended for standard applications.

Each stream instruction occupies a different length of instruction space. It is recommended to check for at least 20 units of remaining space.

1.6.9 PLC Function

PLC mode refers to a PLC-like operating mode. In this mode, users can download a program to the motion controller, which then automatically performs predefined actions based on custom states. Additionally, the controller can continue receiving commands via the communication port while in this state.

Standalone Offline Control: If you need to operate away from a computer or LCD touchscreen, you can directly use PLC mode. In this mode, users can start, stop, or pause programs using external buttons or switches to execute fixed actions.

Application Integration: Fixed actions can be programmed into the controller's internal memory. Communication and control are achieved through global variables, enabling fixed actions while allowing parameters to be dynamically adjusted.

1.6.10 Software Limit Function

Software limit mode allows each axis to be assigned a software limit function. This function defines the permissible movement range of the axis within the software, effectively simulating hardware limit functionality.

Since software limit functions operate at the software pulse position (which defaults to 0 upon power-up), they typically supplement hardware limits or zero positions. After hardware limit initialization, software limits provide additional protection.

Hardware limits and zero position are generally recommended for use.

1.6.11 Synchronous Trigger Function

Some controllers support synchronous trigger functionality.

Synchronous triggering refers to the capability of outputting a synchronous trigger signal of a specified width at designated intervals (either time-based or angle-based) within a user-defined coordinate range. This enables synchronization with external devices.

2 Development Methods and Process

2.1 Overview

1. Based on secondary development requirements, determine whether to use the dynamic link library (DLL) method or ASCII string mode. If the DLL method is selected, proceed with subsequent steps.
2. Determine the operating system: **Windows** or **Linux**
3. Determine the CPU architecture of the operating system: **X86** instruction set **CPU** or **ARM** instruction set **CPU**
4. Determine whether the **X86** CPU is **32-bit** (**i386**) or **64-bit** (**x86_64**), determine whether the **ARM** instruction set is **32-bit** (**ARM**) or **64-bit** (**AARCH64**)
5. Determine the development language and tools to be used
6. Select the example under the directory corresponding to your system and **CPU**, then run the example corresponding to your chosen development language and tools
(If none exist, refer to other examples or contact our technical support)
7. **Ensure existing executable files in the example directory run correctly and connect to the controller properly**
8. **For compiled examples, recompile the example once to verify successful compilation and that the compiled executable runs correctly**
9. **Simply modify parameters within the example or make minor adjustments to functions to grasp the development methodology**
10. Develop your application software based on the example according to your requirements
11. Compare the latest interface files (check version numbers, dates, etc.) with the routine's interface files (**MT_API.dll**, **libmt_api.so**, **MT_API.lib**, **MT_API.h**, etc.) Replace with the latest interface files as needed.

2.2 Software Design Recommendations

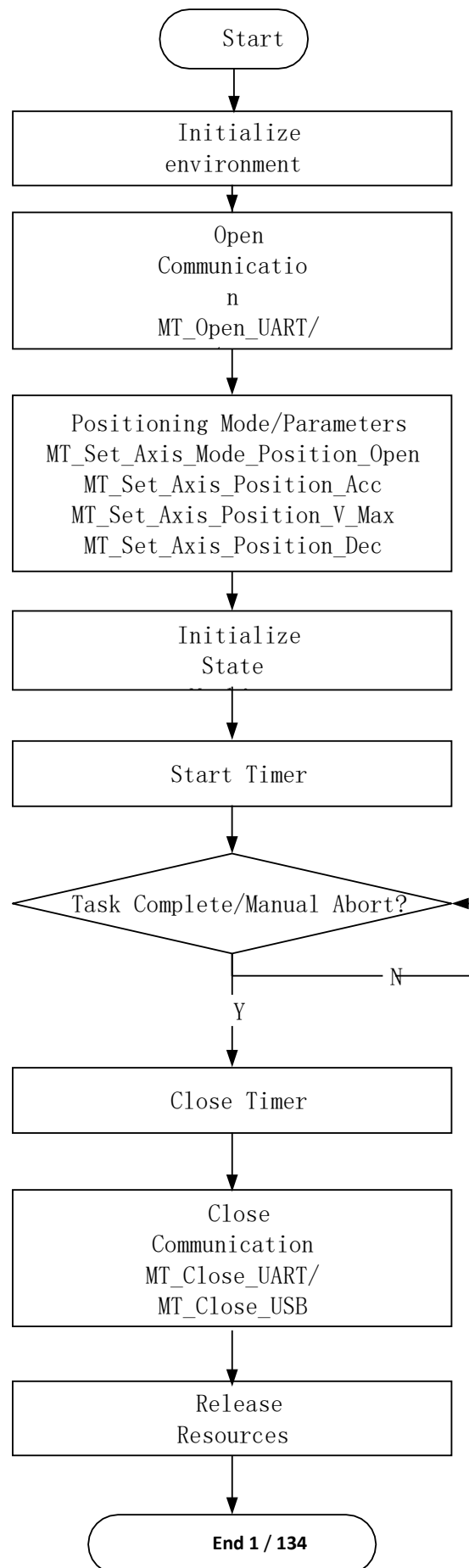
For simple operations like button clicks, directly call interface functions within events or functions. For more complex applications, such as automated sequential actions, two design approaches are recommended. The **state machine** approach is preferred.

2.2.1 State Machine (Timer-Driven)

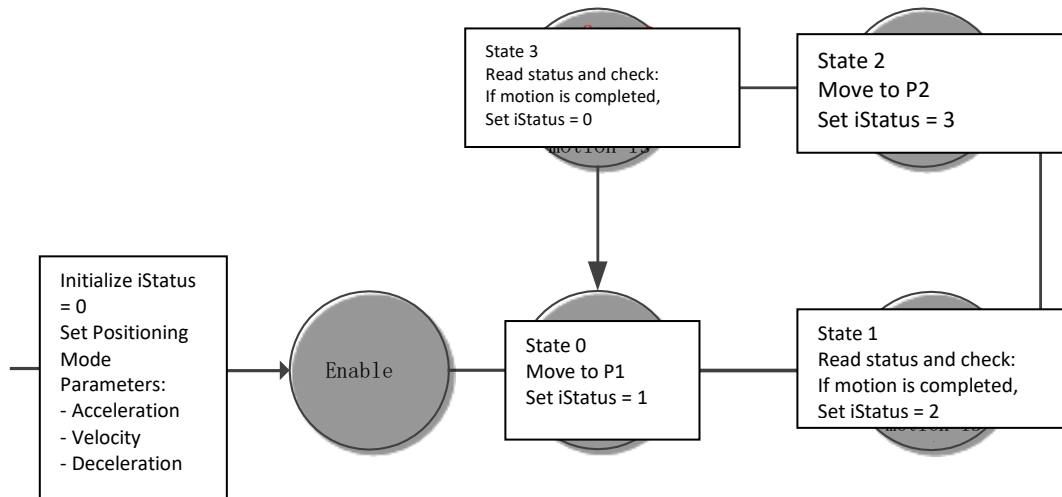
Utilize a timer's periodic interrupt (e.g., **100ms** interval) Within the timer event, evaluate one or more state variables (**iStatus**) to execute the action sequence.

Example: Implementing an application that moves back and forth between coordinates **P1** and **P2**

Within the timer event, evaluate the value of `iStatus` (0, 1, 2, 3) to determine the action to perform (assign a value to modify the state; if conditions are met, transition to a new state) The program flowchart is shown below:



The state transition diagram is shown below:



C/C++ demonstration code is shown below:

```

void CVCDemoDlg::OnTimer(UINT nIDEvent)
{
    // TODO: Add your message handler code here and/or call default
    // Read status in the customizer; continuous combined actions can be
    achieved by checking the status in this timer static int32_t iRun = 0;
    switch (iStatus)
    {
        case 0:// Move to target P1
            MT_Set_Axis_Position_P_Target_Abs(0,iP1);
            iStatus = 1;
            break;
        case 1:// Is the run complete?
            MT_Get_Axis_Status_Run(0, &iRun);
            if (0 == iRun) // Already stopped; can also check limit switch status, position, etc.
            {
                iStatus = 2;
            }
            break;
        case 2: // Move to target P1
            MT_Set_Axis_Position_P_Target_Abs(0, iP2);
            iStatus = 3;
            break;
        case 3:// Is the run complete?
            MT_Get_Axis_Status_Run(0, &iRun);
    }
}

```

```

    if (0 == iRun) // Can also check limit status, position, etc.
    {
        iStatus = 0;
    }
    break;
default: // Illegal state, stop
    state machine KillTimer(1);
}
CDialog::OnTimer(nIDEvent);
}

```

2.2.2 Thread

Controlling complex processes through a worker thread is an indirect approach. However, since **the worker thread must not directly manipulate the graphical interface**, **thread synchronization** is required to prevent random crashes. If you are unfamiliar with threads, use the state machine pattern for operation.

C/C++ demonstration code is shown below:

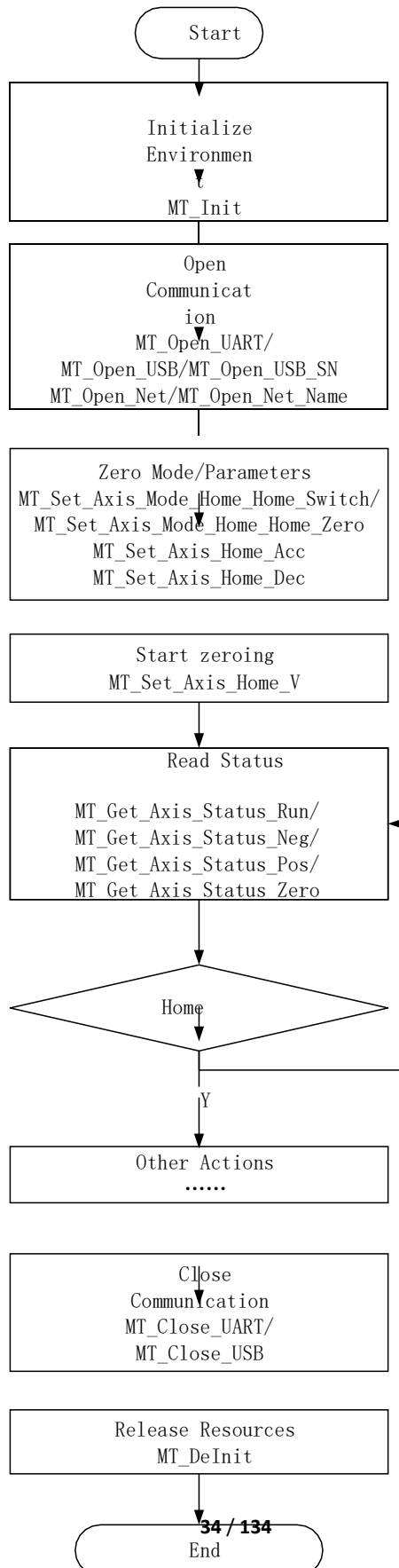
```

while (1)
{
    // Move to target point P1
    MT_Set_Axis_Position_P_Target_Abs(0, iP1);
    // Is execution
    complete? do
    {
        MT_Get_Axis_Status_Run(0, &iRun);
        Sleep(100); // Delay before checking
    }
    while (iRun > 0);

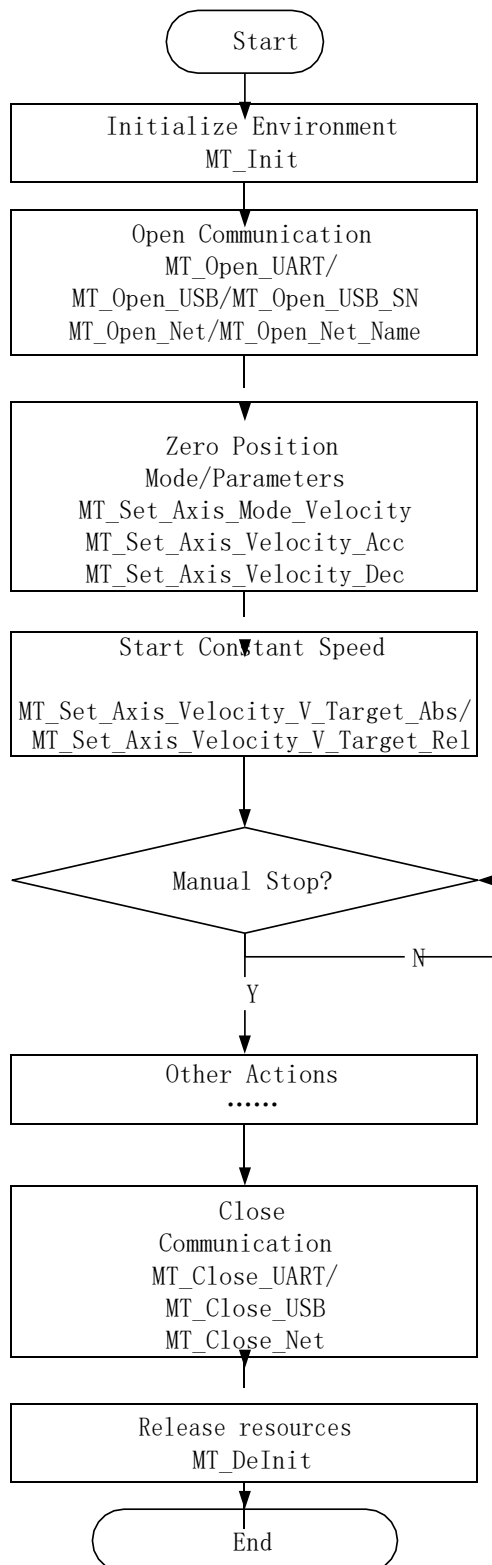
    // Move to target P2
    MT_Set_Axis_Position_P_Target_Abs(0, iP2);
    // Is the run
    complete? do
    {
        MT_Get_Axis_Status_Run(0, &iRun);
        Sleep(100); // Delay before checking
    }
    while (iRun > 0);
}

```

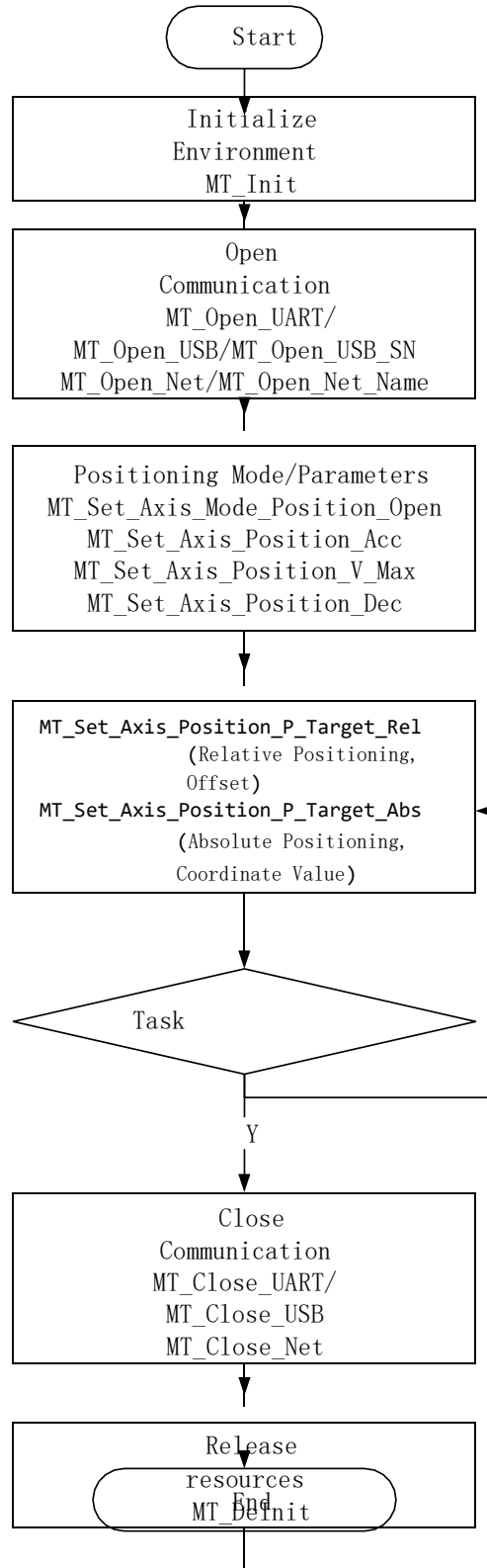
2.3 Zero Position Mode



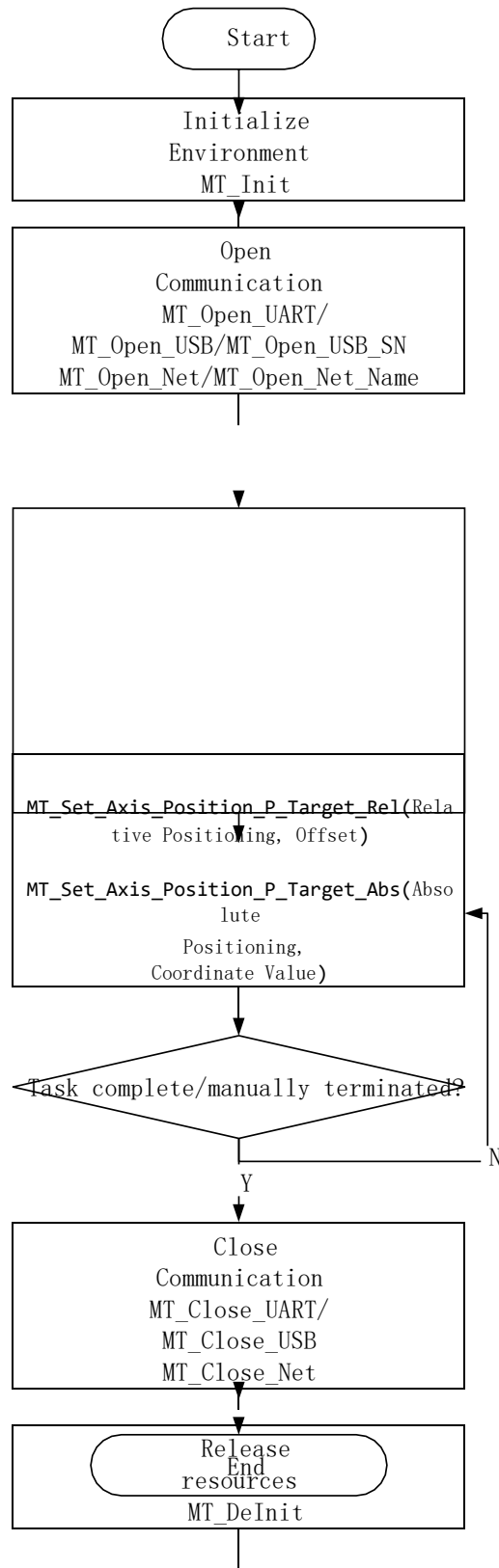
2.4 Constant Speed Mode



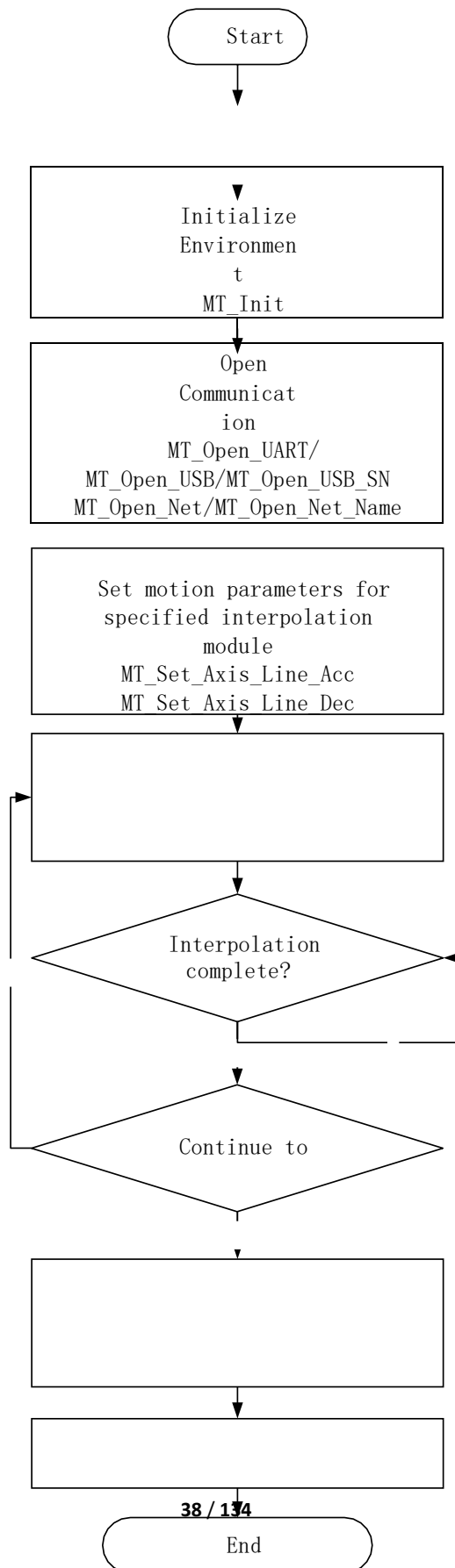
2.5 Open-Loop Positioning Mode



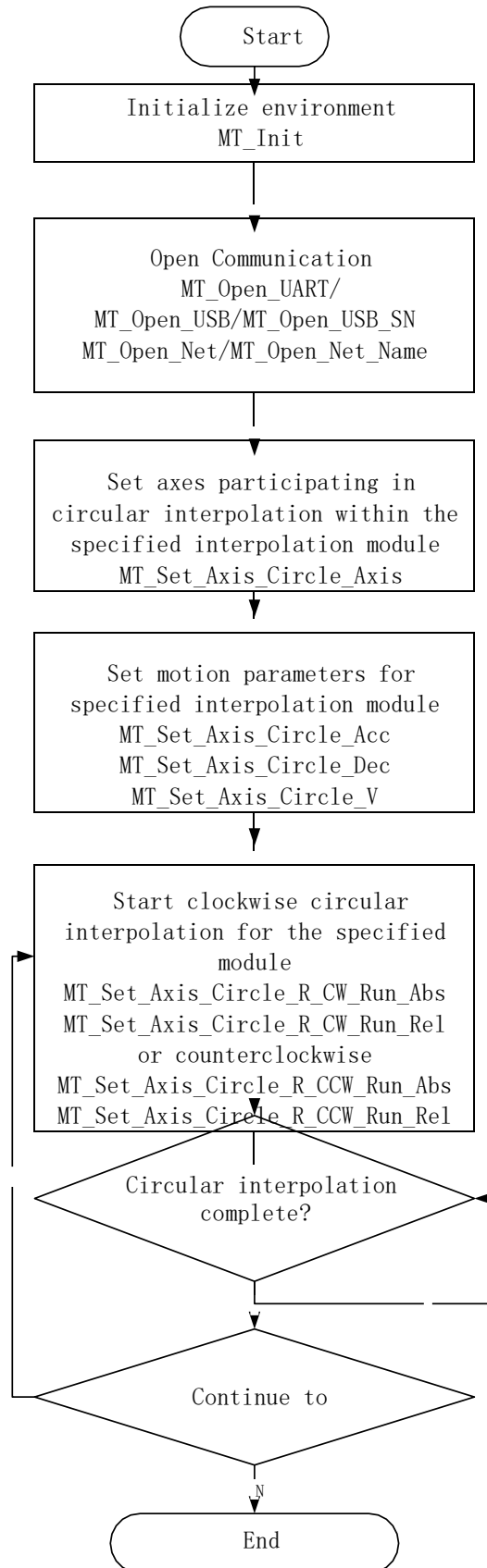
2.6 Closed-loop positioning mode



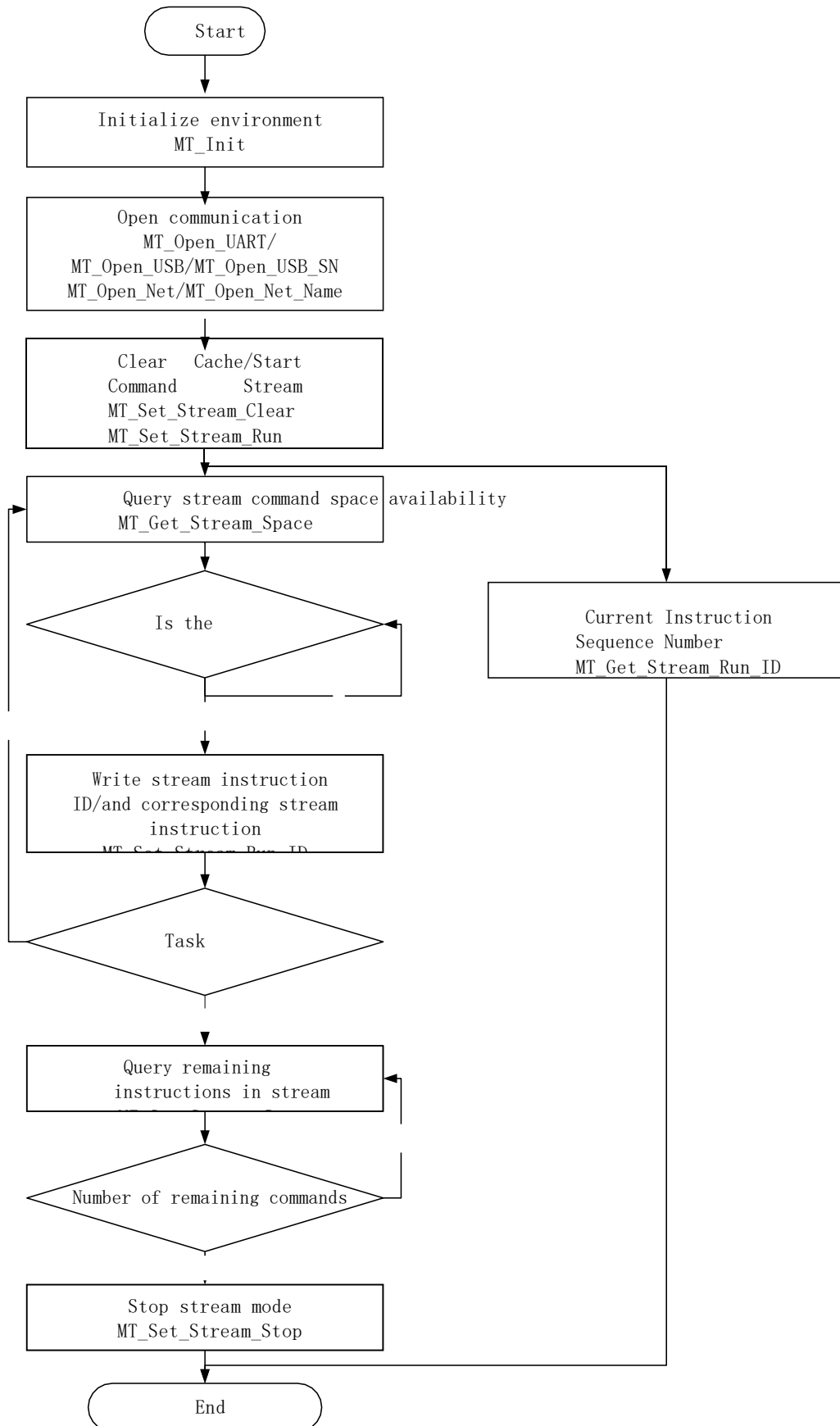
2.7 Multi-axis Interpolation Mode (Including Linear Interpolation)



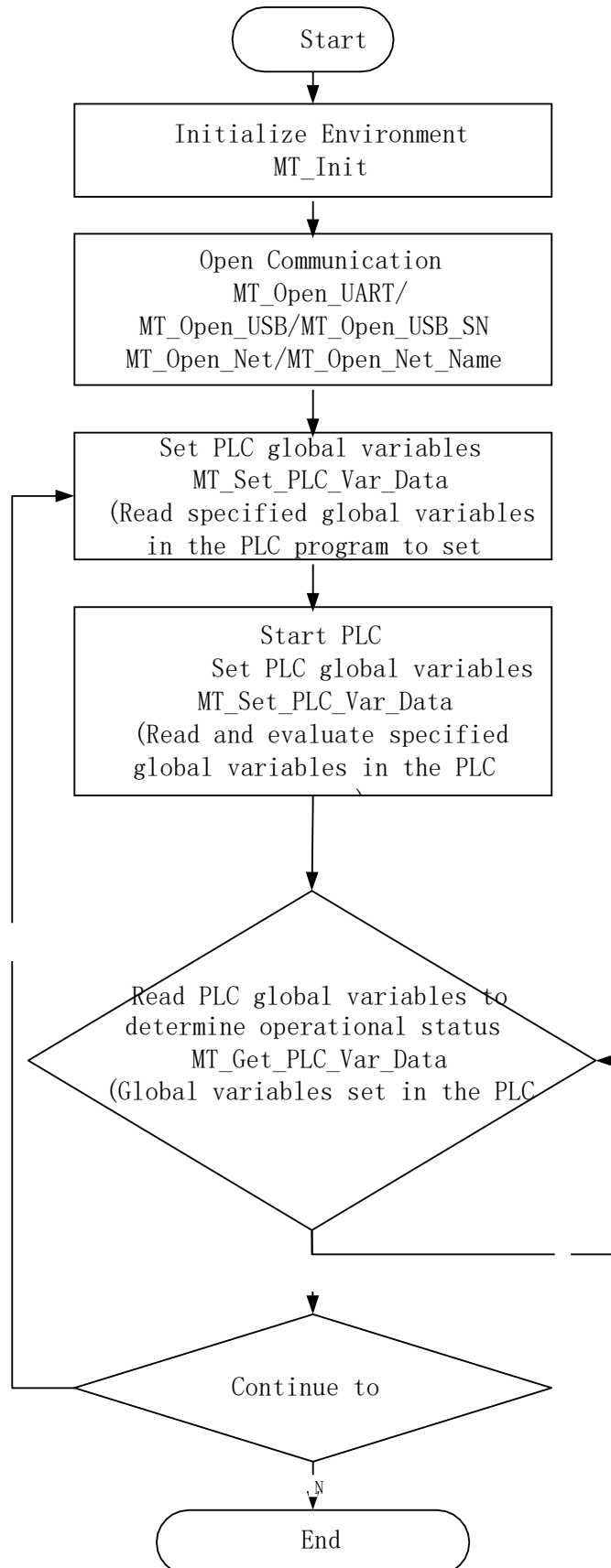
2.8 Arc interpolation mode



2.9 Instruction Streaming Mode



2.10 PLC Mode



2.11 Thread Safety

Since communication between the dynamic library and the controller must ensure integrity, multiple instructions are executed serially. The dynamic library internally uses critical section locks for synchronization during communication to meet multithreading requirements.

However, considering the complexity of multithreading, it is **strongly** recommended to adopt the design pattern provided in Section 2.2: **Single Timer State Machine Scheduling Mode**. A dedicated thread should handle MT-related functions, while other threads should not perform such operations.

2.12 Calling Conventions

To accommodate different platforms and languages, the dynamic library uses **standard calling** conventions (stdcall) as defined in the header files. Altering the calling convention due to coding errors may cause stack exceptions.

2.13 Return Value Checking

Generally, only critical initialization (**MT_Init**) and communication check (**MT_Check**) functions require return value verification; other functions may omit this check. However, in harsh environments, when communication interruptions are possible, or where extremely high reliability is demanded, verify the return value for every function call.

3 Function Description

3.1 Initialization

3.1.1 MT_Init

Function Description: Creates resources required for the dynamic library to function. **Must** be called **before using other** functions. **Typically executed in the software's initialization section and can only be executed once.** Otherwise, exceptions such as memory access errors or leaks may occur.

C/VC/Gcc	int32_t MT_Init(void)	
VB	Function MT_Init () As Long	
Delphi	function MT_Init():Integer	
C#	public static extern int MT_Init()	
Input Parameters	None	
Output Parameters	None	
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks		

3.1.2 MT_DeInit

Function Description: Releases resources required for the dynamic link library to operate. Executes when the software exits and **only needs to be performed once.**

C/VC/Gcc	int32_t MT_DeInit (void)	
VB	Function MT_DeInit () As Long	
Delphi	function MT_DeInit ():Integer;	
C#	public static extern int MT_DeInit ();	
Input Parameters	None	
Output Parameters	None	
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks		

3.1.3 MT_Get_Dll_Version

Function Description: Reads the version number of this **DLL**. The **DLL** version can also be viewed via **Windows** Explorer. This function can be used to determine compatibility with different versions of **DLLs** or **SO** files, but it **is generally not used**.

C/VC/Gcc	int32_t MT_Get_Dll_Version(char** sVer)
----------	---

VB	Function MT_Get_Dll_Version (ByRef sVer As String) As Long	
Delphi	function MT_Get_Dll_Version(sVer:PPChar):Integer;	
C#	public static extern int MT_Get_Dll_Version(ref String sVer);	
Input Parameters	None	
Output Parameters	sVer	Version string
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks		

3.2 Communication port

3.2.1 MT_Close_UART

Function Description: Closes the serial communication port. Call this function when switching communication ports, exiting the software, or releasing occupied serial ports.

C/VC/GCC	int32_t MT_Close_UART (void)	
VB	Function MT_Close_UART () As Long	
Delphi	function MT_Close_UART():Integer;	
C#	public static extern int MT_Close_UART();	
Input Parameters	None	
Output Parameters	None	
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks		

3.2.2 MT_Open_UART

Function Description: Opens the serial communication port. Call this function before communicating with the controller via the serial port. Specify the serial port number. The default serial parameters are 115200, N, 8, 1 and require no configuration. Note whether the connected hardware interface is RS232, RS422, RS485, or TTL. Standard hardware uses RS232.

C/VC/GCC	int32_t MT_Open_UART (char* sCOM)	
VB	Function MT_Open_UART (ByVal sCOM As String) As Long	
Delphi	function MT_Open_UART(sCOM:PAnsiChar):Integer;	
C#	public static extern int MT_Open_UART(string sCOM);	

Input Parameters	sCOM	String, serial port name, e.g. Windows: COM1, COM2 Linux: /dev/ttyUSB0, /dev/ttyS1
Output Parameters	None	
Function Returns	0	Function execution successful
	Non-zero	Function execution failed

Remarks	Successful function execution does not guarantee serial port functionality; it merely completes the initialization process for serial communication. Level errors or wiring mistakes may cause subsequent communication failures.
---------	---

3.2.3 MT_Open_UART_Unicode

Function Description: The wide-character version of **MT_Open_UART**. Suitable for development tools supporting only wide characters, though **MT_Open_UART_ID is recommended**. Opens the serial communication port. Call before communicating with the controller via the serial port. Requires specifying the serial port number. The default serial parameters are 115200, N, 8, 1 and do not need to be set. Note the hardware interface type (RS232, RS422, RS485, or TTL). Standard hardware uses RS232.

VC	int32_t MT_Open_UART_Unicode(TCHAR* sCOM)	
GCC	int32_t MT_Open_UART_Unicode(wchar* sCOM)	
VB	Function MT_Open_UART_Unicode(ByVal sCOM As String) As Long	
Delphi	function MT_Open_UART_Unicode(sCOM:PChar):Integer;	
C#	public static extern int MT_Open_UART_Unicode(string sCOM);	
Input Parameters	sCOM	String, serial port name, e.g. Windows: COM1, COM2 Linux: /dev/ttyUSB0, /dev/ttyS1
Output Parameters	None	
Function Returns	0	Function execution successful
	Non-zero	Function execution failed
Note	Successful function execution does not guarantee serial port functionality; it merely completes the initialization process for serial communication. Level errors or wiring mistakes may cause subsequent communication failures.	

3.2.4 MT_Open_UART_ID

Function Description: Version **number (1, 2, 3, ...)** of **MT_Open_UART**. Opens the serial communication port. Call this function before communicating with the controller via the serial port. Specify the serial port number. The default serial parameters are 115200, N, 8, 1 and do not require configuration. Note the connected hardware interface type: RS232, RS422, RS485, or TTL. Standard hardware uses RS232.

C/VC/GCC	int32_t MT_Open_UART_ID(int32_t AID)
----------	--------------------------------------

VB	Function MT_Open_UART_ID (ByVal AID As Integer) As Long	
Delphi	function MT_Open_UART_ID(AID:Integer):Integer;	
C#	public static extern int MT_Open_UART_ID(Int32 ADev);	
Input Parameters	AID	Serial port ID, 0,1,2,3,... Corresponding strings are Windows: COM0, COM1, COM2, COM3,... Linux: /dev/ttyS0, /dev/ttyS1, /dev/ttyS2,...
Output Parameters	None	
Function Returns	0	Function execution successful
	Non-zero	Function execution failed

Remarks	Successful function execution does not guarantee serial port functionality; it merely completes the initialization process for serial communication. Level errors or wiring mistakes may cause subsequent communication failures.
---------	---

3.2.5 MT_Close_USB

Function Description: Closes the **USB** communication port. Call this function when switching communication ports or exiting the software.

C/VC/GCC	int32_t MT_Close_USB (void)	
VB	Function MT_Close_USB () As Long	
Delphi	function MT_Close_USB(): Integer;	
C#	public static extern int MT_Close_USB();	
Input Parameters	None	
Output Parameters	None	
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks		

3.2.6 MT_Open_USB

Function Description: Opens the **USB** communication port. Call this function before communicating with the controller via **USB**. When executed, it automatically **scans** the computer for controller devices.

C/VC/GCC	int32_t MT_Open_USB (void)	
VB	Function MT_Open_USB () As Long	
Delphi	function MT_Open_USB(): Integer;	
C#	public static extern int MT_Open_USB();	
Input Parameters	None	
Output Parameters	None	
Function Returns	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Function execution success does not guarantee USB functionality; it merely indicates completion of communication initialization.	

3.2.7 MT_Open_USB_SN

Function Description: Opens the **USB** communication port. Call this function before communicating with the controller via **USB**. When executed, it automatically scans the computer for a controller device with the specified serial number.

VC	int32_t MT_Open_USB_SN(TCHAR* ASN)
GCC	int32_t MT_Open_USB_SN(wchar* ASN)
VB	Function MT_Open_USB_SN(ByVal ASN As String) As Long

Delphi	function MT_Open_USB_SN(ASN:PChar):Integer;	
C#	public static extern int MT_Open_USB_SN(string ASN);	
Input Parameters	ASN	Serial number string, typically the serial port name, e. g. Windows: COM1, COM2 Linux: /dev/ttyUSB0, /dev/ttyS1
Output Parameters	None	
Function Returns	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Function execution success does not guarantee USB functionality; it merely indicates completion of communication initialization.	

3.2.8 MT_Close_Net

Function Description: Closes the network communication port. Call this when switching communication ports or exiting the software.

C/VC/GCC	int32_t MT_Close_Net (void)	
VB	Function MT_Close_Net () As Long	
Delphi	function MT_Close_Net():Integer;	
C#	public static extern int MT_Close_Net();	
Input Parameters	None	
Output Parameters	None	
Function Returns	0	Function execution successful
	Non-zero	Function execution failed
Remarks		

3.2.9 MT_Open_Net

Function Description: Opens the network communication port. Call this function before communicating with the controller over the network. Requires setting the controller's IP address and port number.

C/VC/GCC	int32_t MT_Open_Net (BYTE IP0, BYTE IP1, BYTE IP2, BYTE IP3, WORD IPPort)	
VB	Function MT_Open_Net (ByVal IP0 As Byte, ByVal IP1 As Byte, ByVal IP2 As Byte, ByVal IP3 As Byte, ByVal IPPort As Integer) As Long	
Delphi	function MT_Open_Net(IP0, IP1, IP2, IP3: Byte; IPPort: Word): Integer;	

C#	public static extern int MT_Open_Net(BYTE IP0, BYTE IP1, BYTE IP2, BYTE IP3, WORD IPPort);	
Input Parameters	IP0, IP1, IP2, IP3	IP address, e. g. , 192.168.0.2
	IPPort	Port address, e. g. , 8888
Output Parameters	None	
Function Returns	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Function execution success indicates the network has successfully connected to the server port and established the network channel.	

3.2.10 MT_Open_Net_Name

Function Description: Opens the network communication port. Call this before communicating with the controller over the network. Requires setting the controller's IP address or network name and communication port number.

C/VC/GCC	int32_t MT_Open_Net_Name (char* IP_or_Name, uint16_t IPPort)	
VB	Function MT_Open_Net_Name(ByVal IP_or_Name As String, ByVal IPPort As Integer) As Long	
Delphi	function MT_Open_Net_Name(IP_or_Name :PAnsiChar; IPPort :Word) : Integer;	
C#	public static extern int MT_Open_Net_Name(string IP_or_Name, word IPPort);	
Input Parameters	IP_or_Name	IP address, e. g. , 192.168.0.2, or network name, e. g. , MT-340
	IPPort	Port address, e. g. , 8888
Output Parameters	None	
Function Returns	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Function execution success indicates the network has successfully connected to the server port and established the network channel.	

3.3 Communication Handshake

3.3.1 MT_Check

Function Description: After opening the communication port, detects whether a controller exists.

C/VC/GCC	int32_t MT_Check (void)	
VB	Function MT_Check () As Long	
Delphi	function MT_Check (): Integer;	
C#	public static extern int MT_Check();	
Input Parameters	None	
Output Parameters	None	
Function Returns	0	Function execution successful, controller detected, communication possible
	Non-zero	Function execution failed

Remarks	
---------	--

Example:

```
// Select one of the following  
communication methods:  
MT_Open_USB();  
MT_Open_UART("COM1");  
MT_Open_Net(192, 168, 0, 1, 8888);
```

```

if (0 == MT_Check())
{
    // Communication successful, proceed with subsequent tasks

}
else
{
    // Communication failed. Display prompt or attempt reconnection

}

```

3.4 Hardware Information

3.4.1 MT_Get_Product_Resource

Function Description: Detects the types of resources available in the hardware, including the number of motor axes controlled, input/output information, etc. Combined with functions for reading the quantity of different resource types, this enables the final determination of specific resources. Different controllers support different resources. If the resources of the controller in use are already known, this function need not be called. For software compatibility considerations, this function may be invoked. Resources can also be viewed using official tools.

C/VC/GCC	int32_t MT_Get_Product_Resource (int32_t* pValue)	
VB	Function MT_Get_Product_Resource (ByRef Value As Long) As Long	
Delphi	function MT_Get_Product_Resource (pValue:PInteger):Integer;	
C#	public static extern int MT_Get_Product_Resource (ref Int32 Value);	
Input Parameters	None	

Output Parameters	Value	Outputs the resource type in the hardware, which is the result of an OR operation between the following resource types: Currently defined resources <pre> #define RESOURCE_MOTOR 0x00000001 //Motor #define RESOURCE_TTL_IN 0x00000002 // TTL Input #define RESOURCE_TTL_OUT 0x00000004 //TTL Output #define RESOURCE_OPTIC_IN 0x00000008 //Optical Isolator Input #define RESOURCE_OPTIC_OUT 0x00000010 //Optically isolated output #define RESOURCE_AD 0x00000020 // Analog Input #define RESOURCE_OC 0x00000040 //Overcurrent Output #define RESOURCE_LINE 0x00000080 //Linear interpolation #define RESOURCE_CIRCLE 0x00000100 //Circular interpolation #define RESOURCE_PLC 0x00000200 //PLC Function Support #define RESOURCE_STREAM 0x00000400 //Stream Mode Support #define RESOURCE_ENCODER 0x00000800 // Encoder Ruler #define RESOURCE_TRIGGER 0x00004000 // Synchronous comparison trigger </pre>
Function Returns	0	Function execution successful, read controller resource valid
	Non-zero	Function execution failed; the read controller resource is invalid

Remarks	
---------	--

3.4.2 MT_Get_Product_ID

Function Description: Retrieves the controller model. Note: This function is not required. The model can also be viewed using official tools.

C/VC/GCC	int32_t MT_Get_Product_ID (char* sID)	
VB	Function MT_Get_Product_ID (ByRef sID As String) As Long	
Delphi	function MT_Get_Product_ID(sID: PAnsiChar): Integer;	
C#	public static extern int MT_Get_Product_ID (ref String sID);	
Input Parameters	None	
Output Parameters	sID	16-byte character buffer
Function Return	0	Function executed successfully, model read is valid
	Non-zero	Function execution failed; the read model is invalid
Note	The buffer must be allocated by the user beforehand.	

3.4.3 MT_Get_Product_SN

Function Description: Reads the controller's serial number. This information can also be viewed using official tools. The serial number serves as a criterion for our company to verify product-related information. Users can also utilize this unique serial number to complete tasks such as confirming their products.

C/VC/GCC	int32_t MT_Get_Product_SN (char* sSN)	
VB	Function MT_Get_Product_SN (ByRef sSN As String) As Long	
Delphi	function MT_Get_Product_SN(sID: PAnsiChar): Integer;	
C#	public static extern int MT_Get_Product_SN (ref String sSN);	
Input Parameters	None	
Output Parameters	sSN	Character buffer of length 12
Function Returns	0	Function execution successful, serial number read is valid
	Non-zero	Function execution failed; the read serial number is invalid
Note	The buffer requires the user to allocate memory in advance	

3.4.4 MT_Get_Product_Version [Compatible]

Function Description: Reads the version number of the controller firmware. This information can also be viewed using official tools. The version number serves as a criterion for our company to verify product-related details.

C/VC/GCC	int32_t MT_Get_Product_Version(int32_t* pMajor, int32_t* pMinor)
VB	Function MT_Get_Product_Version (ByRef Major As Long, ByRef Minor As Long) As Long
Delphi	function MT_Get_Product_Version(pMajor: PInteger; pMinor: PInteger): Integer;

C#	public static extern int MT_Get_Product_Version(ref int32 Major, ref int32 Minor);	
Input Parameters	None	
Output Parameters	Major	Major Version Number
	Minor	Minor Version Number
Function Returns	0	Function execution successful, version number read is valid
	Non-zero	Function execution failed; the read version number is invalid
Remarks		

3.4.5 MT_Get_Product_Version2

Function Description: Reads the firmware version number of the controller. This information can also be viewed using official tools. The version number serves as a criterion for our company to verify product-related information.

C/VC/GCC	int32_t MT_Get_Product_Version2(int32_t* pMajor, int32_t* pMinor, int32_t* pRelease, int32_t* pBuild)	
VB	Function MT_Get_Product_Version2 (ByRef pMajor As Long, ByRef pMinor As Long, ByRef pRelease As Long, ByRef pBuild As Long) As Long	
Delphi	function MT_Get_Product_Version2(pMajor: PInteger; pMinor: PInteger; pRelease: PInteger; pBuild: PInteger): Integer;	
C#	public static extern int MT_Get_Product_Version2(ref Int32 pMajor, ref Int32 pMinor, ref Int32 pRelease, ref Int32 pBuild);	
Input Parameters	None	
Output Parameters	pMajor	Major Version Number
	pMinor	Minor Version Number
	pRelease	Release Count
	pBuild	Build count
Function Return	0	Function executed successfully; version number read is valid
	Non-zero	Function execution failed; the read version number is invalid
Remarks		

3.5 Zero Position Mode (Open-Loop and Closed-Loop)

3.5.1 MT_Set_Axis_Mode_Home[Compatible]

Function Description: Sets the specified axis to open-loop zero position mode,

equivalent to ***MT_Set_Axis_Mode_Home_Home_Switch***

C/VC/GCC	int32_t MT_Set_Axis_Mode_Home(WORD AObj)	
VB	Function MT_Set_Axis_Mode_Home (ByVal AObj As Integer) As Long	
Delphi	function MT_Set_Axis_Mode_Home(AObj:Word):Integer;	
C#	public static extern int MT_Set_Axis_Mode_Home(UInt16 AObj);	
Input Parameters	AObj	Motor control axis serial number

Output Parameters	None	
Function Return	0	Function Execution Success
	Non-zero	Function execution failed
Remarks	Motor shaft numbers start from 0. For an N-axis card, the shaft numbers are 0, 1, 2, 3...N-1 Functions utilizing motor shaft serial numbers must ensure no out-of-bounds access occurs.	

3.5.2 MT_Set_Axis_Mode_Home_Home_Switch

Function Description: Sets the specified axis to open-loop zero-position mode. Zero-position detection relies on zero-position switches and limit switches. The first encounter with either a zero-position switch or a limit switch satisfies the stop condition, halting motion and adjusting **the open-loop logical coordinates** of the current stop position to **0**. Refer to Section 2.3 for detailed usage instructions. In zero-position mode, if a closed-loop encoder is present, it operates normally, reflecting changes corresponding to the encoder.

C/VC/GCC	int32_t MT_Set_Axis_Mode_Home_Home_Switch (WORD AObj)	
VB	Function MT_Set_Axis_Mode_Home_Home_Switch (ByVal AObj As Integer) As Long	
Delphi	function MT_Set_Axis_Mode_Home_Home_Switch (AObj:Word):Integer;	
C#	public static extern int MT_Set_Axis_Mode_Home_Home_Switch (UInt16 AObj);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	None	
Function Return	0	Function Execution Success
	Non-zero	Function execution failed
Remarks	Motor shaft numbers start from 0. For an N-axis card, the shaft numbers are 0, 1, 2, 3...N-1 Functions utilizing motor axis serial numbers must avoid out-of-bounds access	

3.5.3 MT_Set_Axis_Mode_Home_Home_Zero

Function Description: Sets the specified axis to open-loop zero-position mode. Prioritizes searching for zero-position switches. If the first encountered switch is a zero-position switch, immediately stops, completes zero-positioning, halts motion, and adjusts **the open-loop logical coordinates** of the current stop position to **0**. If the first switch encountered is a limit switch, it automatically reverses direction and attempts the process again. Both zero position switches and limit switches serve as stopping criteria. Encountering either a zero position switch or a limit switch satisfies the stopping condition, halting motion and adjusting **the open-loop logical coordinates** of the

current stop position to **0**. For detailed usage, refer to Section **2.3**. In zero position mode, if a closed-loop sensor is present, its readings function normally and reflect changes in the sensor's

.

C/VC/GCC	int32_t MT_Set_Axis_Mode_Home_Home_Zero (WORD AObj)	
VB	Function MT_Set_Axis_Mode_Home_Home_Zero (ByVal AObj As Integer) As Long	
Delphi	function MT_Set_Axis_Mode_Home_Home_Zero (AObj:Word):Integer;	
C#	public static extern int MT_Set_Axis_Mode_Home_Home_Zero(UInt16 AObj);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	None	
Function Return	0	Function Execution Success
	Non-zero	Function execution failed

Remarks	Motor shaft numbers start from 0. For an N-axis card, the shaft numbers are 0, 1, 2, 3...N-1 Functions utilizing motor axis indices must avoid out-of-bounds access
---------	--

3.5.4 MT_Set_Axis_Mode_Home_Encoder_Index

Function Description: Sets the specified axis to closed-loop zero-position mode. Determines zero-position based on the encoder zero position and the linear scale zero position. The first occurrence of either the encoder zero position or the linear scale zero position satisfies the stop condition, halting motion and adjusting the closed-loop logic coordinate of the current stop position to 0. Refer to Section 2.3 for detailed usage. Note: Some controllers have closed-loop sensor interfaces with only AB/Z axes.

Closed-loop logic coordinates at the current stop position to **0**. Refer to Section 2.3 for detailed usage. Note: Some controllers only have AB phase and no Z phase, and thus do not support this function. In closed-loop zero mode, pulse counting in the open-loop coordinate system functions normally, reflecting the the number of pulses generated.

C/VC/GCC	int32_t MT_Set_Axis_Mode_Home_Encoder_Index (WORD AObj)	
VB	Function MT_Set_Axis_Mode_Home_Encoder_Index (ByVal AObj As Integer) As Long	
Delphi	function MT_Set_Axis_Mode_Home_Encoder_Index(AObj: Word): Integer;	
C#	public static extern int MT_Set_Axis_Mode_Home_Encoder_Index(UInt16 AObj);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	None	
Function Return	0	Function Execution Success
	Non-zero	Function execution failed
Remarks	Motor shaft numbers start from 0. For an N-axis card, the shaft numbers are 0, 1, 2, 3...N-1 Functions utilizing motor axis serial numbers must avoid out-of-bounds access	

3.5.5 MT_Set_Axis_Mode_Home_Encoder_Home_Switch

Function Description: Sets the specified axis to closed-loop zero-position mode. Zero-position detection relies on zero switches and limit switches. The first encounter with either a zero switch or limit switch satisfies the stop condition,

halting motion and adjusting the **closed-loop logical coordinates** of the current stop position to **0**. Refer to Section **2.3** for detailed usage. In closed-loop zero-position mode, pulse counting in the open-loop coordinate system functions normally, reflecting the number of pulses emitted.

C/VC/GCC	int32_t MT_Set_Axis_Mode_Home_Encoder_Home_Switch (WORD AObj)	
VB	Function MT_Set_Axis_Mode_Home_Encoder_Home_Switch (ByVal AObj As Integer) As Long	
Delphi	function MT_Set_Axis_Mode_Home_Encoder_Home_Switch (AObj:Word):Integer;	
C#	public static extern int MT_Set_Axis_Mode_Home_Encoder_Home_Switch (UInt16 AObj);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	None	
Function Return	0	Function Execution Success
	Non-zero	Function execution failed
Remarks	Motor shaft numbers start from 0. For an N-axis card, the shaft numbers are 0, 1, 2, 3...N-1	

	Functions utilizing motor axis serial numbers must avoid out-of-bounds access
--	---

3.5.6 MT_Set_Axis_Home_V

Function Description: Sets the specified axis to search for zero position at the specified speed. A positive speed value indicates forward zero search; a negative speed value indicates reverse zero search. This function **immediately initiates** the zero search action without requiring additional start functions. Use the open-loop coefficient for unit conversion. This function is only effective in zero position mode and does not operate in other modes.

C/VC/GCC	int32_t MT_Set_Axis_Home_V (WORD AObj, int32_t pValue);	
VB	Function MT_Set_Axis_Home_V (ByVal AObj As Integer, ByVal Value As Long) As Long	
Delphi	function MT_Set_Axis_Home_V (AObj:Word;Value:Integer):Integer;	
C#	public static extern int MT_Set_Axis_Home_V (UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	Value	Set zero-position search speed for specified axis Hz/S Positive/Negative
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Motor shaft serial numbers start from 0. For N shafts, the serial numbers are 0, 1, 2, 3...N-1 Ensure the motor shaft number function does not exceed bounds. Slower speeds improve search accuracy.	

3.5.7 MT_Set_Axis_Home_Stop

Function Description: Stops the zero-position movement mode of the specified axis, decelerating to a halt at the preset deceleration rate. This function is only effective in zero-position mode and does not operate in other modes. For immediate emergency stops, **use MT_Set_Axis_Halt or MT_Set_Axis_Halt_All**.

C/VC/GCC	int32_t MT_Set_Axis_Home_Stop (WORD AObj)	
VB	Function MT_Set_Axis_Home_Stop (ByVal AObj As Integer) As Long	
Delphi	function MT_Set_Axis_Home_Stop (AObj:Word):Integer;	
C#	public static extern int MT_Set_Axis_Home_Stop (UInt16 AObj);	
Input Parameters	AObj	Motor control axis serial number

Output Parameters	None	
Function	0	Function Execution Success
Return	Non-zero	Function execution failed
Remarks	Motor shaft numbers start from 0 . For an N-axis card, the shaft numbers are 0, 1, 2, 3...N-1 Functions utilizing motor axis serial numbers must avoid out-of-bounds access	

3.5.8 MT_Set_Axis_Home_V_Start

Function Description: Sets the initial speed for zero-position mode startup of the specified axis. Default value is 50Hz/s. Generally, modifying this parameter is prohibited. Use the open-loop coefficient for unit conversion.

C/VC/GCC	int32_t MT_Set_Axis_Home_V_Start (WORD AObj, int32_t pValue);	
VB	Function MT_Set_Axis_Home_V_Start(ByVal AObj As Integer, ByVal Value As Long) As Long	
Delphi	function MT_Set_Axis_Home_V_Start (AObj:Word;Value:Integer):Integer;	
C#	public static extern int MT_Set_Axis_Home_V_Start (UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	Value	Set the start speed for the specified axis Hz/S Positive only
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Motor shaft serial numbers start from 0. For the Nth shaft card, serial numbers are 0, 1, 2, 3...N-1 Ensure the motor axis number function does not exceed bounds	

3.5.9 MT_Set_Axis_Home_Acc

Function Description: Sets the acceleration value for the zero position mode of the specified axis. Default is 500Hz/s². Use the open-loop coefficient for unit conversion.

C/VC/GCC	int32_t MT_Set_Axis_Home_Acc (WORD AObj, int32_t pValue);	
VB	Function MT_Set_Axis_Home_Acc(ByVal AObj As Integer, ByVal Value As Long) As Long	
Delphi	function MT_Set_Axis_Home_Acc (AObj: Word; Value: Integer): Integer;	
C#	public static extern int MT_Set_Axis_Home_Acc (UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	Value	Set acceleration for specified axis Hz/S ² Positive Only
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Motor shaft serial numbers start from 0. For the Nth shaft card, serial numbers are 0, 1, 2, 3...N-1 Ensure the motor axis number function does not exceed bounds	

3.5.10 MT_Set_Axis_Home_Dec

Function Description: Sets the deceleration value for the zero position mode of the specified axis. Default is 500Hz/s². Use the open-loop coefficient for unit conversion.

C/VC/GCC	int32_t MT_Set_Axis_Home_Dec (WORD AObj, int32_t pValue);	
----------	---	--

VB	Function MT_Set_Axis_Home_Dec(ByVal AObj As Integer, ByVal Value As Long) As Long	
Delphi	function MT_Set_Axis_Home_Dec (AObj:Word;Value:Integer):Integer;	
C#	public static extern int MT_Set_Axis_Home_Dec (UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	Value	Set deceleration for specified axis Hz/S ² Positive Only

Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Motor shaft serial numbers start from 0. For an N-axis card, the serial numbers are 0, 1, 2, 3...N-1 Ensure the motor axis number function does not exceed bounds	

3.5.11 MT_Get_Axis_Home_Acc

Function Description: Reads the acceleration value for the specified control axis's zero-position mode. Use the open-loop coefficient for unit conversion.

C/VC/GCC	int32_t MT_Get_Axis_Home_Acc(WORD AObj, int32_t* pValue);	
VB	Function MT_Get_Axis_Home_Acc(ByVal AObj As Integer, ByRef Value As Long) As Long	
Delphi	function MT_Get_Axis_Home_Acc(AObj: Word; pValue: PInteger): Integer;	
C#	public static extern int MT_Get_Home_Acc(UInt16 AObj, ref Int32 Value);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	Value	Current acceleration of the specified axis Hz/S ²
Function Return	0	Function execution successful, data read is valid
	Non-zero	Function execution failed; the read data is invalid
Remarks	Motor shaft serial numbers start from 0. For an N-axis card, serial numbers are 0, 1, 2, 3...N-1 Ensure the motor shaft index function does not exceed bounds	

3.5.12 MT_Get_Axis_Home_Dec

Function Description: Reads the deceleration value for the zero-position mode of the specified control axis. Use the open-loop coefficient for unit conversion.

C/VC/GCC	int32_t MT_Get_Axis_Home_Dec(WORD AObj, int32_t* pValue);	
VB	Function MT_Get_Axis_Home_Dec(ByVal AObj As Integer, ByRef Value As Long) As Long	
Delphi	function MT_Get_Axis_Home_Dec(AObj: Word; pValue: PInteger): Integer;	
C#	public static extern int MT_Get_Home_Decc(UInt16 AObj, ref Int32 Value);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	Value	Current deceleration of the specified axis Hz/s ²
Function returns	0	Function execution successful, read data is valid
	Non-zero	Function execution failed; the read data is invalid

Notes	Motor shaft serial numbers start from 0 . For an N-axis card, serial numbers are 0, 1, 2, 3...N-1 Ensure the motor shaft serial number function does not exceed bounds
-------	---

3.6 Constant Speed Mode

3.6.1 MT_Set_Axis_Mode_Velocity

Function Description: Sets the specified axis to constant speed mode. If a closed-loop encoder is present, it operates normally and responds to changes corresponding to the encoder.

C/VC/GCC	int32_t MT_Set_Axis_Mode_Velocity(WORD AObj)	
VB	Function MT_Set_Axis_Mode_Velocity (ByVal AObj As Integer) As Long	
Delphi	function MT_Set_Axis_Mode_Velocity(AObj:Word):Integer;	
C#	public static extern int MT_Set_Axis_Mode_Velocity(UInt16 AObj);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	None	
Function Return	0	Function Execution Success
	Non-zero	Function execution failed
Remarks	Motor shaft numbers start from 0. For an N-axis card, the shaft numbers are 0, 1, 2, 3...N-1 Functions utilizing motor axis serial numbers must avoid out-of-bounds access	

3.6.2 MT_Get_Axis_Velocity_V_Target

Function Description: Reads the target speed in constant speed mode. Use the open-loop conversion factor for unit conversion.

C/VC/GCC	int32_t MT_Get_Axis_Velocity_V_Target (WORD AObj, int32_t* pValue);	
VB	Function MT_Get_Axis_Velocity_V_Target (ByVal AObj As Integer, ByRef Value As Long) As Long	
Delphi	function MT_Get_Axis_Velocity_V_Target (AObj:Word;pValue:PInteger):Integer;	
C#	public static extern int MT_Get_Axis_Velocity_V_Target (UInt16 AObj, ref Int32 Value);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	Value	Target speed of the specified axis Hz/S
Function Return	0	Function execution successful, read data valid
	Non-zero	Function execution failed; the read data is invalid
Notes	Motor shaft serial numbers start from 0. For an N-axis card, serial numbers are 0, 1, 2, 3...N-1 Ensure motor axis number functions do not exceed bounds	

3.6.3 MT_Set_Axis_Velocity_V_Target_Abs

Function Description: Sets the absolute target velocity (open-loop) in constant speed mode and **immediately initiates** constant speed operation. Use the open-loop conversion factor for unit conversion. This function is only effective in constant speed mode and does not operate in other modes.

C/VC/GCC	int32_t MT_Set_Axis_Velocity_V_Target_Abs (WORD AObj, int32_t pValue);
VB	Function MT_Set_Axis_Velocity_V_Target_Abs (ByVal AObj As Integer, ByVal Value As Long) As Long
Delphi	function MT_Set_Axis_Velocity_V_Target_Abs (AObj: Word; Value: Integer): Integer;

C#	public static extern int MT_Set_Axis_Velocity_V_Target_Abs (UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	Value	Set absolute target speed for specified axis Hz/s Positive or Negative
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Motor shaft serial numbers start from 0. For the Nth shaft card, serial numbers are 0, 1, 2, 3...N-1 Ensure motor axis number functions do not exceed bounds	

3.6.4 MT_Set_Axis_Velocity_V_Target_Rel

Function Description: Sets the relative velocity of the specified axis relative to the current speed in constant speed mode and **immediately initiates** constant speed operation. Use the open-loop coefficient for unit conversion. This function is only effective in constant speed mode and does not operate in other modes.

C/VC/GCC	int32_t MT_Set_Axis_Velocity_V_Target_Rel (WORD AObj, int32_t pValue);	
VB	Function MT_Set_Axis_Velocity_V_Target_Rel (ByVal AObj As Integer, ByVal Value As Long) As Long	
Delphi	function MT_Set_Axis_Velocity_V_Target_Rel(AObj: Word; Value: Integer): Integer;	
C#	public static extern int MT_Set_Axis_Velocity_V_Target_Rel(UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	Value	Set the relative speed of the specified axis relative to the current speed Hz/S Can be positive or negative
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Motor shaft serial numbers start from 0. For an N-axis card, serial numbers are 0, 1, 2, 3...N-1 Ensure the function for motor shaft serial numbers does not exceed bounds	

3.6.5 MT_Set_Axis_Velocity_Stop

Function Description: Stops the constant-speed motion mode of the specified axis, decelerating to a halt at the set deceleration rate. This function is only effective in zero constant-speed mode and does not work in other modes. For immediate emergency stop, use **MT_Set_Axis_Halt** or **MT_Set_Axis_Halt_All**.

C/VC/GCC	int32_t MT_Set_Axis_Velocity_Stop (WORD AObj)	
VB	Function MT_Set_Axis_Velocity_Stop (ByVal AObj As Integer) As Long	
Delphi	function MT_Set_Axis_Velocity_Stop (AObj:Word):Integer;	
C#	public static extern int MT_Set_Axis_Velocity_Stop (UInt16 AObj);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	None	
Function Return	0	Function Execution Success
	Non-zero	Function execution failed
Remarks	Motor shaft serial numbers start from 0. For N shafts, the serial numbers are 0, 1, 2, 3...N-1.	

	Functions utilizing motor shaft serial numbers must ensure they do not exceed the bounds.
--	---

3.6.6 MT_Set_Axis_Velocity_V_Start

Function Description: Sets the initial velocity for constant-speed mode startup on the specified axis. Default value is 50Hz/s. Generally, modifying this parameter **is prohibited**. For unit conversion, use the open-loop coefficient.

C/VC/GCC	int32_t MT_Set_Axis_Velocity_V_Start (WORD AObj, int32_t pValue);	
VB	Function MT_Set_Axis_Velocity_V_Start(ByVal AObj As Integer, ByVal Value As Long) As Long	
Delphi	function MT_Set_Axis_Velocity_V_Start (AObj:Word;Value:Integer):Integer;	
C#	public static extern int MT_Set_Axis_Velocity_V_Start (UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	Value	Set the start speed for the specified axis Hz/S Positive only
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Motor shaft serial numbers start from 0. For an N-axis card, serial numbers are 0, 1, 2, 3...N-1 Ensure the function for motor shaft serial numbers does not exceed bounds	

3.6.7 MT_Set_Axis_Velocity_Acc

Function Description: Sets the acceleration value for the constant-speed mode of the specified axis. Default value is 500Hz/s². Use the open-loop coefficient for unit conversion.

C/VC/GCC	int32_t MT_Set_Axis_Velocity_Acc (WORD AObj, int32_t pValue);	
VB	Function MT_Set_Axis_Velocity_Acc(ByVal AObj As Integer, ByVal Value As Long) As Long	
Delphi	function MT_Set_Axis_Velocity_Acc (AObj: Word; Value: Integer): Integer;	
C#	public static extern int MT_Set_Axis_Velocity_Acc (UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Motor control axis number
Output Parameters	Value	Set acceleration for specified axis Hz/S ² Positive Only
Function Return	0	Function execution successful
	Non-zero	Function execution failed

Remarks	Motor shaft serial numbers start from 0 . For an N-axis card, serial numbers are 0, 1, 2, 3...N-1 Ensure the function for motor shaft serial numbers does not exceed bounds
---------	--

3.6.8 MT_Set_Axis_Velocity_Dec

Function Description: Sets the deceleration value for constant-speed mode on the specified axis. Default value is 500Hz/s². Use the open-loop coefficient for unit conversion.

C/VC/GCC	int32_t MT_Set_Axis_Velocity_Dec (WORD AObj, int32_t pValue);	
VB	Function MT_Set_Axis_Velocity_Dec(ByVal AObj As Integer, ByVal Value As Long) As Long	
Delphi	function MT_Set_Axis_Velocity_Dec (AObj:Word;Value:Integer):Integer;	
C#	public static extern int MT_Set_Axis_Velocity_Dec(UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	Value	Set deceleration for specified axis Hz/S ² Positive Only
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Motor shaft serial numbers start from 0. For an N-axis card, the serial numbers are 0, 1, 2, 3...N-1 Ensure the motor shaft index function does not exceed bounds	

3.6.9 MT_Get_Axis_Velocity_Acc

Function Description: Reads the acceleration of the specified control axis in constant-speed mode. Use the open-loop coefficient for unit conversion.

C/VC/GCC	int32_t MT_Get_Axis_Velocity_Acc(WORD AObj, int32_t* pValue);	
VB	Function MT_Get_Axis_Velocity_Acc(ByVal AObj As Integer, ByRef Value As Long) As Long	
Delphi	function MT_Get_Axis_Velocity_Acc(AObj: Word; pValue: PInteger): Integer;	
C#	public static extern int MT_Get_Velocity_Acc(UInt16 AObj, ref Int32 Value);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	Value	Current acceleration of the specified axis Hz/S ²
Function Return	0	Function execution successful, data read is valid
	Non-zero	Function execution failed; the read data is invalid
Notes	Motor shaft serial numbers start from 0. For an N-axis card, serial numbers are 0, 1, 2, 3...N-1 Ensure motor axis number functions do not exceed bounds	

3.6.10 MT_Get_Axis_Velocity_Dec

Function Description: Reads the deceleration value for the specified control axis in constant-speed mode. Use the open-loop coefficient for unit conversion.

C/VC/GCC	int32_t MT_Get_Axis_Velocity_Dec(WORD AObj, int32_t* pValue);
----------	---

VB	Function MT_Get_Axis_Velocity_Dec(ByVal AObj As Integer, ByRef Value As Long) As Long	
Delphi	function MT_Get_Axis_Velocity_Dec(AObj: Word; pValue: PInteger): Integer;	
C#	public static extern int MT_Get_Velocity_Decc(UInt16 AObj, ref Int32 Value);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	Value	Current deceleration of the specified axis Hz/S ²
Function Return	0	Function execution successful, read data valid
	Non-zero	Function execution failed; the read data is invalid

Remarks	Motor shaft serial numbers start from 0. For an N-axis card, serial numbers are 0, 1, 2, 3...N-1 When using functions related to motor shaft serial numbers, ensure no out-of-bounds access occurs.
---------	--

3.7 Positioning Mode (Open-Loop and Closed-Loop)

3.7.1 MT_Set_Axis_Mode_Position[Compatible]

Function Description: Sets the specified axis to open-loop positioning mode, equivalent to **MT_Set_Axis_Mode_Position_Open**.

C/VC/GCC	int32_t MT_Set_Axis_Mode_Position (WORD AObj)	
VB	Function MT_Set_Axis_Mode_Position (ByVal AObj As Integer) As Long	
Delphi	function MT_Set_Axis_Mode_Position (AObj:Word):Integer;	
C#	public static extern int MT_Set_Axis_Mode_Position (UInt16 AObj);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	None	
Function Return	0	Function Execution Success
	Non-zero	Function execution failed
Remarks	Motor shaft numbers start from 0. For an N-axis card, the shaft numbers are 0, 1, 2, 3...N-1 Functions utilizing motor shaft serial numbers must avoid out-of-bounds access	

3.7.2 MT_Set_Axis_Mode_Position_Open

Function Description: Sets the specified axis to open-loop positioning mode. The controller will operate based on the number of pulses issued, executing actions within the open-loop logical coordinate system. In open-loop positioning mode, closed-loop sensor readings **function normally**, reflecting corresponding sensor changes.

C/VC/GCC	int32_t MT_Set_Axis_Mode_Position_Open (WORD AObj)	
VB	Function MT_Set_Axis_Mode_Position_Open (ByVal AObj As Integer) As Long	
Delphi	function MT_Set_Axis_Mode_Position_Open (AObj: Word): Integer;	
C#	public static extern int MT_Set_Axis_Mode_Position_Open (UInt16 AObj);	
Input Parameters	AObj	Motor control axis serial number

Output Parameters	None	
Function	0	Function Execution Success
Return	Non-zero	Function execution failed
Remarks	Motor shaft numbers start from 0 . For an N-axis card, the shaft numbers are 0, 1, 2, 3...N-1 Functions utilizing motor shaft serial numbers must avoid out-of-bounds access	

3.7.3 MT_Set_Axis_Mode_Position_Close

Function Description: Sets the specified axis to closed-loop positioning mode. The controller will use the received sensor pulse count as the reference standard to

The pulse counting in the open-loop coordinate system functions normally, reflecting the number of pulses emitted.

C/VC/GCC	int32_t MT_Set_Axis_Mode_Position_Close (WORD AObj)	
VB	Function MT_Set_Axis_Mode_Position_Close(ByVal AObj As Integer) As Long	
Delphi	function MT_Set_Axis_Mode_Position_Close(AObj: Word): Integer;	
C#	public static extern int MT_Set_Axis_Mode_Position_Close (UInt16 AObj);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	None	
Function Return	0	Function Execution Success
	Non-zero	Function execution failed
Remarks	Motor shaft serial numbers start from 0. For N shafts, the serial numbers are 0, 1, 2, 3...N-1. Functions utilizing motor shaft serial numbers must ensure they do not exceed the bounds.	

3.7.4 MT_Get_Axis_Position_V_Max

Function Description: Reads the maximum speed in positioning mode. This function **does not distinguish between closed-loop and open-loop modes**. For unit conversion, use the **open-loop conversion factor**.

C/VC/GCC	int32_t MT_Get_Axis_Position_V_Max (WORD AObj, int32_t* pValue);	
VB	Function MT_Get_Axis_Position_V_Max (ByVal AObj As Integer, ByRef Value As Long) As Long	
Delphi	function MT_Get_Axis_Position_V_Max (AObj: Word; pValue: PInteger): Integer;	
C#	public static extern int MT_Get_Axis_Position_V_Max (UInt16 AObj, ref Int32 Value);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	Value	Maximum speed in positioning mode for the specified axis Hz/S
Function Return	0	Function execution successful, read data valid
	Non-zero	Function execution failed; the read data is invalid
Notes	Motor shaft serial numbers start from 0. For an N-axis card, serial numbers are 0, 1, 2, 3...N-1 Ensure the motor shaft index function does not exceed bounds	

3.7.5 MT_Set_Axis_Position_V_Max

Function Description: Sets the maximum speed in positioning mode. This function does not distinguish between closed-loop and open-loop modes. For unit conversion, use the **open-loop** conversion **factor**. This function **does not** initiate positioning actions and differs from zero-positioning and constant-speed modes.

C/VC/GCC	int32_t MT_Set_Axis_Position_V_Max (WORD AObj, int32_t Value);	
VB	Function MT_Set_Axis_Position_V_Max (ByVal AObj As Integer, ByVal Value As Long) As Long	
Delphi	function MT_Set_Axis_Position_V_Max (AObj: Word; Value: Integer): Integer;	
C#	public static extern int MT_Set_Axis_Position_V_Max (UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Motor control axis serial number

Output Parameters	Value	Maximum speed in positioning mode for the specified axis Hz/S
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Motor shaft serial numbers start from 0. For an N-axis card, the serial numbers are 0, 1, 2, 3...N-1 Ensure the motor shaft index function does not exceed bounds	

3.7.6 MT_Get_Axis_Position_P_Target

Function Description: Reads the absolute target position in positioning mode. In open-loop mode, it returns the target open-loop pulse count, i.e., the coordinate value in the open-loop logical coordinate system. In closed-loop mode, it returns the target closed-loop sensor pulse count, i.e., the coordinate value in the closed-loop logical coordinate system. This function **distinguishes between closed-loop** and open-loop modes. For unit conversion in open-loop mode, use the open-loop conversion factor; for closed-loop mode, use the closed-loop conversion factor.

C/VC/GCC	int32_t MT_Get_Axis_Position_P_Target (WORD AObj, int32_t* pValue);	
VB	Function MT_Get_Axis_Position_P_Target (ByVal AObj As Integer, ByRef Value As Long) As Long	
Delphi	function MT_Get_Axis_Position_P_Target (AObj: Word; pValue: PInteger): Integer;	
C#	public static extern int MT_Get_Axis_Position_P_Target (UInt16 AObj, ref Int32 Value);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	Value	Absolute target position in positioning mode for the specified axis; can be positive or negative
Function Return	0	Function execution successful, read data valid
	Non-zero	Function execution failed; the read data is invalid.
Notes	Motor shaft serial numbers start from 0. For an N-axis card, serial numbers are 0, 1, 2, 3...N-1 Ensure the motor shaft index function does not exceed bounds	

3.7.7 MT_Set_Axis_Position_P_Target_Abs

Function Description: Sets the absolute target position in positioning mode. In open-loop mode, this represents the target open-loop pulse count, i.e., the coordinate value in the open-loop logical coordinate system. In closed-loop mode, it represents the target closed-loop sensor pulse count, i.e., the coordinate value in the closed-loop logical coordinate system. This function **distinguishes between closed-loop** and open-loop **modes**. For unit conversion in open-loop mode, use the open-loop conversion factor; for closed-loop mode, use the closed-loop conversion factor.

C/VC/GCC	int32_t MT_Set_Axis_Position_P_Target_Abs (WORD AObj, int32_t Value);	
VB	Function MT_Set_Axis_Position_P_Target_Abs (ByVal AObj As Integer, ByVal Value As Long) As Long	
Delphi	function MT_Set_Axis_Position_P_Target_Abs (AObj: Word; Value: Integer): Integer;	
C#	public static extern int MT_Set_Axis_Position_P_Target_Abs (UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	Value	Sets the absolute target position for the specified axis in positioning mode; can be positive or negative

Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Motor shaft serial numbers start from 0. For N shafts, the serial numbers are 0, 1, 2, 3...N-1 Ensure the motor shaft index function does not exceed bounds	

3.7.8 MT_Set_Axis_Position_P_Target_Rel

Function Description: Sets the relative target position (offset relative to the current position) in positioning mode. In open-loop mode, this represents the offset relative to the open-loop logical coordinate system; in closed-loop mode, it represents the offset relative to the closed-loop logical coordinate system. This function

distinguishes between closed-loop and open-loop modes. For unit conversion in open-loop mode, use the open-loop conversion factor; for closed-loop mode, use the closed-loop conversion factor.

C/VC/GCC	int32_t MT_Set_Axis_Position_P_Target_Rel (WORD AObj, int32_t Value);	
VB	Function MT_Set_Axis_Position_P_Target_Rel (ByVal AObj As Integer, ByVal Value As Long) As Long	
Delphi	function MT_Set_Axis_Position_P_Target_Rel (AObj:Word;Value:Integer):Integer;	
C#	public static extern int MT_Set_Axis_Position_P_Target_Rel (UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	Value	Sets the target movement amount relative to the current position in positioning mode for the specified axis. Can be positive Negative
Function Returns	0	Function execution successful
	Non-zero	Function execution failed
Remarks	The motor shaft serial numbers start from 0. For the Nth shaft, the serial numbers are 0, 1, 2, 3...N-1. When using the motor shaft index function, ensure that the index does not exceed the bounds.	

3.7.9 MT_Set_Axis_Position_Stop

Function Description: Stop the positional motion mode of the specified axis.

C/VC/GCC	int32_t MT_Set_Axis_Position_Stop (WORD AObj)
----------	---

VB	Function MT_Set_Axis_Position_Stop (ByVal AObj As Integer) As Long	
Delphi	function MT_Set_Axis_Position_Stop (AObj:Word):Integer;	
C#	public static extern int MT_Set_Axis_Position_Stop (UInt16 AObj);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	None	
Function Return	0	Function Execution Success
	Non-zero	Function execution failed
Remarks	Motor shaft numbers start from 0. For an N-axis card, the shaft numbers are 0, 1, 2, 3...N-1 Functions utilizing motor shaft serial numbers must avoid out-of-bounds access	

3.7.10 MT_Set_Axis_Position_V_Start

Function Description: Sets the initial speed for the positioning mode startup of the specified axis. Default is 50Hz/s. Generally, modification is prohibited.

C/VC/GCC	int32_t MT_Set_Axis_Position_V_Start (WORD AObj, int32_t pValue);	
VB	Function MT_Set_Axis_Position_V_Start(ByVal AObj As Integer, ByVal Value As Long) As Long	
Delphi	function MT_Set_Axis_Position_V_Start (AObj:Word;Value:Integer):Integer;	
C#	public static extern int MT_Set_Axis_Position_V_Start (UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	Value	Set the start-up speed Hz/S for the specified axis. Only positive values are allowed.
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Motor shaft serial numbers start from 0. For N shafts, the serial numbers are 0, 1, 2, 3...N-1 Ensure the motor shaft index function avoids out-of-bounds access Slower speeds enhance positioning accuracy	

3.7.11 MT_Set_Axis_Position_Acc

Function Description: Sets the acceleration value for the positioning mode of the specified axis. Default is 500Hz/s².

C/VC/GCC	int32_t MT_Set_Axis_Position_Acc (WORD AObj, int32_t pValue);	
VB	Function MT_Set_Axis_Position_Acc(ByVal AObj As Integer, ByVal Value As Long) As Long	
Delphi	function MT_Set_Axis_Position_Acc (AObj:Word;Value:Integer):Integer;	
C#	public static extern int MT_Set_Axis_Position_Acc (UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	Value	Set the acceleration of the specified axis in Hz/s ² , positive values only
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Motor shaft serial numbers start from 0. For N shafts, the serial numbers are 0, 1, 2, 3...N-1 Ensure the motor shaft index function does not exceed bounds	

3.7.12 MT_Set_Axis_Position_Dec

Function Description: Sets the deceleration value for the positioning mode of the specified axis. Default value is 500Hz/s².

C/VC/GCC	int32_t MT_Set_Axis_Position_Dec (WORD AObj, int32_t pValue);
VB	Function MT_Set_Axis_Position_Dec(ByVal AObj As Integer, ByVal Value As Long) As Long

Delphi	function MT_Set_Axis_Position_Dec (AObj:Word;Value:Integer):Integer;	
C#	public static extern int MT_Set_Axis_Position_Dec (UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Motor control axis number
Output Parameters	Value	Set the deceleration rate in Hz/s ² for the specified axis. Only positive values are allowed.
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Motor shaft serial numbers start from 0. For an N-axis card, the serial numbers are 0, 1, 2, 3...N-1 Ensure the motor shaft index function does not exceed bounds	

3.7.13 MT_Get_Axis_Position_Acc

Function Description: Reads the acceleration of the specified control axis in positioning mode.

C/VC/GCC	int32_t MT_Get_Axis_Position_Acc(WORD AObj, int32_t* pValue);	
VB	Function MT_Get_Axis_Position_Acc(ByVal AObj As Integer, ByRef Value As Long) As Long	
Delphi	function MT_Get_Axis_Position_Acc(AObj: Word; pValue: PInteger): Integer;	
C#	public static extern int MT_Get_Position_Acc (UInt16 AObj, ref Int32 Value);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	Value	Current acceleration of the specified axis Hz/S ²
Function Return	0	Function execution successful, data read is valid
	Non-zero	Function execution failed; the read data is invalid
Notes	Motor shaft serial numbers start from 0. For an N-axis card, serial numbers are 0, 1, 2, 3...N-1 Ensure the motor shaft index function does not exceed bounds	

3.7.14 MT_Get_Axis_Position_Dec

Function Description: Reads the deceleration value for the specified control axis positioning mode.

C/VC/GCC	int32_t MT_Get_Axis_Position_Dec(WORD AObj, int32_t* pValue);	
VB	Function MT_Get_Axis_Position_Dec(ByVal AObj As Integer, ByRef Value As Long) As Long	
Delphi	function MT_Get_Axis_Position_Dec(AObj: Word; pValue: PInteger): Integer;	
C#	public static extern int MT_Get_Position_Decc (UInt16 AObj, ref Int32 Value);	

Input Parameters	AObj	Motor control axis serial number
Output Parameters	Value	Current deceleration of the specified axis Hz/S ²
Function returns	0	Function execution successful, read data is valid
	Non-zero	Function execution failed; the read data is invalid
Notes	Motor shaft serial numbers start from 0 . For an N-axis card, serial numbers are 0, 1, 2, 3...N-1 Ensure the motor shaft index function does not exceed bounds	

3.7.15 MT_Set_Axis_Position_Close_Dec_Factor

Function Description: Sets the deceleration factor for closed-loop positioning mode of the specified axis. Default value is 1. This parameter **must** be set once before performing closed-loop positioning motion to ensure the controller executes automatic deceleration correctly. Failure to do so may result in premature or delayed deceleration during closed-loop operation and unsteady stopping. **Deceleration Factor = Open-Loop Coefficient / Closed-Loop Coefficient**. Generally, this ratio must be greater than 1; otherwise, precise closed-loop positioning may not be achievable.

C/VC/GCC	int32_t MT_Set_Axis_Position_Close_Dec_Factor (WORD AObj, float pValue);	
VB	Function MT_Set_Axis_Position_Close_Dec_Factor(ByVal AObj As Integer, ByVal Value As Single) As Long	
Delphi	function MT_Set_Axis_Position_Dec (AObj:Word;Value:Single):Integer;	
C#	public static extern int MT_Set_Axis_Position_Dec (UInt16 AObj, float Value);	
Input Parameters	AObj	Motor control axis number
Output Parameters	Value	Sets the closed-loop deceleration coefficient for the specified axis. Only positive floating-point numbers are allowed.
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Motor shaft serial numbers start from 0. For N shafts, the serial numbers are 0, 1, 2, 3...N-1 Ensure the motor shaft index function does not exceed bounds	

3.7.16 MT_Set_Encoder_Over_Enable

Function Description: Enables automatic compensation during closed-loop motion for the specified axis. Default is enabled. When enabled, if the controller fails to precisely stop at the target position on the first attempt, it will automatically adjust repeatedly based on sensor readings to achieve accurate positioning.

C/VC/GCC	int32_t MT_Set_Encoder_Over_Enable (WORD AObj, int32_t pValue);	
VB	Function MT_Set_Encoder_Over_Enable(ByVal AObj As Integer, ByVal Value As Long) As Long	
Delphi	function MT_Set_Encoder_Over_Enable (AObj:Word;Value:Integer):Integer;	
C#	public static extern int MT_Set_Encoder_Over_Enable (UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Closed-loop control motor control axis serial number
Output Parameters	Value	1: Enable compensation function 0: Disable compensation function
Function	0	Function execution successful

Return	Non-zero	Function execution failed
Remarks	Motor shaft serial numbers start from 0. For N shafts, the serial numbers are 0, 1, 2, 3...N-1 Ensure the motor shaft index function does not exceed bounds	

3.7.17 MT_Set_Encoder_Over_Max

Function Description: After enabling closed-loop compensation, this parameter determines the maximum number of compensation steps. The default value is **100**. If the number of steps (pulses) required for back-and-forth adjustment exceeds this value and precise positioning is still unachievable, the closed-loop compensation function will be disabled.

exceeds this value without achieving precise positioning, the closed-loop compensation function will stop.

C/VC/GCC	int32_t MT_Set_Encoder_Over_Max (WORD AObj, int32_t pValue);	
VB	Function MT_Set_Encoder_Over_Max(ByVal AObj As Integer, ByVal Value As Long) As Long	
Delphi	function MT_Set_Encoder_Over_Max (AObj:Word;Value:Integer):Integer;	
C#	public static extern int MT_Set_Encoder_Over_Max (UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Closed-loop control motor control axis serial number
Output Parameters	Value	Compensation step count, must be positive
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Motor shaft serial numbers start from 0. For an N-axis card, the serial numbers are 0, 1, 2, 3...N-1 Ensure the motor shaft index function does not exceed bounds	

3.7.18 MT_Set_Encoder_Over_Stable

Function Description: After enabling closed-loop compensation, this parameter determines the stability threshold for sensor readings. Default value is 50. During iterative adjustments, if sensor readings remain unchanged for consecutive cycles exceeding this value, compensation will cease.

C/VC/GCC	int32_t MT_Set_Encoder_Over_Stable (WORD AObj, int32_t pValue);	
VB	Function MT_Set_Encoder_Over_Stable(ByVal AObj As Integer, ByVal Value As Long) As Long	
Delphi	function MT_Set_Encoder_Over_Stable (AObj:Word;Value:Integer):Integer;	
C#	public static extern int MT_Set_Encoder_Over_Stable (UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Closed-loop control motor control axis number
Output Parameters	Value	Compensation step count, must be positive
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Motor shaft serial numbers start from 0. For an N-axis card, the serial numbers are 0, 1, 2, 3...N-1 Ensure the motor shaft index function does not exceed bounds	

3.7.19 MT_Set_Encoder_Z_Polarity

Function Description: Sets the level definition for the Z signal of the encoder/ruler interface. Used when determining the zero position using the Z signal.

(*MT_Set_Axis_Mode_Home_ Encoder_Index*) Typically no configuration is required. Some controllers do not support the Z phase; refer to the relevant wiring manual. This parameter resets to default values after power loss; reconfigure after powering on.

C/VC/GCC	int32_t MT_Set_Encoder_Z_Polarity (WORD AObj, int32_t Value);
VB	Function MT_Set_Encoder_Z_Polarity (ByVal AObj As Integer, ByVal Value As Long) As Long
Delphi	function MT_Set_Encoder_Z_Polarity (AObj: Word; Value: Integer): Integer;

C#	public static extern int MT_Set_Encoder_Z_Polarity (UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Encoder/Ruler Interface Serial Number
Output Parameters	Value	0: Normal Level 1: Inverted Level
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Encoder/Ruler Interface Serial Numbers start at 0 . For an N-axis card, serial numbers are 0, 1, 2, 3...N-1 Ensure the function for the encoder/ruler interface serial number does not exceed bounds	

3.7.20 MT_Set_Encoder_Dir_Polarity

Function Description: Sets the counting direction for the encoder/ruler interface. Typically no configuration is required. When the motor rotation direction differs from the sensor counting direction, this function can be used to adjust it. This parameter resets to default values after power loss. Please configure it after power-up. It is recommended to resolve inconsistencies by adjusting the motor direction.

C/VC/GCC	int32_t MT_Set_Encoder_Dir_Polarity (WORD AObj, int32_t Value);	
VB	Function MT_Set_Encoder_Dir_Polarity (ByVal AObj As Integer, ByVal Value As Long) As Long	
Delphi	function MT_Set_Encoder_Z_Polarity (AObj:Word;Value:Integer):Integer;	
C#	public static extern int MT_Set_Encoder_Z_Polarity (UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Encoder/Ruler Interface Serial Number
Output Parameters	Value	0: Normal Direction 1: Reverse Direction
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Encoder/Ruler Interface Serial Numbers start at 0 . For an N-axis card, serial numbers are 0, 1, 2, 3...N-1 Ensure functions referencing encoder/ruler interface serial numbers do not exceed bounds	

3.8 Software limit

3.8.1 MT_Set_Axis_Software_Limit_Neg_Value

Function Description: Sets the coordinate value for the negative limit of the software limit function in positioning mode. When software limits are enabled, movement stops if the negative direction exceeds this value, mimicking the effect of hitting a hardware limit switch.

C/VC/GCC	int32_t MT_Set_Axis_Software_Limit_Neg_Value (WORD AObj, int32_t Value);
VB	Function MT_Set_Axis_Software_Limit_Neg_Value (ByVal AObj As Integer, ByVal Value As Long) As Long
Delphi	function MT_Set_Axis_Software_Limit_Neg_Value

	(AObj: Word; Value: Integer): Integer;	
C#	public static extern int MT_Set_Axis_Software_Limit_Neg_Value (UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Motor Control Shaft Serial Number
Output Parameters	Value	Sets the software lower limit value for the specified axis in positioning mode; can be positive or negative
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Motor shaft serial numbers start from 0. For N shafts, the serial numbers are 0, 1, 2, 3...N-1 Ensure the motor shaft index function does not exceed bounds. This value is effective only when software limits are enabled. The negative soft limit value must be less than the positive soft limit value.	

3.8.2 MT_Set_Axis_Software_Limit_Pos_Value

Function Description: Sets the positive limit value for software limits in positioning mode. When software limits are enabled, movement stops if the positive direction exceeds this value, mimicking the effect of hitting a hardware limit switch.

C/VC/GCC	int32_t MT_Set_Axis_Software_Limit_Pos_Value (WORD AObj, int32_t Value);	
VB	Function MT_Set_Axis_Software_Limit_Pos_Value (ByVal AObj As Integer, ByVal Value As Long) As Long	
Delphi	function MT_Set_Axis_Software_Limit_Pos_Value (AObj: Word; Value: Integer): Integer;	
C#	public static extern int MT_Set_Axis_Software_Limit_Pos_Value (UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	Value	Sets the software positive limit value for the specified axis in positioning mode; can be positive or negative
Function Return	0	Function execution successful
	Non-zero	Function execution failed

Remarks	Motor shaft serial numbers start from 0. For N shafts, the serial numbers are 0, 1, 2, 3...N-1 Ensure the motor shaft serial number function does not exceed bounds. This value is only effective when software limits are enabled. Negative software limit values must be smaller than positive software limit values
---------	--

3.8.3 MT_Set_Axis_Software_Limit_Enable

Function Description: Enables the software limit mode for the specified axis.

C/VC/GCC	int32_t MT_Set_Axis_Software_Limit_Enable (WORD AObj)
VB	Function MT_Set_Axis_Software_Limit_Enable (ByVal AObj As Integer) As Long
Delphi	function MT_Set_Axis_Software_Limit_Enable (AObj:Word):Integer;
C#	public static extern int MT_Set_Axis_Software_Limit_Enable (UInt16 AObj);

Input Parameters	AObj	Motor control axis serial number
Output Parameters	None	
Function Return	0	Function Execution Success
	Non-zero	Function execution failed
Remarks	Motor shaft numbers start from 0. For an N-axis card, the shaft numbers are 0, 1, 2, 3...N-1 Functions utilizing motor axis serial numbers must avoid out-of-bounds access	

3.8.4 MT_Set_Axis_Software_Limit_Disable

Function Description: Disables software limit mode for the specified axis.

C/VC/GCC	int32_t MT_Set_Axis_Software_Limit_Disable (WORD AObj)	
VB	Function MT_Set_Axis_Software_Limit_Disable (ByVal AObj As Integer) As Long	
Delphi	function MT_Set_Axis_Software_Limit_Disable (AObj:Word):Integer;	
C#	public static extern int MT_Set_Axis_Software_Limit_Disable (UInt16 AObj);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	None	
Function Returns	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Motor shaft numbers start from 0. For an N-axis card, the shaft numbers are 0, 1, 2, 3...N-1 Functions utilizing motor shaft serial numbers must avoid out-of-bounds access	

3.9 Status-related functions

3.9.1 MT_Get_Axis_Num

Function Description: Reads the number of motor control axes in the controller.

C/VC/GCC	int32_t MT_Get_Axis_Num (int32_t* pValue)	
VB	Function MT_Get_Axis_Num (ByRef Value As Long) As Long	
Delphi	function MT_Get_Axis_Num (pValue:PInteger):Integer;	
C#	public static extern int MT_Get_Axis_Num (ref Int32 Value);	

Input Parameters	None	
Output Parameters	Value	Number of motors supported by the controller 0-N
Function Return	0	Function execution successful, number of axes read is valid
	Non-zero	Function execution failed; the number of axes read is invalid
Notes	Motor shaft numbers start from 0 . For an N-axis card, the shaft numbers are 0, 1, 2, 3...N-1 Functions requiring motor shaft numbers later must ensure they do not exceed the bounds.	

3.9.2 MT_Get_Encoder_Num

Function Description: Reads the number of closed-loop interface axes (linear encoders or encoders) on the controller.

C/VC/GCC	int32_t MT_Get_Encoder_Num (int32_t* pValue)	
VB	Function MT_Get_Encoder_Num (ByRef Value As Long) As Long	
Delphi	function MT_Get_Encoder_Num (pValue:Plnteger):Integer;	
C#	public static extern int MT_Get_Encoder_Num (ref Int32 Value);	
Input Parameters	None	
Output Parameters	Value	Number of closed-loop interfaces supported by the controller 0-N
Function Return	0	Function execution successful, number of axes read is valid
	Non-zero	Function execution failed; the number of axes read is invalid
Notes	Motor shafts are numbered starting from 0 . For an N-axis card, the shaft numbers are 0, 1, 2, 3...N-1 Functions requiring closed-loop interface IDs later must avoid out-of-bounds access	

3.9.3 MT_Get_Axis_Mode

Function Description: Reads the current operating mode of the specified axis on the controller.

C/VC/GCC	int32_t MT_Get_Axis_Mode(WORD AObj, int32_t* pValue)	
VB	Function MT_Get_Axis_Mode(ByVal AObj As Integer, ByRef Value As Long) As Long	
Delphi	function MT_Get_Axis_Mode(AObj: Word; pValue: Plnteger): Integer;	
C#	public static extern int MT_Get_Axis_Mode(UInt16 AObj, ref Int32 Value);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	Value	Current operating mode of the controller-specified axis 0: Zero Position Mode 1: Constant Speed Mode 2: Positioning Mode (Default mode after power-on) 3: Linear Interpolation 4: Circular Interpolation
Function Returns	0	Function executed successfully; acceleration data read is valid
	Non-zero	Function execution failed; the read acceleration

		value is invalid
Notes	Motor shaft serial numbers start from 0. For an N-axis card, serial numbers are 0, 1, 2, 3...N-1 Ensure the motor shaft index function does not exceed bounds. Operations become invalid if the index is out of bounds.	

3.9.4 MT_Get_Axis_Acc[Compatible]

Function Description: Reads the current acceleration of the specified motor control axis from the controller.

C/VC/GCC	int32_t MT_Get_Axis_Acc(WORD AObj, int32_t* pValue)
----------	---

VB	Function MT_Get_Axis_Acc(ByVal AObj As Integer, ByRef Value As Long) As Long	
Delphi	function MT_Get_Axis_Acc(AObj: Word; pValue: PInteger): Integer;	
C#	public static extern int MT_Get_Axis_Acc(UInt16 AObj, ref Int32 Value);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	Value	Current acceleration value of the controller-specified axis, in $\text{Hz/s}^2 \geq 0$
Function Return	0	Function execution successful, acceleration reading valid
	Non-zero	Function execution failed; the read acceleration value is invalid
Notes	Motor shaft numbers start from 0. For an N-axis card, the shaft numbers are 0, 1, 2, 3...N-1 Ensure the motor shaft index function does not exceed bounds. Operations become invalid if the index is out of bounds.	

3.9.5 MT_Set_Axis_Acc[Compatible]

Function Description: Sets the acceleration for the specified motor control axis on the controller.

C/VC/GCC	int32_t MT_Set_Axis_Acc(WORD AObj, int32_t Value)	
VB	Function MT_Set_Axis_Acc(ByVal AObj As Integer, ByVal Value As Long) As Long	
Delphi	function MT_Set_Axis_Acc(AObj: Word; Value: Integer): Integer;	
C#	public static extern int MT_Set_Axis_Acc(UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	Value	Sets the acceleration value for the specified axis of the controller, in $\text{Hz/s}^2 \geq 0$
Function Return	0	Function execution successful; current setting operation successful
	Non-zero	Function execution failed; the current setting operation failed
Remarks	Motor shaft numbers start from 0. For an N-axis card, the shaft numbers are 0, 1, 2, 3...N-1 Ensure the motor shaft number function does not exceed bounds. If an out-of-bounds index is used, the operation may appear successful but the data will be discarded and invalid	

3.9.6 MT_Get_Axis_Dec [Compatible]

Function Description: Reads the current deceleration of the specified motor control axis from the controller.

C/VC/GCC	int32_t MT_Get_Axis_Dec(WORD AObj, int32_t* pValue)	
VB	Function MT_Get_Axis_Dec(ByVal AObj As Integer, ByRef Value As Long) As Long	
Delphi	function MT_Get_Axis_Dec(AObj: Word; pValue: PInteger): Integer;	
C#	public static extern int MT_Get_Axis_Dec(UInt16 AObj, ref Int32 Value);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	Value	Current deceleration value of the controller-specified axis, in $\text{Hz/s}^2 \geq 0$
Function Return	0	Function execution successful, read deceleration valid
	Non-zero	Function execution failed; the read deceleration value is invalid
Notes	Motor shaft numbers start from 0. For an N-axis card, the shaft numbers are 0, 1, 2, 3...N-1	

	Ensure motor shaft number functions do not exceed bounds If the index is out of bounds, the current operation is invalid
--	---

3.9.7 MT_Set_Axis_Dec[Compatible]

Function Description: Sets the acceleration for the specified motor control axis on the controller.

C/VC/GCC	int32_t MT_Set_Axis_Dec(WORD AObj, int32_t Value)	
VB	Function MT_Set_Axis_Dec(ByVal AObj As Integer, ByVal Value As Long) As Long	
Delphi	function MT_Set_Axis_Dec(AObj: Word; Value: Integer): Integer;	
C#	public static extern int MT_Set_Axis_Dec(UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Motor control axis number
Output Parameters	Value	Sets the deceleration value for the specified axis of the controller, in Hz/s ² >=0
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Motor shaft serial numbers start from 0. For N shafts, the serial numbers are 0, 1, 2, 3...N-1 Ensure the motor shaft serial number function does not exceed bounds If an out-of-bounds index is used, the operation may succeed but the data will be discarded and invalid	

3.9.8 MT_Get_Axis_V_Now

Function Description: Reads the current real-time speed of the specified control axis.

C/VC/GCC	int32_t MT_Get_Axis_V_Now(WORD AObj, int32_t* pValue);	
VB	Function MT_Get_Axis_V_Now (ByVal AObj As Integer, ByRef Value As Long) As Long	
Delphi	function MT_Get_Axis_V_Now(AObj: Word; pValue: PInteger): Integer;	
C#	public static extern int MT_Get_Axis_V_Now (UInt16 AObj, ref Int32 Value);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	Value	Current speed of specified axis Hz/S
Function Return	0	Function execution successful, read data valid
	Non-zero	Function execution failed; the read data is invalid

Notes	Motor shaft serial numbers start from 0. For an N-axis card, serial numbers are 0, 1, 2, 3...N-1 Ensure the motor shaft number function does not exceed bounds
-------	---

3.9.9 MT_Get_Axis_P_Now

Function Description: Reads the current open-loop coordinate position of the specified control axis.

C/VC/GCC	int32_t MT_Get_Axis_P_Now(WORD AObj, int32_t* pValue)
----------	---

VB	Function MT_Get_Axis_P_Now (ByVal AObj As Integer, ByRef Value As Long) As Long	
Delphi	function MT_Get_Axis_P_Now (AObj: Word; pValue: PInteger): Integer;	
C#	public static extern int MT_Get_Axis_P_Now(UInt16 AObj, ref Int32 Value);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	Value	Current software position of the specified axis in steps
Function Return	0	Function execution successful, read data is valid
	Non-zero	Function execution failed; the read data is invalid
Notes	Motor shaft serial numbers start from 0. For an N-axis card, the serial numbers are 0, 1, 2, 3...N-1 Ensure the motor shaft index function does not exceed bounds	

3.9.10 MT_Get_Axis_Software_P_Now [Compatible]

Function Description: Reads the current open-loop coordinate position of the specified control axis. Equivalent to *MT_Get_Axis_P_Now*.

C/VC/GCC	int32_t MT_Get_Axis_Software_P_Now(WORD AObj, int32_t* pValue)	
VB	Function MT_Get_Axis_Software_P_Now (ByVal AObj As Integer, ByRef Value As Long) As Long	
Delphi	function MT_Get_Axis_Software_P_Now (AObj:Word;pValue:PInteger):Integer;	
C#	public static extern int MT_Get_Axis_Software_P_Now(UInt16 AObj, ref Int32 Value);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	Value	Current software position of the specified axis in steps
Function Return	0	Function execution successful, read data is valid
	Non-zero	Function execution failed; the read data is invalid
Notes	Motor shaft serial numbers start from 0. For an N-axis card, serial numbers are 0, 1, 2, 3...N-1 Ensure the motor shaft number function does not exceed bounds	

3.9.11 MT_Get_Axis_Software_P [Compatible]

Function Description: Reads the current open-loop coordinate position of the specified control axis. Equivalent to *MT_Get_Axis_P_Now*.

C/VC/GCC	int32_t MT_Get_Axis_Software_P(WORD AObj, int32_t* pValue)	
VB	Function MT_Get_Axis_Software_P (ByVal AObj As Integer, ByRef Value As Long) As Long	

Delphi	function MT_Get_Axis_Software_P(AObj: Word; pValue: PInteger): Integer;	
C#	public static extern int MT_Get_Axis_Software_P(UInt16 AObj, ref Int32 Value);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	Value	Current software position of the specified axis in steps
Function Return	0	Function execution successful, read data is valid
	Non-zero	Function execution failed; the read data is invalid
Remarks	The motor shaft serial numbers start from 0. For the Nth shaft, the serial numbers are 0, 1, 2, 3...N-1. When using the motor shaft index function, ensure that the index does not exceed the bounds.	

3.9.12 MT_Set_Axis_P_Now

Function Description: Sets the current open-loop coordinate value for the specified control axis to adjust the coordinates.

C/VC/GCC	int32_t MT_Set_Axis_Software_P_Now(WORD AObj, int32_t Value)	
VB	Function MT_Set_Axis_Software_P_Now (ByVal AObj As Integer, ByVal Value As Long) As Long	
Delphi	function MT_Set_Axis_Software_P (AObj:Word;Value:Integer):Integer;	
C#	public static extern int MT_Set_Axis_Software_P (UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Motor control axis number
Output Parameters	Value	Current software position of the specified axis in steps
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Motor shaft serial numbers start from 0. For N shafts, the serial numbers are 0, 1, 2, 3...N-1 Ensure motor axis number functions do not exceed bounds	

3.9.13 MT_Set_Axis_Software_P [Compatible]

Function Description: Sets the current open-loop coordinate position for the specified control axis, equivalent to *MT_Set_Axis_P_Now*.

C/VC/GCC	int32_t MT_Set_Axis_Software_P(WORD AObj, int32_t Value)	
VB	Function MT_Set_Axis_Software_P(ByVal AObj As Integer, ByVal Value As Long) As Long	
Delphi	function MT_Set_Axis_Software_P(AObj: Word; Value: Integer): Integer;	
C#	public static extern int MT_Set_Axis_Software_P(UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	Value	Current software position of the specified axis in steps
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Motor shaft serial numbers start from 0. For an N-axis card, the serial numbers are 0, 1, 2, 3...N-1 Ensure motor axis number functions do not exceed bounds	

3.9.14 MT_Set_Axis_Software_P_Now [Compatible]

Function Description: Sets the current open-loop coordinate position for the specified control axis, equivalent to *MT_Set_Axis_P_Now*.

C/VC/GCC	int32_t MT_Set_Axis_Software_P_Now(WORD AObj, int32_t Value)
VB	Function MT_Set_Axis_Software_P_Now(ByVal AObj As Integer, ByVal Value As Long) As Long

Delphi	function MT_Set_Axis_Software_P_Now(AObj:Word;Value:Integer):Integer;	
C#	public static extern int MT_Set_Axis_Software_P_Now(UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	Value	Current software position of the specified axis in steps
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Motor shaft serial numbers start from 0. For an N-axis card, serial numbers are 0, 1, 2, 3...N-1 Ensure the motor axis number function does not exceed bounds	

3.9.15 MT_Get_Axis_Status[Compatible]

Function Description: Reads the current status information of the specified control axis

C/VC/GCC	int32_t MT_Get_Axis_Status(WORD AObj, BYTE* pRun, BYTE* pDir, BYTE* pNeg, BYTE* pPos, BYTE* pZero, BYTE* pMode);	
VB	Function MT_Get_Axis_Status Lib "MT_API.dll" _ (ByVal AObj As Integer, ByRef Run As Byte, ByRef Dir As Byte, _ ByRef Neg As Byte, ByRef Pos As Byte, ByRef Zero As Byte, ByRef Mode As Byte) As Long	
Delphi	function MT_Get_Axis_Status(AObj: Word; pRun: PByte; pDir: PByte; pNeg: PByte; pPos: PByte; pZero: PByte; pMode: PByte): Integer;	
C#	public static extern int MT_Get_Axis_Status(WORD AObj, ref BYTE Run, ref BYTE Dir, ref BYTE Neg, ref BYTE Pos, ref BYTE Zero, ref BYTE Mode);	
Input Parameters	AObj	Motor control axis serial number
Output	Run	Current motion status of the specified axis: 1 indicates motion, 0 indicates no motion

Parameters	Dir	Specifies the current direction of the axis: 1 for positive direction, 0 for negative direction
	Neg	Specifies the current negative limit for the axis. 1 indicates negative limit enabled, 0 indicates negative limit disabled
	Pos	Specifies the current positive limit for the axis. 1 indicates positive limit enabled, 0 indicates positive limit disabled
	Zero	Specifies the current zero position for the axis. 1 indicates zero position is enabled, 0 indicates zero position is disabled.
	Mode	Operating mode: 0 = zero mode, 1 = constant speed mode, 2 = positioning mode
Function Return	0	Function execution successful, read data is valid.

	Non-zero	Function execution failed; read data is invalid
Notes	Motor shaft serial numbers start from 0 . For an N-axis card, serial numbers are 0, 1, 2, 3...N-1 Ensure the motor shaft number function does not exceed bounds	

3.9.16 MT_Get_Axis_Status2

Function Description: Reads the current status information of the specified control axis.

C/VC/GCC	<pre>int32_t MT_Get_Axis_Status2(WORD AObj, int32_t* pRun, int32_t* pDir, int32_t* pNeg, int32_t* pPos, int32_t* pZero, int32_t* pMode);</pre>	
VB	<pre>Function MT_Get_Axis_Status2 Lib "MT_API.dll" _ (ByVal AObj As Integer, ByRef Run As Integer, ByRef Dir As Integer, _ ByRef Neg As Integer, ByRef Pos As Integer, ByRef Zero As Integer, ByRef Mode As Integer) As Long</pre>	
Delphi	<pre>function MT_Get_Axis_Status2(AObj: Word; pRun: PInteger; pDir: PInteger; pNeg: PInteger; pPos: PInteger; pZero: PInteger; pMode: PInteger): Integer;</pre>	
C#	<pre>public static extern int MT_Get_Axis_Status2(WORD AObj, ref Int32 Run, ref Int32 Dir, ref Int32 Neg, ref Int32 Pos, ref Int32 Zero, ref Int32 Mode);</pre>	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	Run	Current motion status of the specified axis: 1 indicates motion, 0 indicates no motion
	Dir	Specifies the current direction of the axis: 1 for positive direction, 0 for negative direction
	Neg	Specifies the current negative limit for the axis. 1 indicates negative limit enabled, 0 indicates negative limit disabled
	Pos	Specifies the current positive limit for the axis. 1 indicates positive limit enabled, 0 indicates positive

		limit disabled
	Zero	Specifies the current zero position for the axis. 1 indicates zero position is enabled, 0 indicates zero position is disabled.
	Mode	Operating mode: 0 = zero mode, 1 = constant speed mode, 2 = positioning mode
Function	0	Function execution successful, read data is valid.
Return	Non-zero	Function execution failed; read data is invalid
Notes	Motor shaft serial numbers start from 0 . For an N-axis card, serial numbers are 0, 1, 2, 3...N-1 Ensure the motor shaft number function does not exceed bounds	

3.9.17 MT_Get_Axis_Status_Run

Function Description: Reads the current run status of the specified control axis. This is the single-status version of *MT_Get_Axis_Status2*.

C/VC/GCC	int32_t MT_Get_Axis_Status_Run(WORD AObj, int32_t* pRun);	
VB	Function MT_Get_Axis_Status_Run Lib "MT_API.dll" _ (ByVal AObj As Integer, ByRef Run As Integer) As Long	
Delphi	function MT_Get_Axis_Status_Run(AObj: Word; pRun: PInteger): Integer;	
C#	public static extern int MT_Get_Axis_Status_Run(WORD AObj, ref Int32 Run);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	Run	Current motion status of the specified axis: 1 indicates motion, 0 indicates no motion
Function Return	0	Function execution successful, read data valid
	Non-zero	Function execution failed; the read data is invalid
Remarks	The motor shaft serial numbers start from 0 . For the Nth shaft, the serial numbers are 0, 1, 2, 3...N-1 . When using functions for motor shaft serial numbers, ensure they do not exceed the bounds.	

3.9.18 MT_Get_Axis_Status_Dir

Function Description: Reads the current running direction of the specified control axis. This is the single-item status version of *MT_Get_Axis_Status2*.

C/VC/GCC	int32_t MT_Get_Axis_Status_Dir(WORD AObj, int32_t* pDir);	
VB	Function MT_Get_Axis_Status_Dir Lib "MT_API.dll" _ (ByVal AObj As Integer, ByRef Dir As Integer) As Long	
Delphi	function MT_Get_Axis_Status_Dir(AObj: Word; pDir: PInteger): Integer;	
C#	public static extern int MT_Get_Axis_Status_Dir(WORD AObj, ref Int32 Dir);	
Input Parameters	AObj	Motor control axis serial number
	Dir	Specifies the current direction of the axis. 1 indicates positive direction, 0 indicates negative direction
Function Return	0	Function executed successfully, read data is valid
	Non-zero	Function execution failed; the read data is invalid

Notes	Motor shaft serial numbers start from 0. For an N-axis card, serial numbers are 0, 1, 2, 3...N-1 Ensure the motor shaft number function does not exceed bounds
-------	---

3.9.19 MT_Get_Axis_Status_Neg

Function Description: Reads the current negative limit status of the specified control axis. This is the single-item status version of *MT_Get_Axis_Status2*.

C/VC/GCC	int32_t MT_Get_Axis_Status_Neg(WORD AObj, int32_t* pNeg);
----------	---

VB	Function MT_Get_Axis_Status_Neg Lib "MT_API.dll" _ (ByVal AObj As Integer, ByRef Neg As Integer) As Long	
Delphi	function MT_Get_Axis_Status_Neg(AObj:Word; pNeg: PInteger): Integer;	
C#	public static extern int MT_Get_Axis_Status_Neg(WORD AObj, ref Int32 Neg);	
Input Parameters	AObj	Motor control axis serial number
	Neg	Specifies the current negative limit for the axis. 1 indicates the negative limit is enabled, 0 indicates the negative limit is disabled
Function Return	0	Function executed successfully, read data is valid
	Non-zero	Function execution failed; the read data is invalid.
Remarks	Motor shaft serial numbers start from 0 . For an N-axis card, serial numbers are 0, 1, 2, 3...N-1 Ensure the motor shaft number function does not exceed bounds	

3.9.20 MT_Get_Axis_Status_Pos

Function Description: Reads the current positive limit status of the specified control axis. This is the single-item status version of *MT_Get_Axis_Status2*.

C/VC/GCC	int32_t MT_Get_Axis_Status_Pos(WORD AObj, int32_t* pPos);	
VB	Function MT_Get_Axis_Status_Pos Lib "MT_API.dll" _ (ByVal AObj As Integer, ByRef Pos As Integer) As Long	
Delphi	function MT_Get_Axis_Status_Pos(AObj:Word; pPos: PInteger): Integer;	
C#	public static extern int MT_Get_Axis_Status_Pos(WORD AObj, ref Int32 Pos);	
Input Parameters	AObj	Motor Control Shaft Serial Number
	Pos	Specifies the current positive limit for the axis. 1 indicates positive limit enabled, 0 indicates positive limit disabled
Function Return	0	Function executed successfully; read data is valid
	Non-zero	Function execution failed; the read data is invalid.
Notes	Motor shaft serial numbers start from 0 . For an N-axis card, serial numbers are 0, 1, 2, 3...N-1 Ensure the motor shaft number function does not exceed bounds	

3.9.21 MT_Get_Axis_Status_Zero

Function Description: Reads the current zero position status of the specified

control axis. This is the single-axis status version of *MT_Get_Axis_Status2*.

C/VC/GCC	<code>int32_t MT_Get_Axis_Status_Zero(WORD AObj, int32_t* pZero);</code>
VB	<code>Function MT_Get_Axis_Status_Zero Lib "MT_API.dll" _ (ByVal AObj As Integer, ByRef Zero As Integer) As Long</code>
Delphi	<code>function MT_Get_Axis_Status_Zero(AObj:Word; pZero: PInteger): Integer;</code>
C#	<code>public static extern int MT_Get_Axis_Status_Zero(WORD AObj,</code>

	ref Int32 Zero);	
Input Parameters	AObj	Motor control axis serial number
	Zero	Specifies the current zero position of the axis. 1 indicates zero position is active, 0 indicates zero position is inactive
Function Return	0	Function executed successfully, read data is valid
	Non-zero	Function execution failed; the read data is invalid.
Notes	Motor shaft serial numbers start from 0 . For an N-axis card, serial numbers are 0, 1, 2, 3...N-1 Ensure the motor shaft number function does not exceed bounds	

3.9.22 MT_Get_Axis_Status_Mode

Function Description: Reads the current operating mode of the specified control axis. This is the single-item status version of *MT_Get_Axis_Status2*.

C/VC/GCC	int32_t MT_Get_Axis_Status_Mode(WORD AObj, int32_t* pMode);	
VB	Function MT_Get_Axis_Status_Mode Lib "MT_API.dll" _ (ByVal AObj As Integer, ByRef Mode As Integer) As Long	
Delphi	function MT_Get_Axis_Status_Mode(AObj:Word; pMode: PInteger): Integer;	
C#	public static extern int MT_Get_Axis_Status_Mode(WORD AObj, ref Int32 Mode);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	Mode	Operating mode: 0=Zero Position Mode, 1=Constant Speed Mode, 2=Positioning Mode
Function Return	0	Function executed successfully, data read is valid
	Non-zero	Function execution failed; read data is invalid
Notes	Motor shaft serial numbers start from 0 . For an N-axis card, serial numbers are 0, 1, 2, 3...N-1 Ensure the motor shaft index function does not exceed bounds	

3.9.23 MT_Get_Encoder_Pos

Function Description: Reads the current closed-loop coordinate value of the specified control axis.

C/VC/GCC	int32_t MT_Get_Encoder_Pos(WORD AObj, int32_t* pValue)	
VB	Function MT_Get_Encoder_Pos(ByVal AObj As Integer, ByRef Value As Long) As Long	
Delphi	function MT_Get_Encoder_Pos (AObj:Word;pValue:PInteger):Integer;	

C#	public static extern int MT_Get_Axis_Encoder_Pos (UInt16 AObj, ref Int32 Value);	
Input Parameters	AObj	Motor control axis number
Output Parameters	Value	Current software position of the specified axis in pulses
Function Return	0	Function execution successful, read data valid
	Non-zero	Function execution failed; the read data is invalid
Notes	Closed-loop motor shafts are numbered starting from 0. For an N-axis card, the shafts are numbered 0, 1, 2, 3...N-1 Ensure the function for closed-loop motor shaft serial numbers does not exceed bounds	

3.9.24 MT_Set_Encoder_Pos

Function Description: Sets the current closed-loop coordinate value for the specified control axis.

C/VC/GCC	int32_t MT_Set_Encoder_Pos_Now(WORD AObj, int32_t Value)	
VB	Function MT_Set_Encoder_Pos_Now (ByVal AObj As Integer, ByVal Value As Long) As Long	
Delphi	function MT_Set_Encoder_Pos (AObj:Word;Value:Integer):Integer;	
C#	public static extern int MT_Set_Encoder_Pos (UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Motor control axis number
Output Parameters	Value	Current software position of the specified axis in pulses
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Closed-loop motor shafts are numbered starting from 0. For an N-axis card, the shafts are numbered 0, 1, 2, 3...N-1 Ensure the closed-loop motor axis number function does not exceed bounds	

3.9.25 MT_Set_Axis_Halt

Function Description: Immediately halts the movement of the specified axis without deceleration. Effective for all motion modes.

C/VC/GCC	int32_t MT_Set_Axis_Halt (WORD AObj)	
VB	Function MT_Set_Axis_Halt (ByVal AObj As Integer) As Long	
Delphi	function MT_Set_Axis_Halt (AObj:Word):Integer;	
C#	public static extern int MT_Set_Axis_Halt (UInt16 AObj);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	None	
Function Return	0	Function Execution Success
	Non-zero	Function execution failed
Remarks	Motor shaft numbers start from 0. For an N-axis card, the shaft numbers are 0, 1, 2, 3...N-1 Functions utilizing motor shaft serial numbers must avoid out-of-bounds access Emergency stop Inertial objects may cause impact to the driver	

	during movement. Use only in emergencies. Each mode employs a deceleration process for reduced impact.
--	--

3.9.26 MT_Set_Axis_Halt_All

Function Description: Immediately halts all axes without deceleration. Effective for all motion modes.

C/VC/GCC	int32_t MT_Set_Axis_Halt_All (void)
VB	Function MT_Set_Axis_Halt_All () As Long

Delphi	function MT_Set_Axis_Halt_All (): Integer;	
C#	public static extern int MT_Set_Axis_Halt_All ();	
Input Parameters	None	Motor control axis number
Output Parameters	None	
Function Return	0	Function Execution Success
	Non-zero	Function execution failed
Remarks	Motor shaft numbers start from 0. For an N-axis card, the shaft numbers are 0, 1, 2, 3...N-1 Functions utilizing motor shaft serial numbers must avoid out-of-bounds access Emergency stop Inertial objects cause impact on the driver during movement. Use only in emergencies. Stopping in each mode is a deceleration process, minimizing impact.	

3.10 Memory Operations

Note: If the controller operates in PLC mode, the memory may be fully or partially occupied by PLC functions. Use the read capacity value to determine availability. This memory is suitable for storing frequently used parameters, passwords, or for encryption purposes.

3.10.1 MT_Get_Param_Mem_Len

Function Description: Reads the capacity of the controller's memory in bytes (Byte).

C/VC/GCC	int32_t MT_Get_Param_Mem_Len (int32_t* pValue)	
VB	Function MT_Get_Param_Mem_Len (ByRef Value As Long) As Long	
Delphi	function MT_Get_Param_Mem_Len (pValue:PInteger):Integer;	
C#	public static extern int MT_Get_Axis_Num (ref Int32 Value);	
Input Parameters	None	
Output Parameters	Value	Memory capacity
Function Return	0	Function execution successful, read capacity is valid
	Non-zero	Function execution failed; the read capacity is invalid
Notes	Memory address indices start at 0. For a memory with length N, the address indices are 0, 1, 2, 3...N-1	

3.10.2 MT_Get_Param_Mem_Data

Function Description: Reads data from a specified memory address, one byte at a time

C/VC/GCC	int32_t MT_Get_Param_Mem_Data (WORD AObj, BYTE* pValue);	
VB	Function MT_Get_Param_Mem_Data (ByVal AObj As Integer, ByRef Value As BYTE) As Long	
Delphi	function MT_Get_Param_Mem_Data (AObj: Word; pValue: PByte): Integer;	
C#	public static extern int MT_Get_Param_Mem_Data (UInt16 AObj, ref Byte Value);	
Input Parameters	AObj	Memory Address
Output Parameters	Value	Byte data at the specified address

Function	0	Function execution successful, read data is valid
Return	Non-zero	Function execution failed; the read data is invalid
Note	Memory address indices start at 0 . For a memory with length N , the address indices are 0, 1, 2, 3...N-1	

3.10.3 MT_Set_Param_Mem_Data

Function Description: Sets the data at a specified memory address, one byte at a time

C/VC/GCC	int32_t MT_Set_Param_Mem_Data (WORD AObj, BYTE Value);	
VB	Function MT_Set_Param_Mem_Data (ByVal AObj As Integer, ByVal Value As BYTE) As Long	
Delphi	function MT_Set_Param_Mem_Data (AObj: Word; Value: BYTE): Integer;	
C#	public static extern int MT_Set_Param_Mem_Data (UInt16 AObj, BYTE Value);	
Input Parameters	AObj	Memory Address
Output Parameters	Value	Byte data at the specified address
Function	0	Function execution successful, read data valid
Return	Non-zero	Function execution failed; the read data is invalid
Note	Memory address numbering starts at 0 . For a memory with length N , the address numbers are 0, 1, 2, 3...N-1	

3.11 General-purpose digital inputs

General digital inputs serve as switch detection inputs with customizable functions. A portion of these inputs is multiplexed for negative limit, positive limit, and zero position detection for motion axes. Both input status and limit/zero position detection can monitor these states. These dedicated functions can be disabled via configuration tools to use them solely as general digital inputs. Typically, general digital inputs feature isolated channels for interference resistance and protection, though they cannot handle high-speed signals. Some controllers include non-isolated input channels for high-speed applications. Refer to the relevant hardware manual.

isolated channels, offering anti-interference and protective functions. The disadvantage is that they cannot accept high-speed signals. Some controllers also feature non-isolated input channels for high-speed input. Please refer to the relevant hardware manual.

3.11.1 MT_Get_Optic_In_Num

Function Description: Reads the number of general-purpose digital input channels

on the controller

C/VC/GCC	int32_t MT_Get_Optic_In_Num (int32_t* pValue)	
VB	Function MT_Get_Optic_In_Num (ByRef Value As Long) As Long	
Delphi	function MT_Get_Optic_In_Num (pValue:PInteger):Integer;	
C#	public static extern int MT_Get_Optic_In_Num (ref Int32 Value);	
Input Parameters	None	
Output Parameters	Value	Number of input channels
Function Returns	0	Function execution successful, valid number of channels read
	Non-zero	Function execution failed; the number of channels read is invalid
Notes	Channel indices start at 0. For N channels, indices are 0, 1, 2, 3...N-1	

3.11.2 MT_Get_Optic_In_Single

Function Description: Reads the level status on the specified digital input channel

C/VC/GCC	int32_t MT_Get_Optic_In_Single (WORD AObj, int32_t* pValue);	
VB	Function MT_Get_Optic_In_Single (ByVal AObj As Integer, ByRef Value As Long) As Long	
Delphi	function MT_Get_Optic_In_Single (AObj: Word; pValue: PInteger): Integer;	
C#	public static extern int MT_Get_Optic_In_Single (UInt16 AObj, ref Int32 Value);	
Input Parameters	AObj	Input channel number
Output Parameters	Value	Level status on the specified channel: 1 indicates high level, 0 indicates low level
Function Return	0	Function execution successful, read data valid
	Non-zero	Function execution failed; the read data is invalid
Remarks	Channel numbers start at 0. For N channels, the numbers are 0, 1, 2, 3...N-1.	

3.11.3 MT_Get_Optic_In_All

Function Description: Reads the level status on all input channels at once.

C/VC/GCC	int32_t MT_Get_Optic_In_All (int32_t* pValue);	
VB	Function MT_Get_Optic_In_All (ByRef Value As Long) As Long	
Delphi	function MT_Get_Optic_In_All (pValue:PInteger):Integer;	
C#	public static extern int MT_Get_Optic_In_All (ref Int32 Value);	
Input Parameters	None	
Output Parameters	Value	Level status on all channels: 1 indicates high level, 0 indicates low level Channel 0 is LSB (Least Significant Bit) Channel N corresponds to bit N-1 Reading 0x00000103 indicates channels 0, 1, and 8 are high
Function Return	0	Function execution successful, read data is valid
	Non-zero	Function execution failed; the read data is invalid.
Notes	Channel indices start at 0. For N channels, the indices are 0, 1, 2, 3...N-1	

3.12 General-purpose digital outputs

General-purpose digital outputs are used to output control signals to drive external digital switching devices such as relays and solenoid valves. The drive capability of general-purpose digital outputs varies by controller; some can only output signals, while others can directly drive small-to-medium relays and solenoid valves. Refer to the relevant hardware manual.

3.12.1 MT_Get_Optic_Out_Num

Function Description: Reads the number of output channels on the controller.

C/VC/GCC	int32_t MT_Get_Optic_Out_Num (int32_t* pValue)	
VB	Function MT_Get_Optic_Out_Num (ByRef Value As Long) As Long	
Delphi	function MT_Get_Optic_Out_Num (pValue:PInteger):Integer;	
C#	public static extern int MT_Get_Optic_Out_Num (ref Int32 Value);	
Input Parameters	None	
Output Parameters	Value	Number of output channels
Function Return	0	Function execution successful, valid channel count read
	Non-zero	Function execution failed; the number of channels read is invalid
Notes	Channel numbers start from 0. For N channels, the numbers are 0, 1, 2, 3...N-1	

3.12.2 MT_Set_Optic_Out_Single

Function Description: Sets the level state on the specified output channel

C/VC/GCC	int32_t MT_Set_Optic_Out_Single (WORD AObj, int32_t pValue);	
VB	Function MT_Set_Optic_Out_Single (ByVal AObj As Integer, ByVal Value As Long) As Long	
Delphi	function MT_Set_Optic_Out_Single (AObj: Word; Value: Integer): Integer;	
C#	public static extern int MT_Set_Optic_Out_Single (UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Output channel number
Output Parameters	Value	Level status on the specified channel: 1 indicates high level, 0 indicates low level
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Channel numbers start at 0. For N channels, the numbers are 0, 1, 2, 3...N-1	

3.12.3 MT_Set_Optic_Out_All

Function Description: Sets the level state on all output channels at once

C/VC/GCC	int32_t MT_Set_Optic_Out_All (int32_t Value);	
VB	Function MT_Set_Optic_Out_All (ByVal Value As Long) As Long	
Delphi	function MT_Set_Optic_Out_All (Value:Integer):Integer;	

C#	public static extern int MT_Set_Optic_Out_All (Int32 Value);	
Input Parameters	None	
Output Parameters	Value	Level status on all channels: 1 indicates high level, 0 indicates low level Channel 0 is LSB (Least Significant Bit) Channel N corresponds to bit N-1 Setting 0x00000103 indicates channels 0 , 1 , and 8 are high

Function	0	Function execution successful
Return	Non-zero	Function execution failed
Remarks	Channel numbers start from 0. For N channels, the numbers are 0, 1, 2, 3...N-1	

3.13 2-8 Multi-axis Interpolation

In addition to configuring interpolation parameters, it is recommended to use *MT_Set_Axis_Line_X_Run_Rel* and *MT_Set_Axis_Line_X_Run_Abs* to complete the interpolation action in one go.

Note: Acceleration, deceleration, and velocity during interpolation refer to the motion parameters of the fastest axis (the axis with the longest travel distance).

For precise control of acceleration, deceleration, and velocity—such as when performing 2-axis interpolation along a planar straight line—to maintain a linear speed of *V0* with relative distances *X0* and *Y0* (where *X0* > *Y0*) set the interpolation velocity as the projection along the X-axis:

$$V = V0 * X0 / \sqrt{X0*X0 + Y0*Y0}$$

3.13.1 MT_Set_Axis_Line_Axis[Compatible]

Function Description: Configure the motion axes participating in interpolation within a specified interpolation module. Different controllers feature varying numbers of interpolation modules, enabling multiple linear interpolations to occur simultaneously. **Each interpolation module** can freely designate any **two** axes to participate in interpolation concurrently.

Modules performing interpolation must not share the same interpolation axes. For example: Module 0 uses axes 2 and 3 for interpolation, while Module 1 uses axes 1 and 2. These modules can operate independently without conflict, but simultaneous operation is prohibited.

These two modules can operate independently without issues, but they cannot operate simultaneously.

C/VC/GCC	int32_t MT_Set_Axis_Line_Axis (WORD AObj, int32_t Axis_ID0, int32_t Axis_ID1)	
VB	Function MT_Set_Axis_Line_Axis (ByVal AObj As Integer, ByVal Axis_ID0 As Long, ByVal Axis_ID1 As Long) As Long	
Delphi	function MT_Set_Axis_Line_Axis (AObj: Word; Axis_ID0, Axis_ID1: Integer): Integer;	
C#	public static extern int MT_Set_Axis_Line_Axis (UInt16 AObj, Int32 Axis_ID0, Int32 Axis_ID1);	
Input Parameters	AObj	Interpolation module serial number, not the interpolation axis serial number
	Axis_ID0	First interpolation axis number for this interpolation module

	Axis_ID1	Second interpolation axis number of this interpolation module
Function	0	Function execution successful
Return	Non-zero	Function execution failed
Remarks	Interpolation module numbers start at 0 . For N interpolation modules, the numbers are 0, 1, 2, 3...N-1 Interpolation axis numbers start from 0 . For N axes, the numbers are 0, 1, 2, 3...N-1	

3.13.2 MT_Set_Axis_Line_Acc

Function Description: Sets the acceleration for the specified interpolation module.

C/VC/GCC	int32_t MT_Set_Axis_Line_Acc (WORD AObj, int32_t Value)
----------	---

VB	Function MT_Set_Axis_Line_Acc (ByVal AObj As Integer, ByVal Value As Long) As Long	
Delphi	function MT_Set_Axis_Line_Acc (AObj:Word;Value:Integer):Integer;	
C#	public static extern int MT_Set_Axis_Line_Acc (UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Interpolation module number, not the interpolation axis number
	Value	Acceleration for this interpolation module
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Interpolation module numbers start at 0. For the Nth interpolation module, the number is 0, 1, 2, 3...N-1	

3.13.3 MT_Set_Axis_Line_Dec

Function Description: Sets the acceleration for the specified interpolation module.

C/VC/GCC	int32_t MT_Set_Axis_Line_Dec (WORD AObj, int32_t Value)	
VB	Function MT_Set_Axis_Line_Dec (ByVal AObj As Integer, ByVal Value As Long) As Long	
Delphi	function MT_Set_Axis_Line_Dec (AObj:Word;Value:Integer):Integer;	
C#	public static extern int MT_Set_Axis_Line_Dec (UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Interpolation module number, not the interpolation axis number
	Value	Deceleration of this interpolation module
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Interpolation module numbers start at 0. For the Nth interpolation module, the number is 0, 1, 2, 3...N-1	

3.13.4 MT_Set_Axis_Line_V

Function Description: Sets the speed of the specified interpolation module.

C/VC/GCC	int32_t MT_Set_Axis_Line_V (WORD AObj, int32_t Value)	
VB	Function MT_Set_Axis_Line_V (ByVal AObj As Integer, ByVal Value As Long) As Long	
Delphi	function MT_Set_Axis_Line_V (AObj:Word;Value:Integer):Integer;	
C#	public static extern int MT_Set_Axis_Line_V (UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Interpolation module number, not the interpolation axis number
	Value	Speed of this interpolation module
Function Return	0	Function execution successful
	Non-zero	Function execution failed

Remarks	Interpolation module numbers start at 0 . For the Nth interpolation module, the number is 0, 1, 2, 3...N-1
---------	---

3.13.5 MT_Set_Axis_Line_Run[Compatible]

Function Description: After setting interpolation parameters, activates the specified interpolation module. Executes after **MT_Set_Axis_Line_Rel** and

MT_Set_Axis_Line_Abs. Can be replaced by the combined commands
 MT_Set_Axis_Line_Run_Rel and
 MT_Set_Axis_Line_Run_Abs.

C/VC/GCC	int32_t MT_Set_Axis_Line_Run (WORD AObj)	
VB	Function MT_Set_Axis_Line_Run (ByVal AObj As Integer) As Long	
Delphi	function MT_Set_Axis_Line_Run (AObj:Word):Integer;	
C#	public static extern int MT_Set_Axis_Line_Run (UInt16 AObj);	
Input Parameters	AObj	Interpolation module number, not the interpolation axis number
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Interpolation module numbers start at 0. For the Nth interpolation module, the numbers are 0, 1, 2, 3...N-1	

3.13.6 MT_Set_Axis_Line_Stop

Function Description: Decelerate and stop the interpolation axis

C/VC/GCC	int32_t MT_Set_Axis_Line_Stop (WORD AObj)	
VB	Function MT_Set_Axis_Line_Stop (ByVal AObj As Integer) As Long	
Delphi	function MT_Set_Axis_Line_Stop (AObj:Word):Integer;	
C#	public static extern int MT_Set_Axis_Line_Stop (UInt16 AObj);	
Input Parameters	AObj	Interpolation module number, not the interpolation axis number
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	The serial numbers of interpolation modules start from 0. For N interpolation modules, the serial numbers are 0, 1, 2, 3...N-1.	

3.13.7 MT_Set_Axis_Line_Halt

Function Description: Immediately halt the interpolation axis.

C/VC/GCC	int32_t MT_Set_Axis_Line_Halt (WORD AObj)	
VB	Function MT_Set_Axis_Line_Halt (ByVal AObj As Integer) As Long	
Delphi	function MT_Set_Axis_Line_Halt (AObj:Word):Integer;	
C#	public static extern int MT_Set_Axis_Line_Halt (UInt16 AObj);	
Input Parameters	AObj	Interpolation module number, not the interpolation axis number
Function Return	0	Function execution successful
	Non-zero	Function execution failed

Remarks	Interpolation module numbers start at 0 . For the Nth interpolation module, the number is 0, 1, 2, 3...N-1
---------	---

3.13.8 MT_Set_Axis_Line_Rel[Compatible]

Function Description: Sets the target position for linear interpolation relative to the current position. After setting the target, the interpolation axis remains inactive and requires

MT_Set_Axis_Line_Run to initiate movement.

C/VC/GCC	int32_t MT_Set_Axis_Line_Rel (WORD AObj, int32_t Axis_Target0, int32_t Axis_Target1)	
VB	Function MT_Set_Axis_Line_Rel (ByVal AObj As Integer, ByVal Axis_Target0 As Long, ByVal Axis_Target1 As Long) As Long	
Delphi	Function MT_Set_Axis_Line_Rel(AObj: Word; Axis_Target0, Axis_Target1: Integer): Integer	
C#	public static extern int MT_Set_Axis_Line_Rel(UInt16 AObj, Int32 Axis_Target0, Int32 Axis_Target1);	
Input Parameters	AObj	Interpolation module number, not the interpolation axis number
	Axis_Target0	Relative displacement distance for the first interpolation axis of this interpolation module
	Axis_Target1	Relative movement distance for the second interpolation axis in this interpolation module
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Interpolation module numbers start at 0. For the Nth interpolation module, the number is 0, 1, 2, 3...N-1	

3.13.9 MT_Set_Axis_Line_Abs[Compatible]

Function Description: Sets the absolute target position for linear interpolation. After setting the target, the interpolation axis remains stationary and requires

MT_Set_Axis_Line_Run to initiate movement.

C/VC/GCC	int32_t MT_Set_Axis_Line_Abs (WORD AObj, int32_t Axis_Target0, int32_t Axis_Target1)	
VB	Function MT_Set_Axis_Line_Abs (ByVal AObj As Integer, ByVal Axis_Target0 As Long, ByVal Axis_Target1 As Long) As Long	
Delphi	Function MT_Set_Axis_Line_Abs(AObj: Word; Axis_Target0, Axis_Target1: Integer): Integer	
C#	public static extern int MT_Set_Axis_Line_Abs(UInt16 AObj, Int32 Axis_Target0, Int32 Axis_Target1);	
Input Parameters	AObj	Interpolation module number, not the interpolation axis number

	Axis_Target0	Absolute position to which the first interpolation axis of this interpolation module must move
	Axis_Target1	Absolute position to which the second interpolation axis of this interpolation module must move
Function	0	Function execution successful
Return	Non-zero	Function execution failed
Remarks	Interpolation module numbers start at 0 . For the Nth interpolation module, the numbers are 0, 1, 2, 3...N-1	

3.13.10 MT_Set_Axis_Line_Run_Rel[Compatible]

Function Description: Sets the target position for linear interpolation relative to the current position; the interpolation axis moves immediately.

C/VC/GCC	int32_t MT_Set_Axis_Line_Run_Rel (WORD AObj, int32_t Axis_Target0, int32_t Axis_Target1)	
VB	Function MT_Set_Axis_Line_Run_Rel (ByVal AObj As Integer, ByVal Axis_Target0 As Long, ByVal Axis_Target1 As Long) As Long	
Delphi	Function MT_Set_Axis_Line_Run_Rel(AObj: Word; Axis_Target0, Axis_Target1: Integer): Integer	
C#	public static extern int MT_Set_Axis_Line_Run_Rel(UInt16 AObj, Int32 Axis_Target0, Int32 Axis_Target1);	
Input Parameters	AObj	Interpolation module number, not the interpolation axis number
	Axis_Target0	Relative displacement distance for the first interpolation axis of this interpolation module
	Axis_Target1	Relative movement distance for the second interpolation axis in this interpolation module
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Interpolation module numbers start at 0. For the Nth interpolation module, the numbers are 0, 1, 2, 3...N-1	

3.13.11 MT_Set_Axis_Line_Run_Abs[Compatible]

Function Description: Sets the absolute target position for linear interpolation; the interpolation axis moves immediately.

C/VC/GCC	int32_t MT_Set_Axis_Line_Run_Abs (WORD AObj, int32_t Axis_Target0, int32_t Axis_Target1)	
VB	Function MT_Set_Axis_Line_Run_Abs (ByVal AObj As Integer, ByVal Axis_Target0 As Long, ByVal Axis_Target1 As Long) As Long	
Delphi	Function MT_Set_Axis_Line_Run_Abs(AObj: Word; Axis_Target0, Axis_Target1: Integer): Integer	
C#	public static extern int MT_Set_Axis_Line_Run_Abs(UInt16 AObj, Int32 Axis_Target0, Int32 Axis_Target1);	
Input Parameters	AObj	Interpolation module number, not the interpolation axis number
	Axis_Target0	Absolute position to which the first interpolation axis of this interpolation module must move
	Axis_Target1	Absolute position to which the second interpolation axis of this interpolation module must move

Function	0	Function execution successful
Return	Non-zero	Function execution failed
Remarks	Interpolation module numbers start at 0. For the Nth interpolation module, the numbers are 0, 1, 2, 3...N-1	

3.13.12 MT_Get_Axis_Line_Num

Function Description: Reads the number of linear interpolation modules on the controller.

C/VC/GCC	int32_t MT_Get_Axis_Line_Num (int32_t* pValue)
VB	Function MT_Get_Axis_Line_Num (ByRef Value As Long) As Long
Delphi	function MT_Get_Axis_Line_Num (pValue:PInteger):Integer;

C#	public static extern int MT_Get_Axis_Line_Num (ref Int32 Value);	
Input Parameters	None	
Output Parameters	Value	Number of linear interpolation modules
Function Return	0	Function execution successful, valid channel count read
	Non-zero	Function execution failed; the number of channels read is invalid
Notes	Module serial numbers start from 0 . For N channels, serial numbers are 0, 1, 2, 3...N-1	

3.13.13 MT_Get_Axis_Line_Status

Function Description: Reads the current interpolation status of the specified linear interpolation module

C/VC/GCC	int32_t MT_Get_Axis_Line_Status (WORD AObj, int32_t* pValue);	
VB	Function MT_Get_Axis_Line_Status (ByVal AObj As Integer, ByRef Value As Long) As Long	
Delphi	function MT_Get_Axis_Line_Status (AObj: Word; pValue: PInteger): Integer;	
C#	public static extern int MT_Get_Axis_Line_Status (UInt16 AObj, ref Int32 Value);	
Input Parameters	AObj	The serial number of the interpolation module, not the serial number of the interpolation axis
Output Parameters	Value	Interpolation status of the specified module: 0 indicates interpolation completed, 1 indicates interpolation in progress
Function Return	0	Function execution successful, read data is valid
	Non-zero	Function execution failed; the read data is invalid.
Notes	Interpolation module indices start at 0 . For the Nth interpolation module, the indices are 0, 1, 2, 3...N-1	

3.13.14 MT_Get_Axis_Line_Acc

Function Description: Reads the current acceleration of the specified linear interpolation module

C/VC/GCC	int32_t MT_Get_Axis_Line_Acc (WORD AObj, int32_t* pValue);	
VB	Function MT_Get_Axis_Line_Acc (ByVal AObj As Integer, ByRef Value As Long) As Long	
Delphi	function MT_Get_Axis_Line_Acc (AObj: Word; pValue: PInteger): Integer;	
C#	public static extern int MT_Get_Axis_Line_Acc (UInt16 AObj, ref Int32 Value);	
Input	AObj	The serial number of the interpolation module, not

Parameters		the serial number of the interpolation axis
Output Parameters	Value	Acceleration of the specified module
Function Return	0	Function execution successful, data read is valid
	Non-zero	Function execution failed; the read data is invalid
Remarks	The serial numbers of interpolation modules start from 0. For N interpolation modules, the serial numbers are 0, 1, 2, 3...N-1.	

3.13.15 MT_Get_Axis_Line_Dec

Function Description: Reads the current deceleration of the specified linear interpolation module.

C/VC/GCC	int32_t MT_Get_Axis_Line_Dec (WORD AObj, int32_t* pValue);	
VB	Function MT_Get_Axis_Line_Dec (ByVal AObj As Integer, ByRef Value As Long) As Long	
Delphi	function MT_Get_Axis_Line_Dec (AObj: Word; pValue: PInteger): Integer;	
C#	public static extern int MT_Get_Axis_Line_Dec (UInt16 AObj, ref Int32 Value);	
Input Parameters	AObj	The serial number of the interpolation module, not the serial number of the interpolation axis
Output Parameters	Value	Specified deceleration for the module
Function Return	0	Function execution successful, data read is valid
	Non-zero	Function execution failed; the read data is invalid
Notes	Interpolation module indices start at 0. For the Nth interpolation module, the indices are 0, 1, 2, 3...N-1	

3.13.16 MT_Get_Axis_Line_V

Function Description: Reads the current speed of the specified linear interpolation module

C/VC/GCC	int32_t MT_Get_Axis_Line_V (WORD AObj, int32_t* pValue);	
VB	Function MT_Get_Axis_Line_V (ByVal AObj As Integer, ByRef Value As Long) As Long	
Delphi	function MT_Get_Axis_Line_V (AObj: Word; pValue: PInteger): Integer;	
C#	public static extern int MT_Get_Axis_Line_V (UInt16 AObj, ref Int32 Value);	
Input Parameters	AObj	The serial number of the interpolation module, not the serial number of the interpolation axis
Output Parameters	Value	Specified module speed
Function Return	0	Function execution successful, data read is valid
	Non-zero	Function execution failed; the read data is invalid
Remarks	Interpolation module indices start at 0. For the Nth interpolation module, the indices are 0, 1, 2, 3...N-1	

3.13.17 MT_Get_Axis_Line_Axis[Compatible]

Function Description: Reads the current interpolation axis of the specified linear interpolation module

C/VC/GCC	int32_t MT_Get_Axis_Line_Axis (WORD AObj, int32_t* pID0, int32_t* pID1);	
VB	Function MT_Get_Axis_Line_Axis (ByVal AObj As Integer, ByRef pID0 As Long, ByRef pID1 As Long) As Long	
Delphi	function MT_Get_Axis_Line_Axis (AObj:Word; pID0,pID1:PInteger):Integer;	

C#	public static extern int MT_Get_Axis_Line_Axis (UInt16 AObj, ref Int32 ID0, ref Int32 ID1);	
Input Parameters	AObj	Serial number of the interpolation module, not the serial number of the interpolation axis
Output Parameters	pID0	The first interpolation axis of the specified module
	pID1	Specifies the second interpolation axis of the module
Function Return	0	Function execution successful, read data is valid
	Non-zero	Function execution failed; the read data is invalid

Remarks	The serial numbers of interpolation modules start from 0. For N interpolation modules, the serial numbers are 0, 1, 2, 3...N-1. Interpolation axis numbers start from 0. For N motion axes, the numbers are 0, 1, 2, 3...N-1
---------	---

3.13.18 MT_Set_Axis_Line_V_Start

Function Description: Sets the start velocity for the specified interpolation module. Default value: 50Hz/s. Modifying this parameter is generally prohibited.

C/VC/GCC	int32_t MT_Set_Axis_Line_V_Start (WORD AObj, int32_t Value)	
VB	Function MT_Set_Axis_Line_V_Start(ByVal AObj As Integer, ByVal Value As Long) As Long	
Delphi	function MT_Set_Axis_Line_V_Start(AObj: Word; Value: Integer): Integer;	
C#	public static extern int MT_Set_Axis_Line_V_Start (UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Interpolation module number, not the interpolation axis number
	Value	Speed of this interpolation module
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Interpolation module numbers start at 0. For the Nth interpolation module, the number is 0, 1, 2, 3...N-1	

3.13.19 MT_Set_Axis_Line_X_Count[Compatible]

Function Description: Sets the number of axes participating in interpolation.

C/VC/GCC	int32_t MT_Set_Axis_Line_X_Count (WORD AObj, int32_t Value)	
VB	Function MT_Set_Axis_Line_X_Count (ByVal AObj As Integer, ByVal Value As Long) As Long	
Delphi	function MT_Set_Axis_Line_X_Count(AObj: Word; Value: Integer): Integer;	
C#	public static extern int MT_Set_Axis_Line_X_Count(UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Interpolation module number, not the interpolation axis number
	Value	Number of axes to interpolate, range 2-X, where X is the total number of axes on the control card
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Interpolation module numbers start at 0. For the Nth interpolation module, the number is 0, 1, 2, 3...N-1	

3.13.20 MT_Set_Axis_Line_X_Axis[Compatible]

Function Description: Sets the axes participating in interpolation and their sequence numbers within the interpolation group. After setting the sequence numbers and interpolation targets, use `MT_Set_Axis_Line_Run` to initiate.

C/VC/GCC	<code>int32_t MT_Set_Axis_Line_X_Axis(WORD AObj, int32_t AAxisID, int32_t AAxis)</code>
VB	Function <code>MT_Set_Axis_Line_X_Axis</code> (ByVal AObj As Integer, ByVal AAxisID As

	Long, ByVal AAxis As Long) As Long	
Delphi	function MT_Set_Axis_Line_X_Axis (AObj: Word; AAxisID: Integer; AAxis: Integer): Integer;	
C#	public static extern int MT_Set_Axis_Line_X_Axis(UInt16 AObj, Int32 AAxisID, Int32 AAxis)	
Input Parameters	AObj	Interpolation module number, not the interpolation axis number
	AAxisID	The sequence number of the participating axis in the interpolation mode, not the sequence number of the interpolation axis. Range: 0..Count-1 as set in MT_Set_Axis_Line_X_Count
	AAxis	The serial number of the interpolation axis, 0..M-1 M is the number of axes
Function Returns	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Interpolation module numbers start at 0. For the Nth interpolation module, the numbers are 0, 1, 2, 3...N-1	

3.13.21 MT_Set_Axis_Line_X_Target_Rel[Compatible]

Function Description: Sets the relative interpolation target for axes participating in linked interpolation. After setting the index and interpolation target, use

MT_Set_Axis_Line_Run to initiate.

C/VC/GCC	int32_t MT_Set_Axis_Line_X_Target_Rel (WORD AObj, AAxisID, int32_t ATarget)	
VB	Function MT_Set_Axis_Line_X_Target_Rel (ByVal AObj As Integer, ByVal AAxisID As Long, ByVal ATarget As Long) As Long	
Delphi	function MT_Set_Axis_Line_X_Target_Rel (AObj: Integer; AAxisID: Integer; ATarget: Integer): Word;	
C#	public static extern int MT_Set_Axis_Line_X_Target_Rel (UInt16 AObj, Int32 AAxisID, Int32 ATarget)	
Input Parameters	AObj	Interpolation module number, not the interpolation axis number
	AAxisID	The sequence number of the participating axis in the interpolation mode, not the sequence number of the interpolation axis. Range: 0..Count-1 as set in MT_Set_Axis_Line_X_Count
	ATarget	Relative interpolation position
Function	0	Function execution successful

Returns	Non-zero	Function execution failed
Remarks	Interpolation module numbers start at 0. For the Nth interpolation module, the numbers are 0, 1, 2, 3...N-1	

3.13.22 MT_Set_Axis_Line_X_Target_Abs [Compatible]

Function Description: Sets the absolute interpolation target for axes participating in linked interpolation. After setting the index and interpolation target, use

MT_Set_Axis_Line_Run to initiate.

C/VC/GCC	int32_t MT_Set_Axis_Line_X_Target_Abs(WORD AObj, int32_t AAxisID, int32_t ATarget)
----------	--

VB	Function MT_Set_Axis_Line_X_Target_Abs (ByVal AObj As Integer, ByVal AAxisID As Long, ByVal ATarget As Long) As Long	
Delphi	function MT_Set_Axis_Line_X_Target_Abs (AObj: Word; AAxisID: Integer; ATarget: Integer): Integer;	
C#	public static extern int MT_Set_Axis_Line_X_Target_Abs (UInt16 AObj, Int32 AAxisID, Int32 ATarget)	
Input Parameters	AObj	Interpolation module number, not the interpolation axis number
	AAxisID	The sequence number of the participating axis in the interpolation mode, not the sequence number of the interpolation axis. Range: 0..Count-1 as set in MT_Set_Axis_Line_X_Count
	ATarget	Absolute interpolation position
Function Returns	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Interpolation module numbers start at 0. For the Nth interpolation module, the numbers are 0, 1, 2, 3...N-1	

3.13.23 MT_Set_Axis_Line_X_Run_Rel

Function Description: Initiates interpolation with relative offset mode, equivalent to

MT_Set_Axis_Line_X_Count, MT_Set_Axis_Line_X_Axis, MT_Set_Axis_Line_X_Target_Rel, and MT_Set_Axis_Line_Run.

C/VC/GCC	int32_t MT_Set_Axis_Line_X_Run_Rel(WORD AObj, int32_t AAxis_Num, int32_t* pAxis, int32_t* pTarget)	
VB	Function MT_Set_Axis_Line_X_Run_Rel (ByVal AObj As Integer, ByVal AAxis_Num As Long, ByRef pAxis As Long, ByRef pTarget As Long) As Long	
Delphi	function MT_Set_Axis_Line_X_Run_Rel (AObj: Word; AAxis_Num: Integer; pAxis: PInteger; pTarget: PInteger): Integer;	
C#	public static extern int MT_Set_Axis_Line_X_Run_Rel(UInt16 AObj, Int32 AAxis_Num, ref Int32 pAxis, ref Int32 pTarget)	
Input Parameters	AObj	Interpolation module number, not the interpolation axis number
	AAxis_Num	Number of axes participating in interpolation 2-X
	pAxis	Array of axis numbers participating in interpolation. An array must be passed, and its length must be greater than or equal to the number of axes involved in interpolation

	ATarget	An array of relative targets for the interpolating axes. An array must be passed in, and the length of this array must be greater than or equal to the number of interpolation axes.
Function Returns	0	if the function executes successfully
	Non-zero	Function execution failed
Remarks	Interpolation module numbers start at 0 . For the Nth interpolation module, the number is 0, 1, 2, 3...N-1	

3.13.24 MT_Set_Axis_Line_X_Run_Abs

Function Description: Initiates absolute coordinate interpolation, equivalent to

MT_Set_Axis_Line_X_Count, MT_Set_Axis_Line_X_Axis, MT_Set_Axis_Line_X_Target_Abs, and MT_Set_Axis_Line_Run.

C/VC/GCC	int32_t MT_Set_Axis_Line_X_Run_Abs(WORD AObj, int32_t AAxis_Num, int32_t* pAxis, int32_t* pTarget)	
VB	Function MT_Set_Axis_Line_X_Run_Abs (ByVal AObj As Integer, ByVal AAxis_Num As Long, ByRef pAxis As Long, ByRef pTarget As Long) As Long	
Delphi	function MT_Set_Axis_Line_X_Run_Abs (AObj: Word; AAxis_Num: Integer; pAxis: PInteger; pTarget: PInteger): Integer;	
C#	public static extern int MT_Set_Axis_Line_X_Run_Abs(UInt16 AObj, Int32 AAxis_Num, ref Int32 pAxis, ref Int32 pTarget)	
Input Parameters	AObj	Interpolation module number, not the interpolation axis number
	AAxis_Num	Number of axes participating in interpolation 2-X
	pAxis	Array of axis numbers participating in interpolation. An array must be passed, and its length must be greater than or equal to the number of axes involved in interpolation
	ATarget	Array of absolute targets for the interpolating axes. Must pass an array whose length must be greater than or equal to the number of interpolation axes.
Function Returns	0	if the function executes successfully
	Non-zero	Function execution failed
Remarks	Interpolation module numbers start at 0. For the Nth interpolation module, the numbers are 0, 1, 2, 3...N-1	

3.14 Arc Interpolation

Note: Arc interpolation motion parameters differ from those of simultaneous interpolation. Arc interpolation parameters are defined relative to the tangent direction of the arc curve, representing the combined effect of two axes: linear

deceleration, linear deceleration, and linear

velocity. When the start and end coordinates are

identical, a full circle is traversed.

3.14.1 MT_Set_Axis_Circle_Axis

Function Description: Sets the motion axes participating in interpolation within a specified interpolation module. Different controllers have varying numbers of interpolation modules. Multiple modules can perform multiple circular arc interpolations simultaneously. Each interpolation module can freely specify any **two** axes to participate in interpolation.

Modules performing interpolation simultaneously must not share the same interpolation axes. For example, Module 0 uses axes 2 and 3 for interpolation, while Module 1 uses axes 1 and 2. These modules can operate independently without conflict, but cannot run concurrently.

C/VC/GCC	int32_t MT_Set_Axis_Circle_Axis (WORD AObj, int32_t Axis_ID0, int32_t Axis_ID1)
VB	Function MT_Set_Axis_Circle_Axis (ByVal AObj As Integer, ByVal Axis_ID0 As Long, ByVal Axis_ID1 As Long) As Long
Delphi	function MT_Set_Axis_Circle_Axis (AObj: Word; Axis_ID0, Axis_ID1: Integer): Integer;
C#	public static extern int MT_Set_Axis_Circle_Axis (UInt16 AObj, Int32 Axis_ID0, Int32

	Axis_ID1);	
Input Parameters	AObj	Interpolation module serial number, not the interpolation axis serial number
	Axis_ID0	First interpolation axis number for this interpolation module
	Axis_ID1	Second interpolation axis number for this interpolation module
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Interpolation module numbers start at 0. For N interpolation modules, the numbers are 0, 1, 2, 3...N-1 Interpolation axis numbers start from 0. For N axes, the numbers are 0, 1, 2, 3...N-1	

3.14.2 MT_Set_Axis_Circle_Acc

Function Description: Sets the acceleration for the specified interpolation module.

C/VC/GCC	int32_t MT_Set_Axis_Circle_Acc (WORD AObj, int32_t Value)	
VB	Function MT_Set_Axis_Circle_Acc (ByVal AObj As Integer, ByVal Value As Long) As Long	
Delphi	function MT_Set_Axis_Circle_Acc (AObj: Word; Value: Integer): Integer;	
C#	public static extern int MT_Set_Axis_Circle_Acc (UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Interpolation module number, not the interpolation axis number
	Value	Acceleration for this interpolation module
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Interpolation module indices start at 0. For the Nth interpolation module, the indices are 0, 1, 2, 3...N-1	

3.14.3 MT_Set_Axis_Circle_Dec

Function Description: Sets the acceleration for the specified interpolation module.

C/VC/GCC	int32_t MT_Set_Axis_Circle_Dec (WORD AObj, int32_t Value)	
VB	Function MT_Set_Axis_Circle_Dec (ByVal AObj As Integer, ByVal Value As Long) As Long	
Delphi	function MT_Set_Axis_Circle_Dec (AObj: Word; Value: Integer): Integer;	
C#	public static extern int MT_Set_Axis_Circle_Dec (UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Interpolation module number, not the interpolation axis number

	Value	Deceleration value for this interpolation module
Function	0	Function execution successful
Return	Non-zero	Function execution failed
Remarks	Interpolation module indices start at 0 . For the Nth interpolation module, the indices are 0, 1, 2, 3...N-1	

3.14.4 MT_Set_Axis_Circle_V

Function Description: Sets the speed for the specified interpolation module.

C/VC/GCC	int32_t MT_Set_Axis_Circle_V (WORD AObj, int32_t Value)	
VB	Function MT_Set_Axis_Circle_V (ByVal AObj As Integer, ByVal Value As Long) As Long	
Delphi	function MT_Set_Axis_Circle_V (AObj:Word;Value:Integer):Integer;	
C#	public static extern int MT_Set_Axis_Circle_V (UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Interpolation module number, not the interpolation axis number
	Value	Speed of this interpolation module
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Interpolation module numbers start at 0. For the Nth interpolation module, the numbers are 0, 1, 2, 3...N-1	

3.14.5 MT_Set_Axis_Circle_Stop

Function Description: Decelerate and stop the interpolation axis

C/VC/GCC	int32_t MT_Set_Axis_Circle_Stop (WORD AObj)	
VB	Function MT_Set_Axis_Circle_Stop (ByVal AObj As Integer) As Long	
Delphi	function MT_Set_Axis_Circle_Stop (AObj:Word):Integer;	
C#	public static extern int MT_Set_Axis_Circle_Stop (UInt16 AObj);	
Input Parameters	AObj	Interpolation module number, not the interpolation axis number
Function returns	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Interpolation module numbers start at 0. For the Nth interpolation module, the number is 0, 1, 2, 3...N-1	

3.14.6 MT_Set_Axis_Circle_Halt

Function Description: Immediately stop the interpolation axis

C/VC/GCC	int32_t MT_Set_Axis_Circle_Halt (WORD AObj)	
VB	Function MT_Set_Axis_Circle_Halt (ByVal AObj As Integer) As Long	
Delphi	function MT_Set_Axis_Circle_Halt (AObj:Word):Integer;	
C#	public static extern int MT_Set_Axis_Circle_Halt (UInt16 AObj);	
Input Parameters	AObj	Interpolation module number, not the interpolation axis number
Function Return	0	Function execution successful
	Non-zero	Function execution failed

Remarks	Interpolation module numbers start at 0 . For the Nth interpolation module, the numbers are 0, 1, 2, 3...N-1
---------	---

3.14.7 MT_Set_Axis_Circle_R_CW_Run_Rel

Function Description: Performs clockwise circular interpolation based on parameters relative to the current position, starting immediately.

C/VC/GCC	int32_t MT_Set_Axis_Circle_R_CW_Run_Rel (WORD AObj, int32_t AR, int32_t Axis_Target0, int32_t Axis_Target1)	
VB	Function MT_Set_Axis_Circle_R_CW_Run_Rel (ByVal AObj As Integer, ByVal AR As Long, ByVal Axis_Target0 As Long, ByVal Axis_Target1 As Long) As Long	
Delphi	Function MT_Set_Axis_Circle_R_CW_Run_Rel (AObj: Word; AR, Axis_Target0, Axis_Target1: Integer): Integer	
C#	public static extern int MT_Set_Axis_Circle_R_CW_Run_Rel (UInt16 AObj, Int32 AR, Int32 Axis_Target0, Int32 Axis_Target1);	
Input Parameters	AObj	Interpolation module number, not the interpolation axis number
	AR	Radius for arc interpolation. Positive values represent minor arcs, negative values represent major arcs. The radius is half the arc's diameter. . No distinction between relative and absolute
	Axis_Target0	Relative displacement distance for the first interpolation axis in this module
	Axis_Target1	The relative displacement distance for the second interpolation axis in this interpolation module
	Function Return	0 Non-zero
Remarks	Interpolation module numbers start at 0. For the Nth interpolation module, the numbers are 0, 1, 2, 3...N-1	

3.14.8 MT_Set_Axis_Circle_R_CW_Run_Abs

Function Description: Performs clockwise circular interpolation at absolute position parameters, starting immediately.

C/VC/GCC	int32_t MT_Set_Axis_Circle_R_CW_Run_Abs (WORD AObj, int32_t AR, int32_t Axis_Target0, int32_t Axis_Target1)	
VB	Function MT_Set_Axis_Circle_R_CW_Run_Abs (ByVal AObj As Integer, ByVal AR As Long, ByVal Axis_Target0 As Long, ByVal Axis_Target1 As Long) As Long	
Delphi	Function MT_Set_Axis_Circle_R_CW_Run_Abs (AObj: Word; AR, Axis_Target0, Axis_Target1: Integer): Integer	
C#	public static extern int MT_Set_Axis_Circle_R_CW_Run_Abs (UInt16 AObj, Int32 AR, Int32 Axis_Target0, Int32 Axis_Target1);	
Input Parameters	AObj	Interpolation module number, not the interpolation axis number

	AR	Radius for arc interpolation. Positive values represent minor arcs, negative values represent major arcs. The radius is half the arc's diameter. . No distinction between relative and absolute
	Axis_Target0	Absolute position to which the first interpolation axis of this module must move
	Axis_Target1	The absolute position to which the second interpolation axis of this interpolation module must move.
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Interpolation module numbers start at 0 . For the Nth interpolation module, the numbers are 0, 1, 2, 3...N-1	

3.14.9 MT_Set_Axis_Circle_R_CCW_Run_Rel

Function Description: Performs counterclockwise circular interpolation relative to the current position, starting immediately.

C/VC/GCC	int32_t MT_Set_Axis_Circle_R_CCW_Run_Rel (WORD AObj, int32_t AR, int32_t Axis_Target0, int32_t Axis_Target1)	
VB	Function MT_Set_Axis_Circle_R_CCW_Run_Rel (ByVal AObj As Integer, ByVal AR As Long, ByVal Axis_Target0 As Long, ByVal Axis_Target1 As Long) As Long	
Delphi	Function MT_Set_Axis_Circle_R_CCW_Run_Rel (AObj: Word; AR, Axis_Target0, Axis_Target1: Integer): Integer	
C#	public static extern int MT_Set_Axis_Circle_R_CCW_Run_Rel (UInt16 AObj, Int32 AR, Int32 Axis_Target0, Int32 Axis_Target1);	
Input Parameters	AObj	Interpolation module number, not the interpolation axis number
	AR	Radius for arc interpolation. Positive values represent minor arcs, negative values represent major arcs. The radius is half the arc's diameter. . No distinction between relative and absolute
	Axis_Target0	Relative displacement distance for the first interpolation axis in this module
	Axis_Target1	The relative displacement distance for the second interpolation axis in this interpolation module
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Interpolation module numbers start at 0. For the Nth interpolation module, the numbers are 0, 1, 2, 3...N-1	

3.14.10 MT_Set_Axis_Circle_R_CCW_Run_Abs

Function Description: Performs counterclockwise circular interpolation based on absolute position parameters, starting immediately.

C/VC/GCC	int32_t MT_Set_Axis_Circle_R_CCW_Run_Abs (WORD AObj, int32_t AR, int32_t Axis_Target0, int32_t Axis_Target1)	
VB	Function MT_Set_Axis_Circle_R_CCW_Run_Abs (ByVal AObj As Integer, ByVal AR As Long, ByVal Axis_Target0 As Long, ByVal Axis_Target1 As Long) As Long	
Delphi	Function MT_Set_Axis_Circle_R_CCW_Run_Abs (AObj: Word; AR, Axis_Target0, Axis_Target1: Integer): Integer	
C	public static extern int MT_Set_Axis_Circle_R_CCW_Run_Abs (UInt16 AObj, Int32 AR, Int32 Axis_Target0, Int32 Axis_Target1);	
Input Parameters	AObj	Interpolation module number, not the interpolation axis number

	AR	Arc interpolation radius. Positive values indicate minor arcs, negative values indicate major arcs. The radius is half the arc's diameter. Path, no distinction between relative and absolute
	Axis_Target0	The absolute position to which the first interpolation axis of this interpolation module must move
	Axis_Target1	The absolute position to which the second interpolation axis of this interpolation module must move
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Interpolation module numbers start at 0 . For the Nth interpolation module, the number is 0, 1, 2, 3...N-1	

3.14.11 MT_Get_Axis_Circle_Num

Function Description: Reads the number of circular interpolation modules on the controller.

C/VC/GCC	int32_t MT_Get_Axis_Circle_Num (int32_t* pValue)	
VB	Function MT_Get_Axis_Circle_Num (ByRef Value As Long) As Long	
Delphi	function MT_Get_Axis_Circle_Num (pValue:PInteger):Integer;	
C#	public static extern int MT_Get_Axis_Circle_Num (ref Int32 Value);	
Input Parameters	None	
Output Parameters	Value	Number of arc interpolation modules
Function Return	0	Function execution successful, valid channel count read
	Non-zero	Function execution failed; the number of channels read is invalid
Notes	Module indices start at 0. For N channels, indices are 0, 1, 2, 3...N-1	

3.14.12 MT_Get_Axis_Circle_Status

Function Description: Reads the current interpolation status of the specified circular interpolation module.

C/VC/GCC	int32_t MT_Get_Axis_Circle_Status (WORD AObj, int32_t* pValue);	
VB	Function MT_Get_Axis_Circle_Status (ByVal AObj As Integer, ByRef Value As Long) As Long	
Delphi	function MT_Get_Axis_Circle_Status (AObj: Word; pValue: PInteger): Integer;	
C#	public static extern int MT_Get_Axis_Circle_Status (UInt16 AObj, ref Int32 Value);	
Input Parameters	AObj	The serial number of the interpolation module, not the serial number of the interpolation axis
Output Parameters	Value	Interpolation status of the specified module: 0 indicates interpolation completed, 1 indicates interpolation in progress
Function Return	0	Function execution successful; read data is valid
	Non-zero	Function execution failed; the read data is invalid.
Remarks	Interpolation module indices start at 0. For the Nth interpolation module, the indices are 0, 1, 2, 3...N-1	

3.14.13 MT_Get_Axis_Circle_Acc

Function Description: Reads the current acceleration of the specified circular interpolation module.

C/VC/GCC	int32_t MT_Get_Axis_Circle_Acc (WORD AObj, int32_t* pValue);	
VB	Function MT_Get_Axis_Circle_Acc (ByVal AObj As Integer, ByRef Value As Long) As Long	
Delphi	function MT_Get_Axis_Circle_Acc (AObj: Word; pValue: PInteger): Integer;	
C#	public static extern int MT_Get_Axis_Circle_Acc (UInt16 AObj, ref Int32 Value);	
Input Parameters	AObj	The serial number of the interpolation module, not the serial number of the interpolation axis.
Output Parameters	Value	Acceleration of the specified module
Function Returns	0	Function execution successful, data read is valid

	Non-zero	Function execution failed; the read data is invalid
Notes	Interpolation module indices start at 0 . For the Nth interpolation module, the indices are 0, 1, 2, 3...N-1	

3.14.14 MT_Get_Axis_Circle_Dec

Function Description: Reads the current deceleration of the specified circular interpolation module.

C/VC/GCC	int32_t MT_Get_Axis_Circle_Dec (WORD AObj, int32_t* pValue);	
VB	Function MT_Get_Axis_Circle_Dec (ByVal AObj As Integer, ByRef Value As Long) As Long	
Delphi	function MT_Get_Axis_Circle_Dec (AObj: Word; pValue: PInteger): Integer;	
C#	public static extern int MT_Get_Axis_Circle_Dec (UInt16 AObj, ref Int32 Value);	
Input Parameters	AObj	The serial number of the interpolation module, not the serial number of the interpolation axis
Output Parameters	Value	Specified deceleration for the module
Function Return	0	Function execution successful, data read is valid
	Non-zero	Function execution failed; the read data is invalid
Notes	Interpolation module indices start at 0 . For the Nth interpolation module, the indices are 0, 1, 2, 3...N-1	

3.14.15 MT_Get_Axis_Circle_V

Function Description: Reads the current speed of the specified circular interpolation module.

C/VC/GCC	int32_t MT_Get_Axis_Circle_V (WORD AObj, int32_t* pValue);	
VB	Function MT_Get_Axis_Circle_V (ByVal AObj As Integer, ByRef Value As Long) As Long	
Delphi	function MT_Get_Axis_Circle_V (AObj: Word; pValue: PInteger): Integer;	
C#	public static extern int MT_Get_Axis_Circle_V (UInt16 AObj, ref Int32 Value);	
Input Parameters	AObj	The serial number of the interpolation module, not the serial number of the interpolation axis
Output Parameters	Value	Specified module speed
Function Return	0	Function execution successful, data read is valid
	Non-zero	Function execution failed; the read data is invalid
Notes	Interpolation module indices start at 0 . For the Nth interpolation module, the indices are 0, 1, 2, 3...N-1	

3.14.16 MT_Get_Axis_Circle_Axis

Function Description: Reads the current interpolation axis of the specified circular interpolation module.

C/VC/GCC	int32_t MT_Get_Axis_Circle_Axis (WORD AObj, int32_t* pID0, int32_t* pID1);
VB	Function MT_Get_Axis_Circle_Axis (ByVal AObj As Integer, ByRef pID0 As Long, ByRef pID1 As Long) As Long
Delphi	function MT_Get_Axis_Circle_Axis (AObj:Word; pID0,pID1:PInteger):Integer;

C#	public static extern int MT_Get_Axis_Circle_Axis (UInt16 AObj, ref Int32 ID0, ref Int32 ID1);	
Input Parameters	AObj	Serial number of the interpolation module, not the serial number of the interpolation axis
Output Parameters	pID0	The first interpolation axis of the specified module
	pID1	Specifies the second interpolation axis of the module
Function Return	0	Function execution successful, read data is valid
	Non-zero	Function execution failed; the read data is invalid
Notes	The index of the interpolation module starts from 0 . For the Nth interpolation module, the index is 0, 1, 2, 3...N-1 Interpolation axis numbers start from 0 . For N motion axes, the numbers are 0, 1, 2, 3...N-1	

3.14.17 MT_Set_Axis_Circle_V_Start

Function Description: Sets the start velocity for the specified interpolation module.

C/VC/GCC	int32_t MT_Set_Axis_Circle_V_Start (WORD AObj, int32_t Value)	
VB	Function MT_Set_Axis_Circle_V_Start(ByVal AObj As Integer, ByVal Value As Long) As Long	
Delphi	function MT_Set_Axis_Circle_V_Start(AObj: Word; Value: Integer): Integer;	
C#	public static extern int MT_Set_Axis_Circle_V_Start (UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Interpolation module number, not the interpolation axis number
	Value	Speed of this interpolation module
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Interpolation module numbers start from 0 . For N interpolation modules, the numbers are 0, 1, 2, 3...N-1	

3.15 PLC Mode/Offline Mode

PLC mode requires reference to relevant tool software and documentation. Functions in this section facilitate offline PLC program operation in conjunction with host computers or monitor PLC program execution. PLC programs can operate independently or in coordination with host computer programs. When coordinated, PLC program parameters are typically configured via global variables, while the PLC's inherent process actions remain unchanged.

3.15.1 MT_Get_PLC_Var_Num

Function Description: Reads the number of global variables in PLC mode on the controller

C/VC/GCC	int32_t MT_Get_PLC_Var_Num (int32_t* pValue)	
VB	Function MT_Get_PLC_Var_Num (ByRef Value As Long) As Long	
Delphi	function MT_Get_PLC_Var_Num (pValue:PlInteger):Integer;	
C#	public static extern int MT_Get_PLC_Var_Num (ref Int32 Value);	
Input Parameters	None	

Output Parameters	Value	Number of global variables in the PLC
Function returns	0	The function executed successfully and read a valid channel count.
	Non-zero	Function execution failed; the number of channels read is invalid
Notes	Module serial numbers start from 0. For N channels, serial numbers are 0, 1, 2, 3...N-1	

3.15.2 MT_Get_PLC_Var_Data

Function Description: Reads the value of the global variable with the specified index in PLC mode

C/VC/GCC	int32_t MT_Get_PLC_Var_Data (WORD AObj, int32_t* pValue);	
VB	Function MT_Get_PLC_Var_Data (ByVal AObj As Integer, ByRef Value As Long) As Long	
Delphi	function MT_Get_PLC_Var_Data (AObj: Word; pValue: PInteger): Integer;	
C#	public static extern int MT_Get_PLC_Var_Data (UInt16 AObj, ref Int32 Value);	
Input Parameters	AObj	PLC global variable index, 0..N-1
Output Parameters	Value	Value of the global variable with the specified index
Function Return	0	Function execution successful, data read is valid
	Non-zero	Function execution failed; the read data is invalid
Remarks	PLC global variable indices start at 0. For N global variables, indices are 0, 1, 2, 3...N-1	

3.15.3 MT_Set_PLC_Var_Data

Function Description: Sets the value of the global variable with the specified index in PLC mode.

C/VC/GCC	int32_t MT_Set_PLC_Var_Data (WORD AObj, int32_t Value)	
VB	Function MT_Set_PLC_Var_Data (ByVal AObj As Integer, ByVal Value As Long) As Long	
Delphi	function MT_Set_PLC_Var_Data (AObj:Word;Value:Integer):Integer;	
C#	public static extern int MT_Set_PLC_Var_Data (UInt16 AObj, Int32 Value);	
Input Parameters	AObj	PLC global variable index, 0..N-1
	Value	Value of the specified global variable
Function Return	0	Function execution successful
	Non-zero	Function execution failed

Remarks	PLC global variable indices start at 0. For N global variables, indices are 0, 1, 2, 3...N-1
---------	--

3.15.4 MT_Set_PLC_Pause

Function Description: Pauses instruction execution in PLC mode. Resumes execution from the paused point after using MT_Set_PLC_Run.

C/VC/GCC	int32_t MT_Set_PLC_Pause(void)
VB	Function MT_Set_PLC_Pause () As Long

Delphi	function MT_Set_PLC_Pause ():Integer;	
C#	public static extern int MT_Set_PLC_Pause();	
Input Parameters		
Function returns	0	Function execution successful
	Non-zero	Function execution failed
Remarks		

3.15.5 MT_Set_PLC_Stop

Function Description: Stops instruction execution in PLC mode. After using MT_Set_PLC_Run, execution resumes from the beginning.

C/VC/GCC	int32_t MT_Set_PLC_Stop(void)	
VB	Function MT_Set_PLC_Stop () As Long	
Delphi	function MT_Set_PLC_Stop ():Integer;	
C#	public static extern int MT_Set_PLC_Stop();	
Input Parameters		
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks		

3.15.6 MT_Set_PLC_Run

Function Description: Enables PLC mode.

C/VC/GCC	int32_t MT_Set_PLC_Run(void)	
VB	Function MT_Set_PLC_Run () As Long	
Delphi	function MT_Set_PLC_Run ():Integer;	
C#	public static extern int MT_Set_PLC_Run();	
Input Parameters		
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks		

3.15.7 MT_Get_PLC_Status

Function Description: Reads the operational status of **the PLC** mode on the controller

C/VC/GCC	int32_t MT_Get_PLC_Status(int32_t* pValue)	
VB	Function MT_Get_PLC_Status(ByRef Value As Long) As Long	
Delphi	function MT_Get_PLC_Status(pValue: PInteger): Integer;	
C#	public static extern int MT_Get_PLC_Status(ref Int32 Value);	
Input Parameters	None	
Output Parameters	Value	PLC operational status: 0x37 = Running, 0x28 = Stopped, 0x19 = Paused
Function Return	0	Function executed successfully, valid channel count read
	Non-zero	Function execution failed, invalid channel count read
Notes	Module indices start at 0. For N channels, indices are 0, 1, 2, 3...N-1	

3.16 Stream Instruction Mode

In Stream mode, some functions are stream control functions. These functions do not enter the **Stream** instruction queue and return immediately, similar to regular functions described in previous sections. Others are regular **Stream** instructions, which are cached in the **Stream** instruction queue and executed sequentially as they are queued.

Unless explicitly stated as a stream control function, the following are standard **Stream** directives.

3.16.1 MT_Get_Stream_Space

Function Description: Reads the available space in the instruction buffer for the **Stream** mode on the controller. Instructions can only be written if space is available; otherwise, they are discarded. This is a control instruction that returns immediately.

C/VC/GCC	int32_t MT_Get_Stream_Space (int32_t* pValue)	
VB	Function MT_Get_Stream_Space (ByRef Value As Long) As Long	
Delphi	function MT_Get_Stream_Space (pValue:PInteger):Integer;	
C#	public static extern int MT_Get_Stream_Space (ref Int32 Value);	
Input Parameters	None	
Output Parameters	Value	Stream command space remaining
Function	0	Function execution successful, valid channel count read

Return	Non-zero	Function execution failed; the number of channels read is invalid
Remarks		

3.16.2 MT_Set_Stream_Pause

Function Description: Pauses execution of **stream** mode commands. Resumes execution from the paused point after calling **MT_Set_Stream_Run**. Control command; returns immediately.

C/VC/GCC	int32_t MT_Set_Stream_Pause(void)
VB	Function MT_Set_Stream_Pause () As Long

Delphi	function MT_Set_Stream_Pause ():Integer;	
C#	public static extern int MT_Set_Stream_Pause();	
Input Parameters		
Function Returns	0	Function execution successful
	Non-zero	Function execution failed
Remarks		

3.16.3 MT_Set_Stream_Stop

Function Description: Stops the execution of **stream** mode commands. After using **MT_Set_Stream_Run**, execution resumes from the beginning. Control command; returns immediately.

C/VC/GCC	int32_t MT_Set_Stream_Stop(void)	
VB	Function MT_Set_Stream_Stop () As Long	
Delphi	function MT_Set_Stream_Stop ():Integer;	
C#	public static extern int MT_Set_Stream_Stop();	
Input Parameters		
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks		

3.16.4 MT_Set_Stream_Run

Function Description: Enables **Stream** mode. Control command; returns immediately.

C/VC/GCC	int32_t MT_Set_Stream_Run(void)	
VB	Function MT_Set_Stream_Run () As Long	
Delphi	function MT_Set_Stream_Run ():Integer;	
C#	public static extern int MT_Set_Stream_Run();	
Input Parameters		
Function Returns	0	Function execution successful
	Non-zero	Function execution failed
Note		

3.16.5 MT_Set_Stream_Clear

Function Description: Clears unexecuted **stream** instructions from the instruction space. Control instruction; returns immediately.

C/VC/GCC	int32_t MT_Set_Stream_Clear(void)	
VB	Function MT_Set_Stream_Clear () As Long	
Delphi	function MT_Set_Stream_Clear ():Integer;	
C#	public static extern int MT_Set_Stream_Clear();	
Input Parameters		
Function Returns	0	Function execution successful
	Non-zero	Function execution failed
Remarks		

3.16.6 MT_Set_Stream_Run_ID

Function Description: Inserts a run ID into the **stream** instruction sequence. The host computer can read the current ID to determine the execution position within the **stream** instruction queue.

C/VC/GCC	int32_t MT_Set_Stream_Run_ID(int32_t Value);	
VB	Function MT_Set_Stream_Run_ID(ByVal Value As Long) As Long	
Delphi	function MT_Set_Stream_Run_ID(Value:Integer):Integer;	
C#	public static extern int MT_Set_Stream_Run_ID(Int32 Value);	
Input Parameters	None	
Output Parameters	Value	ID
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks		

3.16.7 MT_Get_Stream_Run_ID

Function Description: Reads the current ID of the **Stream** sequence on the controller to determine the execution position within the **Stream** instruction queue.

C/VC/GCC	int32_t MT_Get_Stream_Run_ID (int32_t* pValue)	
VB	Function MT_Get_Stream_Run_ID (ByRef Value As Long) As Long	
Delphi	function MT_Get_Stream_Run_ID (pValue:PInteger):Integer;	
C#	public static extern int MT_Get_Stream_Run_ID (ref Int32 Value);	
Input Parameters	None	
Output	Value	Stream instruction space remaining

Parameters		
Function Return	0	Function execution successful, valid channel count read
	Non-zero	Function execution failed; the number of channels read is invalid
Remarks		

3.16.8 MT_Set_Stream_Delay

Function Description: Inserts a delay wait into the **Stream** instruction sequence with a precision of 1ms. Subsequent instructions will only execute after the delay has elapsed.

C/VC/GCC	int32_t MT_Set_Stream_Delay (int32_t Value);	
VB	Function MT_Set_Stream_Delay (ByVal Value As Long) As Long	
Delphi	function MT_Set_Stream_Delay (Value:Integer):Integer;	
C#	public static extern int MT_Set_Stream_Delay (Int32 Value);	
Input Parameters	None	
Output Parameters	Value	Waiting time in milliseconds
Function returns	0	Function execution successful
	Non-zero	Function execution failed
Remarks		

3.16.9 MT_Set_Stream_Line_Acc

Function Description: Sets the acceleration for the specified interpolation module in **Stream** mode.

C/VC/GCC	int32_t MT_Set_Stream_Line_Acc (WORD AObj, int32_t Value)	
VB	Function MT_Set_Stream_Line_Acc (ByVal AObj As Integer, ByVal Value As Long) As Long	
Delphi	function MT_Set_Stream_Line_Acc (AObj: Word; Value: Integer): Integer;	
C#	public static extern int MT_Set_Stream_Line_Acc (UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Interpolation module number, not the interpolation axis number
	Value	Acceleration for this interpolation module
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Interpolation module numbers start at 0. For the Nth interpolation module, the number is 0, 1, 2, 3...N-1	

3.16.10 MT_Set_Stream_Line_Dec

Function Description: Sets the acceleration for the specified interpolation module in **Stream** mode.

C/VC/GCC	int32_t MT_Set_Stream_Line_Dec (WORD AObj, int32_t Value)
VB	Function MT_Set_Stream_Line_Dec (ByVal AObj As Integer, ByVal Value As Long) As Long
Delphi	function MT_Set_Stream_Line_Dec (AObj:Word;Value:Integer):Integer;
C#	public static extern int MT_Set_Stream_Line_Dec (UInt16 AObj, Int32 Value);

Input Parameters	AObj	Interpolation module number, not the interpolation axis number
	Value	Deceleration value for this interpolation module
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Interpolation module numbers start at 0. For the Nth interpolation module, the number is 0, 1, 2, 3...N-1	

3.16.11 MT_Set_Stream_Line_V_Max

Function Description: Sets the speed of the specified interpolation module in **Stream** mode.

C/VC/GCC	int32_t MT_Set_Stream_Line_V_Max (WORD AObj, int32_t Value)	
VB	Function MT_Set_Stream_Line_V_Max(ByVal AObj As Integer, ByVal Value As Long) As Long	
Delphi	function MT_Set_Stream_Line_V_Max(AObj: Word; Value: Integer): Integer;	
C#	public static extern int MT_Set_Stream_Line_V_Max (UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Interpolation module number, not the interpolation axis number
	Value	Speed of this interpolation module
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	The serial numbers of interpolation modules start from 0. For N interpolation modules, the serial numbers are 0, 1, 2, 3...N-1.	

3.16.12 MT_Set_Stream_Line_V_Start

Function Description: Sets the starting velocity for the specified interpolation module in **Stream** mode. Generally, this instruction should not be used.

C/VC/GCC	int32_t MT_Set_Stream_Line_V_Start (WORD AObj, int32_t Value)	
VB	Function MT_Set_Stream_Line_V_Start(ByVal AObj As Integer, ByVal Value As Long) As Long	
Delphi	function MT_Set_Stream_Line_V_Start(AObj: Word; Value: Integer): Integer;	
C#	public static extern int MT_Set_Stream_Line_V_Start (UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Interpolation module number, not the interpolation axis number
	Value	Speed of this interpolation module
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Interpolation module numbers start at 0. For the Nth interpolation	

	module, the numbers are 0, 1, 2, 3...N-1
--	--

3.16.13 MT_Set_Stream_Line_Stop

Function Description: Decelerates and stops the interpolation axis in **Stream** mode.

C/VC/GCC	int32_t MT_Set_Stream_Line_Stop (WORD AObj)	
VB	Function MT_Set_Stream_Line_Stop (ByVal AObj As Integer) As Long	
Delphi	function MT_Set_Stream_Line_Stop (AObj:Word):Integer;	
C#	public static extern int MT_Set_Stream_Line_Stop (UInt16 AObj);	
Input Parameters	AObj	Interpolation module number, not the interpolation axis number
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Interpolation module numbers start at 0. For the Nth interpolation module, the number is 0, 1, 2, 3...N-1	

3.16.14 MT_Set_Stream_Line_Halt

Function Description: Immediately stops the interpolation axis in **Stream** mode.

C/VC/GCC	int32_t MT_Set_Stream_Line_Halt (WORD AObj)	
VB	Function MT_Set_Stream_Line_Halt (ByVal AObj As Integer) As Long	
Delphi	function MT_Set_Stream_Line_Halt (AObj:Word):Integer;	
C#	public static extern int MT_Set_Stream_Line_Halt (UInt16 AObj);	
Input Parameters	AObj	Interpolation module number, not the interpolation axis number
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Interpolation module numbers start at 0. For the Nth interpolation module, the number is 0, 1, 2, 3...N-1	

3.16.15 MT_Set_Stream_Line_X_Run_Rel

Function Description: **Stream** mode initiates interpolation in a relative manner. This command serves solely as an interpolation start signal. Since interpolation requires time to execute, if you need to wait for the interpolation command to complete, you must subsequently insert *an MT_Set_Stream_Wait_Line* command to coordinate the wait for completion. For example, the action sequence could be as follows: Start interpolation, output control, delay, output control, wait for interpolation completion.

```
//Initiate interpolation
MT_Set_Stream_Line_X_Run_Abs(0, 2, axis, target);
//Simultaneously activate heating relay
MT_Set_Stream_Optic_Out_Single(0, 1);
//Heat for 3 seconds
MT_Set_Stream_Delay(3000);
```

```
//Turn off heating  
MT_Set_Stream_Optic_Out_Single(0, 0);  
//Wait for interpolation to complete  
MT_Set_Stream_Wait_Line(0);
```

C/VC/GCC	int32_t int32_t AStream_Num, int32_t* pStream, int32_t* pTarget)	MT_Set_Stream_Line_X_Run_Rel(WORD AObj,
VB	Function MT_Set_Stream_Line_X_Run_Rel (ByVal Integer, ByVal	As AObj

	AStream_Num As Long, ByRef pStream As Long, ByRef pTarget As Long) As Long	
Delphi	function MT_Set_Stream_Line_X_Run_Rel (AObj: Word; AStream_Num: Integer; pStream: PInteger; pTarget: PInteger): Integer;	
C#	public static extern int MT_Set_Stream_Line_X_Run_Rel(UInt16 AObj, Int32 AStream_Num, ref Int32 pStream, ref Int32 pTarget)	
Input Parameters	AObj	Interpolation module number, not the interpolation axis number
	AStream_Num	Number of axes participating in interpolation 2-X
	pStream	Array of serial numbers for interpolated axes. An array must be passed in, and the length of this array must be greater than or equal to the number of axes participating in interpolation
	ATarget	An array of relative targets for the interpolation axes. An array must be passed in, and the length of this array must be greater than or equal to the number of interpolation axes
Function Returns	0	if the function executes successfully
	Non-zero	Function execution failed
Remarks	Interpolation module numbers start at 0. For the Nth interpolation module, the number is 0, 1, 2, 3...N-1	

3.16.16 MT_Set_Stream_Line_X_Run_Abs

Function Description: Initiates absolute interpolation in **Stream** mode. This command only starts the interpolation process; the actual movement takes time. To wait for the interpolation command to complete, insert *MT_Set_Stream_Wait_Line* afterward to coordinate the wait. For example, the action sequence could be: Start interpolation, output control, delay, output control, wait for interpolation completion.

```
//Start interpolation
MT_Set_Stream_Line_X_Run_Abs(0, 2, axis, target);
// Simultaneously activate heating relay
MT_Set_Stream_Optic_Out_Single(0, 1);
//Heat for 3 seconds
MT_Set_Stream_Delay(3000);
// Turn off heating
MT_Set_Stream_Optic_Out_Single(0, 0);
//Wait for interpolation to complete
MT_Set_Stream_Wait_Line(0);
```

C/VC/GCC	int32_t MT_Set_Stream_Line_X_Run_Abs(WORD AObj, int32_t AStream_Num, int32_t* pStream, int32_t* pTarget)
VB	Function MT_Set_Stream_Line_X_Run_Abs (ByVal AObj As Integer, ByVal AStream_Num As Long, ByRef pStream As Long, ByRef pTarget As Long) As Long
Delphi	function MT_Set_Stream_Line_X_Run_Abs (AObj: Word; AStream_Num: Integer; pStream: PInteger; pTarget: PInteger): Integer;
C#	public static extern int MT_Set_Stream_Line_X_Run_Abs(UInt16 AObj, Int32 AStream_Num, ref Int32 pStream, ref Int32 pTarget)

Input Parameters	AObj	Interpolation module number, not the interpolation axis number
	AStream_Num	Number of axes participating in interpolation 2-X
	pStream	Array of serial numbers for interpolated axes. An array must be passed in, and the length of this array must be greater than or equal to the number of axes participating in interpolation
	ATarget	Array of absolute targets for the interpolation axes. Must pass an array whose length must be greater than or equal to the number of interpolation axes
Function Returns	0	if the function executes successfully
	Non-zero	Function execution failed
Remarks	Interpolation module numbers start at 0. For the Nth interpolation module, the number is 0, 1, 2, 3...N-1	

3.16.17 MT_Set_Stream_Wait_Line

Function Description: Waits for the **Stream** mode interpolation to stop, blocking subsequent instruction execution.

C/VC/GCC	int32_t MT_Set_Stream_Wait_Line (WORD AObj)	
VB	Function MT_Set_Stream_Wait_Line (ByVal AObj As Integer) As Long	
Delphi	function MT_Set_Stream_Wait_Line (AObj:Word):Integer;	
C#	public static extern int MT_Set_Stream_Wait_Line (UInt16 AObj);	
Input Parameters	AObj	Interpolation module number, not the interpolation axis number
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Interpolation module indices start at 0. For the Nth interpolation module, the indices are 0, 1, 2, 3...N-1	

3.16.18 MT_Set_Stream_Circle_Axis

Function Description: In **Stream** mode, configure the motion axes participating in interpolation within a specified interpolation module. Different controllers feature varying numbers of interpolation modules, enabling multiple modules to perform simultaneous circular interpolation. Each interpolation module can freely designate any **two** axes for interpolation.

. Modules performing interpolation simultaneously must not share the same interpolation axis. For example: Module 0 uses axes 2 and 3 for interpolation, while Module 1 uses axes 1 and 2.

axes 1 and 2. These modules operate independently without issues when not running

concurrently and do not block subsequent commands.

C/VC/GCC	int32_t MT_Set_Stream_Circle_Axis (WORD AObj, int32_t Axis_ID0, int32_t Axis_ID1)	
VB	Function MT_Set_Stream_Circle_Axis (ByVal AObj As Integer, ByVal Axis_ID0 As Long, ByVal Axis_ID1 As Long) As Long	
Delphi	function MT_Set_Stream_Circle_Axis (AObj: Word; Axis_ID0, Axis_ID1: Integer): Integer;	
C#	public static extern int MT_Set_Stream_Circle_Axis (UInt16 AObj, Int32 Axis_ID0, Int32 Axis_ID1);	
Input Parameters	AObj	Interpolation module number, not the interpolation axis number
	Axis_ID0	First interpolation axis number for this interpolation module
	Axis_ID1	Second interpolation axis number for this interpolation module
Function Return	0	Function execution successful

	Non-zero	Function execution failed
Remarks	Interpolation module numbers start at 0 . For N interpolation modules, the numbers are 0, 1, 2, 3...N-1 Interpolation axis numbers start from 0 . For N axes, the numbers are 0, 1, 2, 3...N-1	

3.16.19 MT_Set_Stream_Circle_Acc

Function Description: Sets the acceleration for the specified interpolation module in **Stream** mode.

C/VC/GCC	int32_t MT_Set_Stream_Circle_Acc (WORD AObj, int32_t Value)	
VB	Function MT_Set_Stream_Circle_Acc (ByVal AObj As Integer, ByVal Value As Long) As Long	
Delphi	function MT_Set_Stream_Circle_Acc (AObj: Word; Value: Integer): Integer;	
C#	public static extern int MT_Set_Stream_Circle_Acc (UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Interpolation module number, not the interpolation axis number
	Value	Acceleration for this interpolation module
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Interpolation module numbers start at 0 . For the Nth interpolation module, the number is 0, 1, 2, 3...N-1	

3.16.20 MT_Set_Stream_Circle_Dec

Function Description: Sets the acceleration for the specified interpolation module in **Stream** mode.

C/VC/GCC	int32_t MT_Set_Stream_Circle_Dec (WORD AObj, int32_t Value)	
VB	Function MT_Set_Stream_Circle_Dec (ByVal AObj As Integer, ByVal Value As Long) As Long	
Delphi	function MT_Set_Stream_Circle_Dec (AObj: Word; Value: Integer): Integer;	
C#	public static extern int MT_Set_Stream_Circle_Dec (UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Interpolation module number, not the interpolation axis number
	Value	Deceleration value for this interpolation module
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Interpolation module indices start at 0 . For the Nth interpolation module, the indices are 0, 1, 2, 3...N-1	

3.16.21 MT_Set_Stream_Circle_V_Max

Function Description: Sets the speed of the specified interpolation module in Stream mode.

C/VC/GCC	int32_t MT_Set_Stream_Circle_V_Max (WORD AObj, int32_t Value)
VB	Function MT_Set_Stream_Circle_V_Max(ByVal AObj As Integer, ByVal Value As Long)

	As Long	
Delphi	function MT_Set_Stream_Circle_V_Max(AObj: Word; Value: Integer): Integer;	
C#	public static extern int MT_Set_Stream_Circle_V_Max (UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Interpolation module number, not the interpolation axis number
	Value	Speed of this interpolation module
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Interpolation module numbers start at 0. For the Nth interpolation module, the number is 0, 1, 2, 3...N-1	

3.16.22 MT_Set_Stream_Circle_V_Start

Function Description: Sets the start velocity for the specified interpolation module in **Stream** mode.

C/VC/GCC	int32_t MT_Set_Stream_Circle_V_Start (WORD AObj, int32_t Value)	
VB	Function MT_Set_Stream_Circle_V_Start(ByVal AObj As Integer, ByVal Value As Long) As Long	
Delphi	function MT_Set_Stream_Circle_V_Start(AObj: Word; Value: Integer): Integer;	
C#	public static extern int MT_Set_Stream_Circle_V_Start (UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Interpolation module number, not the interpolation axis number
	Value	Start velocity for this interpolation module
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Interpolation module numbers start at 0. For the Nth interpolation module, the number is 0, 1, 2, 3...N-1	

3.16.23 MT_Set_Stream_Circle_Stop

Function Description: Decelerate and stop the interpolation axis in **Stream** mode.

C/VC/GCC	int32_t MT_Set_Stream_Circle_Stop (WORD AObj)	
VB	Function MT_Set_Stream_Circle_Stop (ByVal AObj As Integer) As Long	
Delphi	function MT_Set_Stream_Circle_Stop (AObj:Word):Integer;	
C#	public static extern int MT_Set_Stream_Circle_Stop (UInt16 AObj);	
Input Parameters	AObj	Interpolation module number, not the interpolation axis number
	0	Function execution successful
Function Return	Non-zero	Function execution failed
Remarks	Interpolation module numbers start at 0. For the Nth interpolation	

	module, the number is 0, 1, 2, 3...N-1
--	--

3.16.24 MT_Set_Stream_Circle_Halt

Function Description: Immediately stops the interpolation axis in **Stream** mode.

C/VC/GCC	int32_t MT_Set_Stream_Circle_Halt (WORD AObj)	
VB	Function MT_Set_Stream_Circle_Halt (ByVal AObj As Integer) As Long	
Delphi	function MT_Set_Stream_Circle_Halt (AObj:Word):Integer;	
C#	public static extern int MT_Set_Stream_Circle_Halt (UInt16 AObj);	
Input Parameters	AObj	Interpolation module number, not the interpolation axis number
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Interpolation module numbers start at 0. For the Nth interpolation module, the numbers are 0, 1, 2, 3...N-1	

3.16.25 MT_Set_Stream_Circle_R_CW_Run_Rel

Function Description: Performs clockwise circular interpolation in **Stream** mode using parameters relative to the current position, initiating interpolation immediately.

C/VC/GCC	int32_t MT_Set_Stream_Circle_R_CW_Run_Rel (WORD AObj, int32_t AR, int32_t Axis_Target0, int32_t Axis_Target1)	
VB	Function MT_Set_Stream_Circle_R_CW_Run_Rel (ByVal AObj As Integer, ByVal AR As Long, ByVal Axis_Target0 As Long, ByVal Axis_Target1 As Long) As Long	
Delphi	Function MT_Set_Stream_Circle_R_CW_Run_Rel (AObj: Word; AR, Axis_Target0, Axis_Target1: Integer): Integer	
C	public static extern int MT_Set_Stream_Circle_R_CW_Run_Rel (UInt16 AObj, Int32 AR, Int32 Axis_Target0, Int32 Axis_Target1);	
Input Parameters	AObj	Interpolation module number, not the interpolation axis number
	AR	Radius for arc interpolation. Positive values represent minor arcs, negative values represent major arcs. The radius is half the arc's diameter. . No distinction between relative and absolute
	Axis_Target0	Relative displacement distance for the first interpolation axis in this module
	Axis_Target1	The relative displacement distance for the second interpolation axis in this interpolation module
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Interpolation module numbers start at 0. For the Nth interpolation module, the numbers are 0, 1, 2, 3...N-1	

3.16.26 MT_Set_Stream_Circle_R_CW_Run_Abs

Function Description: In **Stream** mode, perform clockwise arc interpolation based on absolute position parameters, initiating interpolation immediately.

C/VC/GCC	int32_t MT_Set_Stream_Circle_R_CW_Run_Abs (WORD AObj, int32_t AR, int32_t Axis_Target0, int32_t Axis_Target1)
VB	Function MT_Set_Stream_Circle_R_CW_Run_Abs (ByVal AObj As Integer, ByVal AR As Long, ByVal Axis_Target0 As Long, ByVal Axis_Target1 As Long) As Long
Delphi	Function MT_Set_Stream_Circle_R_CW_Run_Abs (AObj: Word; AR, Axis_Target0, Axis_Target1: Integer): Integer

C	public static extern int MT_Set_Stream_Circle_R_CW_Run_Abs (UInt16 AObj, Int32 AR, Int32 Axis_Target0, Int32 Axis_Target1);	
Input Parameters	AObj	Interpolation module number, not the interpolation axis number
	AR	Radius for arc interpolation. Positive values indicate minor arcs, negative values indicate major arcs. The radius represents half the arc's diameter. . No distinction between relative and absolute
	Axis_Target0	Absolute position to which the first interpolation axis of this module must move
	Axis_Target1	The absolute position to which the second interpolation axis of this interpolation module must move.
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Interpolation module numbers start at 0. For the Nth interpolation module, the numbers are 0, 1, 2, 3...N-1	

3.16.27 MT_Set_Stream_Circle_R_CCW_Run_Rel

Function Description: Performs counterclockwise circular interpolation relative to the current position in **Stream** mode, initiating interpolation immediately.

C/VC/GCC	int32_t MT_Set_Stream_Circle_R_CCW_Run_Rel (WORD AObj, int32_t AR, int32_t Axis_Target0, int32_t Axis_Target1)	
VB	Function MT_Set_Stream_Circle_R_CCW_Run_Rel (ByVal AObj As Integer, ByVal AR As Long, ByVal Axis_Target0 As Long, ByVal Axis_Target1 As Long) As Long	
Delphi	Function MT_Set_Stream_Circle_R_CCW_Run_Rel (AObj: Word; AR, Axis_Target0, Axis_Target1: Integer): Integer	
C	public static extern int MT_Set_Stream_Circle_R_CCW_Run_Rel (UInt16 AObj, Int32 AR, Int32 Axis_Target0, Int32 Axis_Target1);	
Input Parameters	AObj	Interpolation module number, not the interpolation axis number
	AR	Radius for arc interpolation. Positive values represent minor arcs, negative values represent major arcs. The radius is half the arc's diameter. . No distinction between relative and absolute
	Axis_Target0	Relative displacement distance for the first interpolation axis in this module
	Axis_Target1	The relative displacement distance for the second interpolation axis in this interpolation module
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Interpolation module numbers start at 0. For the Nth interpolation	

	module, the numbers are 0, 1, 2, 3...N-1
--	--

3.16.28 MT_Set_Stream_Circle_R_CCW_Run_Abs

Function Description: Performs counterclockwise circular interpolation at absolute position parameters in **Stream** mode, initiating interpolation immediately.

C/VC/GCC	int32_t MT_Set_Stream_Circle_R_CCW_Run_Abs (WORD AObj, int32_t AR, int32_t Axis_Target0, int32_t Axis_Target1)
VB	Function MT_Set_Stream_Circle_R_CCW_Run_Abs (ByVal AObj As Integer, ByVal AR As Long, ByVal Axis_Target0 As Long, ByVal Axis_Target1 As Long) As Long
Delphi	Function MT_Set_Stream_Circle_R_CCW_Run_Abs

	(AObj: Word; AR, Axis_Target0, Axis_Target1: Integer): Integer	
C#	public static extern int MT_Set_Stream_Circle_R_CCW_Run_Abs (UInt16 AObj, Int32 AR, Int32 Axis_Target0, Int32 Axis_Target1);	
Input Parameters	AObj	Interpolation module number, not the interpolation axis number
	AR	Radius for arc interpolation. Positive values represent minor arcs, negative values represent major arcs. The radius is half the arc's diameter. . No distinction between relative and absolute
	Axis_Target0	Absolute position to which the first interpolation axis of this module must move
	Axis_Target1	The absolute position to which the second interpolation axis of this interpolation module must move.
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Interpolation module numbers start at 0. For the Nth interpolation module, the number is 0, 1, 2, 3...N-1	

3.16.29 MT_Set_Stream_Wait_Circle

Function Description: Waits for the **Stream** mode circular interpolation to complete, blocking subsequent instruction execution.

C/VC/GCC	int32_t MT_Set_Stream_Wait_Circle (WORD AObj)	
VB	Function MT_Set_Stream_Wait_Circle (ByVal AObj As Integer) As Long	
Delphi	function MT_Set_Stream_Wait_Circle(AObj:Word):Integer;	
C#	public static extern int MT_Set_Stream_Wait_Circle (UInt16 AObj);	
Input Parameters	AObj	Interpolation module number, not the interpolation axis number
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Interpolation module numbers start at 0. For the Nth interpolation module, the number is 0, 1, 2, 3...N-1	

3.16.30 MT_Set_Stream_Optic_Out_Single

Function Description: Sets the level state on a specified general-purpose output channel in **Stream** mode.

C/VC/GCC	int32_t MT_Set_Stream_Optic_Out_Single (WORD AObj, int32_t pValue);
----------	---

VB	Function MT_Set_Stream_Optic_Out_Single (ByVal AObj As Integer, ByVal Value As Long) As Long	
Delphi	function MT_Set_Stream_Optic_Out_Single (AObj:Word;Value:Integer):Integer;	
C#	public static extern int MT_Set_Stream_Optic_Out_Single (UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Optically isolated output channel number
Output Parameters	Value	Level status on the specified channel; 1 indicates high level, 0 indicates low level
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Channel indices start at 0 . For N channels, the indices are 0, 1, 2, 3, ..., N-1 .	

3.16.31 MT_Set_Stream_Optic_Out_All

Function Description: Sets the level state on the universal output channel in **Stream** mode.

C/VC/GCC	int32_t MT_Set_Stream_Optic_Out_All (int32_t Value);	
VB	Function MT_Set_Stream_Optic_Out_All (ByVal Value As Long) As Long	
Delphi	function MT_Set_Stream_Optic_Out_All (Value:Integer):Integer;	
C#	public static extern int MT_Set_Stream_Optic_Out_All (Int32 Value);	
Input Parameters	None	
Output Parameters	Value	Level status on all channels: 1 indicates high level, 0 indicates low level Channel 0 is LSB (Least Significant Bit) Channel N corresponds to the N-1 bit
Function Returns	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Channel indices start at 0 . For N channels, the indices are 0, 1, 2, 3...N-1	

3.16.32 MT_Set_Stream_Dec_Enable

Function Description: Enables automatic deceleration. In **Stream** mode, during motion (multi-axis interpolation and circular interpolation), deceleration is permitted between consecutive segments before acceleration, similar to consecutive interpolations in **non-Stream** mode. Use this configuration for interpolation not involving consecutive small line segments or single-axis motion. Automatic deceleration is enabled by default after power-up.

C/VC/GCC	int32_t MT_Set_Stream_Dec_Enable (void)	
VB	Function MT_Set_Stream_Dec_Enable () As Long	
Delphi	function MT_Set_Stream_Dec_Enable ():Integer;	
C#	public static extern int MT_Set_Stream_Dec_Enable ();	
Input Parameters		
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks		

3.16.33 MT_Set_Stream_Dec_Disable

Function Description: In **Stream** mode, consecutive segments of motion (linear and circular interpolation) must not include deceleration followed by acceleration

between them. However, speed can be modified via speed setting commands. It is recommended to allow acceleration/deceleration only between the first segment and the final path segment, prohibiting deceleration in intermediate segments. Macro-wise, this results in a continuous sequence of acceleration, constant velocity, and deceleration.

C/VC/GCC	int32_t MT_Set_Stream_Dec_Disable (void)	
VB	Function MT_Set_Stream_Dec_Disable () As Long	
Delphi	function MT_Set_Stream_Dec_Disable ():Integer;	
C#	public static extern int MT_Set_Stream_Dec_Disable ();	
Input Parameters		

Function	0	Function execution successful
Return	Non-zero	Function execution failed
Remarks		

3.17 Synchronous Trigger

After power-on, the synchronous trigger function is disabled by default. First configure the synchronous trigger parameters, then enable the synchronous trigger function. Subsequently, when positioning motion is initiated, synchronous trigger pulses will be automatically output during the motion based on the current position.

For example, within the coordinate system (5000, 50000), a positive trigger pulse is output every 5000 units. That is, at coordinates 5000, 10000, 15000...

Currently, only open-loop synchronous triggering is supported.

```
//Set the valid coordinate range for automatic triggering on Axis 0 (first axis), including
start and end points
// Automatic triggering occurs whenever the coordinate falls within this range.
MT_Set_Trigger_Start(0, 5000);
MT_Set_Trigger_End(0, 50000);
//Trigger interval: Outputs one trigger pulse every 5000 coordinate intervals
MT_Set_Trigger_Step(0, 5000);
// Trigger pulse width: 10us
MT_Set_Trigger_Width(0, 10);
// Positive pulse (default). Can be omitted.
MT_Set_Trigger_Polarity(0, 1);
// Enable trigger function
MT_Set_Trigger_Enable(0, 1);
```

3.17.1 MT_Set_Trigger_Enable

Function Description: Sets the enable status of synchronous triggering for the specified axis. By default, synchronous triggering for motor axes is disabled after power-up.

C/VC/GCC	int32_t MT_Set_Trigger_Enable (WORD AObj, int32_t pValue);
VB	Function MT_Set_Trigger_Enable (ByVal AObj As Integer, ByVal Value As Long) As Long
Delphi	function MT_Set_Trigger_Enable (AObj: Word; Value: Integer): Integer;
C#	public static extern int MT_Set_Trigger_Enable (UInt16 AObj, Int32 Value);

Input Parameters	AObj	Motor control axis serial number
Output Parameters	Value	Enables/disables automatic synchronization triggering for the specified axis. 1 = Enabled, 0 = Disabled
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Motor shaft serial numbers start from 0 . For an N-axis card, the serial numbers are 0, 1, 2, 3...N-1 Ensure the motor shaft index function does not exceed bounds	

3.17.2 MT_Get_Trigger_Enable

Function Description: Reads the status of the synchronous trigger switch.

C/VC/GCC	int32_t MT_Get_Trigger_Enable (WORD AObj, int32_t* pValue);	
VB	Function MT_Get_Trigger_Enable(ByVal AObj As Integer, ByRef Value As Long) As Long	
Delphi	function MT_Get_Trigger_Enable (AObj:Word; pValue:PInteger):Integer;	
C#	public static extern int MT_Get_Trigger_Enable (UInt16 AObj, ref Int32 Value);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	Value	Automatic synchronization trigger status for the specified axis: 1=enabled, 0=disabled
Function Return	0	Function executed successfully; read data is valid
	Non-zero	Function execution failed; read data is invalid
Remarks	Motor shaft serial numbers start from 0. For an N-axis card, serial numbers are 0, 1, 2, 3...N-1 Ensure the motor shaft serial number function does not exceed bounds	

3.17.3 MT_Set_Trigger_Start

Function Description: Sets the start coordinate for synchronous triggering on the specified axis. The default value is 0. The start coordinate must be less than the end coordinate.

C/VC/GCC	int32_t MT_Set_Trigger_Start(WORD AObj, int32_t pValue);	
VB	Function MT_Set_Trigger_Start(ByVal AObj As Integer, ByVal Value As Long) As Long	
Delphi	function MT_Set_Trigger_Start (AObj:Word;Value:Integer):Integer;	
C#	public static extern int MT_Set_Trigger_Start(UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	Value	Coordinate Value
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Motor shaft serial numbers start from 0. For an N-axis card, the serial numbers are 0, 1, 2, 3...N-1 Ensure the motor shaft index function does not exceed bounds	

3.17.4 MT_Get_Trigger_Start

Function Description: Reads the start coordinates of the synchronous trigger.

C/VC/GCC	<code>int32_t MT_Get_Trigger_Start(WORD AObj, int32_t* pValue);</code>
VB	<code>Function MT_Get_Trigger_Start(ByVal AObj As Integer, ByRef Value As Long) As Long</code>
Delphi	<code>function MT_Get_Trigger_Start(AObj: Word; pValue: PInteger): Integer;</code>
C#	<code>public static extern int MT_Get_Trigger_Start(UInt16 AObj, ref Int32 Value);</code>

Input Parameters	AObj	Motor control axis serial number
Output Parameters	Value	Coordinate Value
Function Return	0	Function execution successful, read data is valid
	Non-zero	Function execution failed; the read data is invalid
Notes	Motor shaft serial numbers start from 0. For an N-axis card, serial numbers are 0, 1, 2, 3...N-1 Ensure the motor shaft serial number function does not exceed bounds	

3.17.5 MT_Set_Trigger_End

Function Description: Sets the endpoint coordinates for synchronous triggering on the specified axis. The default value is 1000000. The start coordinate must be less than the endpoint coordinate.

C/VC/GCC	int32_t MT_Set_Trigger_End(WORD AObj, int32_t pValue);	
VB	Function MT_Set_Trigger_End(ByVal AObj As Integer, ByVal Value As Long) As Long	
Delphi	function MT_Set_Trigger_End(AObj:Word;Value:Integer):Integer;	
C#	public static extern int MT_Set_Trigger_End(UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	Value	Coordinate Value
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	The motor shaft serial numbers start from 0. For the Nth shaft, the serial numbers are 0, 1, 2, 3...N-1. When using the motor shaft index function, ensure that the index does not exceed the bounds.	

3.17.6 MT_Get_Trigger_End

Function Description: Reads the endpoint coordinates of the synchronous trigger.

C/VC/GCC	int32_t MT_Get_Trigger_End(WORD AObj, int32_t* pValue);	
VB	Function MT_Get_Trigger_End(ByVal AObj As Integer, ByRef Value As Long) As Long	
Delphi	function MT_Get_Trigger_End(AObj: Word; pValue: PInteger): Integer;	
C#	public static extern int MT_Get_Trigger_End(UInt16 AObj, ref Int32 Value);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	Value	Coordinate Value

Function	0	Function execution successful, read data is valid
Return	Non-zero	Function execution failed; the read data is invalid
Notes	Motor shaft serial numbers start from 0 . For an N-axis card, serial numbers are 0, 1, 2, 3...N-1 Ensure the motor shaft index function does not exceed bounds	

3.17.7 MT_Set_Trigger_Step

Function Description: Sets the synchronous trigger interval for the specified axis. The default value after power-up is **1000**. Outputs a trigger pulse every specified coordinate value interval.

pulse.

C/VC/GCC	int32_t MT_Set_Trigger_Step(WORD AObj, int32_t pValue);	
VB	Function MT_Set_Trigger_Step(ByVal AObj As Integer, ByVal Value As Long) As Long	
Delphi	function MT_Set_Trigger_Step(AObj:Word;Value:Integer):Integer;	
C#	public static extern int MT_Set_Trigger_Step(UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	Value	Interval value
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Motor shaft serial numbers start from 0. For an N-axis card, the serial numbers are 0, 1, 2, 3...N-1 Ensure the motor shaft index function does not exceed bounds	

3.17.8 MT_Get_Trigger_Step

Function Description: Reads the trigger interval for synchronous triggering.

C/VC/GCC	int32_t MT_Get_Trigger_Step(WORD AObj, int32_t* pValue);	
VB	Function MT_Get_Trigger_Step(ByVal AObj As Integer, ByRef Value As Long) As Long	
Delphi	function MT_Get_Trigger_Step(AObj: Word; pValue: PInteger): Integer;	
C#	public static extern int MT_Get_Trigger_Step(UInt16 AObj, ref Int32 Value);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	Value	Interval value
Function Return	0	Function execution successful, read data is valid
	Non-zero	Function execution failed; the read data is invalid
Notes	Motor shaft serial numbers start from 0. For the Nth shaft card, serial numbers are 0, 1, 2, 3...N-1. The function for motor shaft serial numbers must ensure no out-of-bounds access occurs.	

3.17.9 MT_Set_Trigger_Width

Function Description: Sets the pulse width (in microseconds) for synchronous triggering pulses on the specified axis. The default value after power-up is 100. This value should be selected based on the controller's capabilities, motion speed, and application requirements.

C/VC/GCC	int32_t MT_Set_Trigger_Width(WORD AObj, int32_t pValue);	
VB	Function MT_Set_Trigger_Width(ByVal AObj As Integer, ByVal Value As Long) As Long	
Delphi	function MT_Set_Trigger_Width(AObj:Word;Value:Integer):Integer;	
C#	public static extern int MT_Set_Trigger_Width(UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	Value	Pulse width, unit: us
Function Return	0	Function execution successful
	Non-zero	Function execution failed

Remarks	Motor shaft serial numbers start from 0. For an N-axis card, the serial numbers are 0, 1, 2, 3...N-1 Ensure the motor shaft index function does not exceed bounds
---------	--

3.17.10 MT_Get_Trigger_Width

Function Description: Reads the synchronous trigger pulse width.

C/VC/GCC	int32_t MT_Get_Trigger_Width (WORD AObj, int32_t* pValue);	
VB	Function MT_Get_Trigger_Width(ByVal AObj As Integer, ByRef Value As Long) As Long	
Delphi	function MT_Get_Trigger_Width(AObj:Word; pValue:PInteger):Integer;	
C#	public static extern int MT_Get_Trigger_Width(UInt16 AObj, ref Int32 Value);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	Value	Pulse width, unit: us
Function Return	0	Function execution successful, read data valid
	Non-zero	Function execution failed; the read data is invalid
Notes	Motor shaft serial numbers start from 0. For an N-axis card, serial numbers are 0, 1, 2, 3...N-1 Ensure the motor shaft serial number function does not exceed bounds	

3.17.11 MT_Set_Trigger_Polarity

Function Description: Sets the level of the synchronous trigger pulse for the specified axis. The default is a positive pulse after power-up.

C/VC/GCC	int32_t MT_Set_Trigger_Polarity(WORD AObj, int32_t pValue);	
VB	Function MT_Set_Trigger_Polarity(ByVal AObj As Integer, ByVal Value As Long) As Long	
Delphi	function MT_Set_Trigger_Polarity(AObj: Word; Value: Integer): Integer;	
C#	public static extern int MT_Set_Trigger_Polarity(UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Motor Control Axis Number
Output Parameters	Value	1 indicates positive pulse, 0 indicates negative pulse
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Motor shaft serial numbers start from 0. For an N-axis card, the serial numbers are 0, 1, 2, 3...N-1 Ensure the motor shaft serial number function does not exceed bounds	

3.17.12 MT_Get_Trigger_Polarity

Function Description: Reads the level of the synchronous trigger pulse.

C/VC/GCC	int32_t MT_Get_Trigger_Polarity(WORD AObj, int32_t* pValue);
VB	Function MT_Get_Trigger_Polarity(ByVal AObj As Integer, ByRef Value As Long) As

	Long	
Delphi	function MT_Get_Trigger_Polarity(AObj: Word; pValue: PInteger): Integer;	
C#	public static extern int MT_Get_Trigger_Polarity(UInt16 AObj, ref Int32 Value);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	Value	1 indicates positive pulse, 0 indicates negative pulse
Function Return	0	Function execution successful, read data valid
	Non-zero	Function execution failed; the read data is invalid
Notes	Motor shaft serial numbers start from 0. For an N-axis card, serial numbers are 0, 1, 2, 3...N-1 Ensure the motor shaft serial number function does not exceed bounds	

3.17.13 MT_Set_Trigger_Channel

Function Description: Sets the channel mapping for synchronous triggering on a specified axis. Generally, avoid using this function. By default, motor axis numbers correspond one-to-one with trigger IO numbers: axes 0..N-1 correspond to IOs 0..N-1. This function allows specifying the IO for an axis, e.g., outputting triggers for **two** axes from a single IO.

C/VC/GCC	int32_t MT_Set_Trigger_Channel(WORD AObj, int32_t pValue);	
VB	Function MT_Set_Trigger_Channel(ByVal AObj As Integer, ByVal Value As Long) As Long	
Delphi	function MT_Set_Trigger_Channel(AObj: Word; Value: Integer): Integer;	
C#	public static extern int MT_Set_Trigger_Channel(UInt16 AObj, Int32 Value);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	Value	Trigger IO Serial Number
Function Return	0	Function execution successful
	Non-zero	Function execution failed
Remarks	Motor shaft serial numbers start from 0. For an N-axis card, the serial numbers are 0, 1, 2, 3...N-1 Ensure the motor shaft index function does not exceed bounds.	

3.17.14 MT_Get_Trigger_Channel

Function Description: Reads the trigger IO sequence number corresponding to the synchronous trigger channel.

C/VC/GCC	int32_t MT_Get_Trigger_Channel(WORD AObj, int32_t* pValue);
----------	---

VB	Function MT_Get_Trigger_Channel(ByVal AObj As Integer, ByRef Value As Long) As Long	
Delphi	function MT_Get_Trigger_Channel(AObj: Word; pValue: PInteger): Integer;	
C#	public static extern int MT_Get_Trigger_Channel(UInt16 AObj, ref Int32 Value);	
Input Parameters	AObj	Motor control axis serial number
Output Parameters	Value	Trigger IO Serial Number
Function Return	0	Function executed successfully, data read is valid
	Non-zero	Function execution failed; the read data is invalid

Notes	Motor shaft serial numbers start from 0. For an N-axis card, serial numbers are 0, 1, 2, 3...N-1 Ensure motor shaft number functions do not exceed bounds
-------	--

3.18 Auxiliary Calculations [Compatibility]

Practical formulas for auxiliary calculations during user programming, facilitating conversion between physical quantities and actual motion controller I/O. If open-loop and closed-loop coefficients from Section 1.2 are calculated via supporting tools, functions in this section are unnecessary. Each axis motor records only 3 parameters.

3.18.1 MT_Help_Step_Line_Real_To_Steps

Function Description: Conversion calculation from linear stage physical quantities (velocity, acceleration, deceleration, etc.) to open-loop pulses

C/VC/GCC	int32_t MT_Help_Step_Line_Real_To_Steps(double AStepAngle, int32_t ADiv, double APitch, double ALineRatio, double AValue)	
VB	Function MT_vStep_Line_Real_To_Steps (ByVal AStepAngle As Double, ByVal ADiv As Long, ByVal APitch As Double, ByVal ALineRatio As Double, ByVal AValue As Double) As Long	
Delphi	function MT_Help_Step_Line_Real_To_Steps(AStepAngle: Double; ADiv: Integer; APitch: Double; ALineRatio: Double; AValue: Double): Integer	
C#	public static extern int MT_Help_Step_Line_Real_To_Steps(double AStepAngle, int32 ADiv, double APitch, double ALineRatio, double AValue)	
Input Parameters	AStepAngle	Motor rotation angle per step, e.g., typically 1.8° for two-phase stepper motors
	ADiv	Driver microstep division, typically 2, 4, 8, 16, etc.
	APitch	Linear stage pitch, i.e., displacement per full motor revolution, in mm
	ALineRatio	Linear stage transmission ratio; 1 when no transmission is present
	AValue	Physical quantity, mm, mm/s, mm/s ²
The function returns	32-bit integer	converted steps
	Non-zero	Function execution failed
Remarks		

3.18.2 MT_Help_Step_Circle_Real_To_Steps

Function Description: Conversion calculation of rotary table physical quantities (velocity, acceleration, deceleration, etc.) to open-loop pulses

C/VC/GCC	int32_t MT_Help_Step_Circle_Real_To_Steps(double AStepAngle, int32_t ADiv, double ACircleRatio, double AValue)
VB	Function MT_Help_Step_Circle_Real_To_Steps (ByVal AStepAngle As Double, ByVal ADiv As Long, ByVal ACircleRatio As Double, ByVal AValue As Double) As Long
Delphi	function MT_vStep_Circle_Real_To_Steps(AStepAngle: Double; ADiv: Integer; ACircleRatio: Double; AValue: Double): Integer

C#	<code>public static extern int MT_ Help_Step_Circle_Real_To_Steps(double AStepAngle, Int32 ADiv, double ACircleRatio, double AValue)</code>	
Input Parameters	AStepAngle	Motor rotation angle per step, e.g., 1.8° for most stepper motors
	ADiv	Driver microstepping resolution, typically 2, 4, 8, 16, etc.
	ACircleRatio	Rotary stage transmission ratio
	AValue	Physical quantity, °, °/s, °/s ²
Function Return	32-bit integer	Converted Steps
	Non-zero	Function execution failed
Remarks		

3.18.3 MT_ Help_Step_Line_Steps_To_Real

Function Description: Linear motion stage pulse-to-physical quantity conversion (velocity, acceleration, deceleration, etc.)

C/VC/GCC	<code>double MT_ Help_Step_Line_Steps_To_Real(double AStepAngle, int32_t ADiv, double APitch, double ALineRatio, double AValue)</code>	
VB	Function MT_ Help_Step_Line_Steps_To_Real(ByVal AStepAngle As Double, ByVal ADiv As Long, ByVal APitch As Double, ByVal ALineRatio As Double, ByVal AValue As Long) As Double	
Delphi	function MT_ Help_Step_Line_Steps_To_Real(AStepAngle: Double; ADiv: Integer; APitch: Double; ALineRatio: Double; AValue: Integer): Double	
C#	<code>public static extern double MT_ Help_Step_Line_Steps_To_Real(double AStepAngle, Int32 ADiv, double APitch, double ALineRatio, int32_t AValue)</code>	
Input Parameters	AStepAngle	Motor rotation angle per step, e.g., typically 1.8° for a two-phase stepper motor
	ADiv	Driver microstepping resolution, typically 2, 4, 8, 16, etc.
	APitch	Linear stage pitch, i.e., displacement per full motor revolution, in mm
	ALineRatio	Linear stage transmission ratio; 1 when no transmission is present
	AValue	Number of pulses
Function Return	Double precision	Converted physical quantity, mm, mm/s, mm/s ²
	Non-zero	Function execution failed
Remarks		

3.18.4 MT_ Help_Step_Circle_Steps_To_Real

Function Description: Converting rotary table pulses to physical quantities

(velocity, acceleration, deceleration, etc.)

C/VC/GCC	double MT_ Help_Step_Circle_Steps_To_Real(double AStepAngle, int32_t ADiv, double ACircleRatio, int32_t AValue)
VB	Function MT_ Help_Step_Circle_Steps_To_Real (ByVal AStepAngle As Double, ByVal ADiv As Long, ByVal ACircleRatio As Double, ByVal AValue As Double) As Long
Delphi	function MT_Help_Step_Circle_Steps_To_Real(AStepAngle: Double; ADiv: Integer; ACircleRatio: Double; AValue: Double): Integer
C#	public static extern int MT_ Help_Step_Circle_Steps_To_Real(double

	AStepAngle, Int32 ADiv, double ACircleRatio, double AValue)	
Input Parameters	AStepAngle	Motor rotation angle per step, e.g., 1.8° for most stepper motors
	ADiv	Driver microstepping resolution, typically 2, 4, 8, 16, etc.
	ACircleRatio	Rotary stage transmission ratio
	AValue	Number of pulses
Function returns	Double-precision floating-point	Converted Physical Quantity °, °/s, °/s ²
	Non-zero	Function execution failed
Remarks		

3.18.5 MT_Help_Encoder_Line_Real_To_Steps

Function Description: Conversion calculation of physical quantities (velocity, acceleration, deceleration, etc.) from a linear motion stage to closed-loop encoder pulses. This conversion function is used when the encoder is mounted on the motor. The encoder's electronic quadrupling is already accounted for during conversion; users need not consider it further.

C/VC/GCC	int32_t MT_Help_Encoder_Line_Real_To_Steps(double APitch, double ALineRatio, int32_t ALineCount, double AValue)	
VB	Function MT_Help_Encoder_Line_Real_To_Steps (ByVal APitch As Double, ByVal ALineRatio As Double, ByVal ALineCount As Long, ByVal AValue As Double) As Long	
Delphi	function MT_Help_Encoder_Line_Real_To_Steps(APitch:Double;ALineRatio:Double;ALineCount:Integer;AValue:Double):Integer	
C#	public static extern int MT_Help_Encoder_Line_Real_To_Steps(double APitch, double ALineRatio, Int32 ALineCount, double AValue)	
	APitch	Linear stage pitch, i.e., displacement per motor revolution, in mm
	ALineRatio	Linear stage transmission ratio; 1 when no transmission
	ALineCount	Number of encoder lines, e.g., 1000, 1024, etc.
	AValue	Physical quantity, mm, mm/s, mm/s ²
Function return	32-bit integer	Converted Encoder Pulse Count
	Non-zero	Function execution failed
Remarks		

3.18.6 MT_Help_Encoder_Circle_Real_To_Steps

Function Description: Conversion calculation of rotary table physical quantities (speed, acceleration, deceleration, etc.) to closed-loop encoder pulses. Use this conversion function for encoders mounted on motors. The encoder's electronic quadrupling is already accounted for during conversion; users need not consider it further.

C/VC/GCC	int32_t MT_Help_Encoder_Circle_Real_To_Steps(double ACircleRatio, int32_t ALineCount, double AValue)
VB	Function MT_Help_Encoder_Circle_Real_To_Steps(ByVal ACircleRatio As Double, ByVal ALineCount As Long, ByVal AValue As Double) As Long
Delphi	function MT_Help_Encoder_Circle_Real_To_Steps(ACircleRatio: Double; ALineCount: Integer; AValue: Double): Integer
C#	public static extern int MT_Help_Encoder_Circle_Real_To_Steps(double

	ACircleRatio, Int32 ALineCount, double AValue)	
	ACircleRatio	Rotary Table Transmission Ratio
	ALineCount	Number of encoder lines, e. g. , 1000, 1024, etc.
	AValue	Physical quantity, ° , °/s, °/s ²
Function Return	32-bit integer	Converted step count
	Non-zero	Function execution failed
Remarks		

3.18.7 MT_Help_Encoder_Line_Steps_To_Real

Function Description: Converts encoder pulses from a linear motion stage into physical quantities (velocity, acceleration, deceleration, etc.). Use this conversion function for encoders mounted on motors. The encoder's electronic quadrature multiplication is already accounted for during conversion; users need not consider it further. Simply input the pulse count after quadrature multiplication.

C/VC/GCC	double MT_Help_Encoder_Line_Steps_To_Real(double APitch, double ALineRatio, int32_t ALineCount, int32_t AValue)	
VB	Function MT_Help_Encoder_Line_Steps_To_Real(ByVal APitch As Double, ByVal ALineRatio As Double, ByVal ALineCount As Long, ByVal AValue As Long) As Long	
Delphi	function MT_Help_Encoder_Line_Steps_To_Real(APitch: Double; ALineRatio: Double; ALineCount: Integer; AValue: Integer): Double	
C#	public static extern double MT_Help_Encoder_Line_Steps_To_Real(double APitch, double ALineRatio, Int32 ALineCount, Int32 AValue)	
Input	APitch	Linear stage pitch, i.e., displacement per motor revolution, in mm
	ALineRatio	Linear stage transmission ratio; 1 when no transmission
	ALineCount	Number of encoder lines, e. g. , 1000, 1024, etc.
	AValue	Number of pulses
Function Return	Double precision	Converted physical quantity, mm, mm/s, mm/s ²
	Non-zero	Function execution failed
Remarks		

3.18.8 MT_Help_Encoder_Circle_Steps_To_Real

Function Description: Converts encoder pulses from the rotary table to physical quantities (velocity, acceleration, deceleration, etc.). For encoders mounted on motors, this conversion function accounts for the encoder's electronic quadrature conversion. Users need not consider this factor; simply input the pulse count after quadrature conversion.

C/VC/GCC	double MT_ Help_Encoder_Circle_Steps_To_Real(double ACircleRatio, int32_t ALineCount, int32_t AValue)
VB	Function MT_ Help_Encoder_Circle_Steps_To_Real Lib "MT_API.dll" (ByVal ACircleRatio As Double, ByVal ALineCount As Long, ByVal AValue As Long) As Long
Delphi	function MT_Help_Encoder_Circle_Steps_To_Real(ACircleRatio: Double; ALineCount: Integer; AValue: Integer): Double
C#	public static extern int MT_ Help_Encoder_Circle_Steps_To_Real(double

	ACircleRatio, Int32 ALineCount, Int32 AValue)	
	ACircleRatio	Rotary Table Transmission Ratio
	ALineCount	Encoder line count, e. g. , 1000, 1024, etc.
	AValue	Number of pulses
Function Return	Double-precision floating-point	Converted physical quantity $^{\circ}$, $^{\circ}/s$, $^{\circ}/s^2$
	Non-zero	Function execution failed
Remarks		

3.18.9 MT_Help_Grating_Line_Real_To_Steps

Function Description: Conversion calculation of linear motion stage physical quantities (velocity, acceleration, deceleration, etc.) to closed-loop grating scale pulse counts. The grating scale is mounted on the mechanical structure, and its electronic **quadruple** frequency multiplication is already accounted for during conversion—users need not consider this further.

C/VC/GCC	int32_t MT_Help_Grating_Line_Real_To_Steps(double AUnit_um, double AValue)	
VB	Function MT_Help_Grating_Line_Real_To_Steps(ByVal AUnit_um As Double, ByVal AValue As Double) As Long	
Delphi	function MT_Help_Grating_Line_Real_To_Steps(AUnit_um:Double;AValue:Double):Integer	
C#	public static extern int MT_Help_Grating_Line_Real_To_Steps(double AUnit_um, double AValue)	
	AUnit_um	Grating pitch of the grating scale, in um
	AValue	Physical quantity, mm, mm/s, mm/s ²
Function Return	32-bit integer	Converted grating scale pulse count
	Non-zero	Function execution failed
Remarks		

3.18.10 MT_Help_Grating_Circle_Real_To_Steps

Function Description: Calculates the conversion of rotational stage physical quantities (velocity, acceleration, deceleration, etc.) to grating scale pulses. The grating scale is mounted on the mechanical structure. The electronic **quadruple** frequency of the grating scale is already accounted for during conversion; users need not consider it further.

C/VC/GCC	int32_t MT_Help_Grating_Circle_Real_To_Steps(int32_t ALineCount, double AValue)	
VB	Function MT_Help_Grating_Circle_Real_To_Steps(ByVal ALineCount As Long, ByVal AValue As Double) As Long	

Delphi	function MT_ Help_Grating_Circle_Real_To_Steps(ALineCount: Integer; AValue: Double): Integer	
C#	public static extern int MT_ Help_Grating_Circle_Real_To_Steps(Int32 ALineCount, double AValue)	
	ALineCount	Number of grating lines per full circle of the grating scale, e.g., 64800
	AValue	Physical quantity, °, °/s, °/s ²
Function Returns	32-bit integer	Converted step count
	Non-zero	Function execution failed
Remarks		

3.18.11 MT_Help_Grating_Line_Steps_To_Real

Function Description: Converts grating scale pulses to physical quantities (velocity, acceleration, deceleration, etc.) for linear motion stages. The grating scale is mounted on the machinery. The electronic 4x frequency multiplication of the grating scale is already accounted for during conversion; users need not consider it further. Simply input the pulse count after 4x multiplication.

C/VC/GCC	double MT_Help_Grating_Line_Steps_To_Real(double AUnit_um, int32_t AValue)	
VB	Function MT_Help_Grating_Line_Steps_To_Real(ByVal AUnit_um As Double, ByVal AValue As Long) As Long	
Delphi	function MT_Help_Grating_Line_Steps_To_Real(AUnit_um:Double;AValue:Integer):Double	
C#	public static extern double MT_Help_Grating_Line_Steps_To_Real(double AUnit_um, Int32 AValue)	
Input	AUnit_um	Grating pitch of the grating scale, in um
	AValue	Number of grating scale pulses
Function returns	Double precision	Converted physical quantity, mm, mm/s, mm/s²
	Non-zero	Function execution failure
Remarks		

3.18.12 MT_Help_Grating_Circle_Steps_To_Real

Function Description: Converts grating scale pulses to physical quantities (velocity, acceleration, deceleration, etc.). This conversion function is used when installing grating scales on mechanical structures. The encoder's electronic quadrature conversion is already accounted for during the conversion process; users need not consider it further. Simply input the pulse count after quadrature conversion.

C/VC/GCC	double MT_Help_Grating_Circle_Steps_To_Real(int32_t ALineCount, int32_t AValue)	
VB	Function MT_Help_Grating_Circle_Steps_To_Real(ByVal ALineCount As Long, ByVal AValue As Long) As Long	
Delphi	function MT_Help_Grating_Circle_Steps_To_Real(ALineCount: Integer; AValue: Integer): Double	
C#	public static extern int MT_Help_Grating_Circle_Steps_To_Real(Int32 ALineCount, Int32 AValue)	
	ALineCount	Number of grating lines per full circle of the grating scale, e.g., 64800
	AValue	Number of pulses
Function	Double-precision	Converted physical quantity ° , °/s, °/s²

Returns	floating-point	
	Non-zero	Function execution failed
Remarks		

3.18.13 MT_Help_Encoder_Factor

Function Description: Estimation of encoder closed-loop reduction factor.

C/VC/GCC	float MT_Help_Encoder_Factor(double AStepAngle, int32_t ADiv, int32_t ALineCount)
VB	Function MT_Help_Encoder_Factor(ByVal AStepAngle As Double, ByVal ADiv As

	Long, ByVal ALineCount As Long) As Single	
Delphi	function MT_Help_Encoder_Factor(AStepAngle: Double; ADiv: Integer; ALineCount: Integer): Single	
C#	public static extern Single MT_Help_Grating_Line_Steps_To_Real(double AUnit_um, Int32 AValue)	
Input	AStepAngle	Motor rotation angle per step, e.g., 1.8° for most stepper motors
	ADiv	Driver microstepping resolution, typically 2, 4, 8, 16, etc.
	ALineCount	Number of encoder lines, e.g., 1000, 1024, etc.
Function Return	Single precision	Deceleration factor
	Non-zero	Function execution failure
Remarks		

3.18.14 MT_Help_Grating_Line_Factor

Function Description: Estimation of closed-loop deceleration coefficient for linear motion of grating scale.

C/VC/GCC	float MT_Help_Grating_Line_Factor(double AStepAngle, int32_t ADiv, double APitch, double ALineRatio, double AUnit_um)	
VB	Function MT_Help_Grating_Line_Factor (ByVal AStepAngle As Double, ByVal ADiv As Long, ByVal APitch As Double, ByVal ALineRatio As Double, ByVal AUnit_um As Double) As Single	
Delphi	function MT_Help_Grating_Line_Factor(StepAngle: Double; Div: Integer; APitch: Double; ALineRatio: Double; AUnit_um: Double): Single	
C#	public static extern Single MT_Help_Grating_Line_Factor(double AStepAngle, Int32 ADiv, double APitch, double ALineRatio, double AUnit_um)	
Input	AStepAngle	Motor rotation angle per step, e.g., typically 1.8° for two-phase stepper motors
	ADiv	Driver microstep division, typically 2, 4, 8, 16, etc.
	APitch	Linear stage pitch, i.e., displacement per full motor revolution, in mm
	ALineRatio	Linear stage transmission ratio; 1 when no transmission
	AUnit_um	The pitch of the linear encoder scale, measured in micrometers (μm)
Function Return	Single precision	Reduction ratio
	Non-zero	Function execution failure
Remarks		

3.18.15 MT_Help_Grating_Circle_Factor

Function Description: Estimation of closed-loop reduction factor for grating scale rotational motion.

C/VC/GCC	float MT_Help_Grating_Circle_Factor(double AStepAngle, int32_t ADiv, double ACircleRatio, int32_t ALineCount)
VB	Function MT_Help_Grating_Circle_Factor (ByVal AStepAngle As Double, ByVal ADiv As Long, ByVal ACircleRatio As Double, ByVal ALineCount As Long) As Single
Delphi	function MT_Help_Grating_Circle_Factor(AStepAngle: Double; ADiv: Integer;

	ACircleRatio: Double; ALineCount: Integer): Single	
C#	public static extern Single MT_Help_Grating_Circle_Factor(double AStepAngle, Int32 ADiv, double ACircleRatio, int32 ALineCount)	
Input	AStepAngle	Motor rotation angle per step, e.g., 1.8° for most stepper motors
	ADiv	Driver microstep division ratio, typically 2, 4, 8, 16, etc.
	ACircleRatio	Rotary stage transmission ratio
	ALine_Count	Number of grating lines per full revolution of the linear encoder
Function Return	Single Precision	Reduction ratio
	Non-zero	Function execution failure
Remarks		

4 Document Version History

Serial Number	Version	Document Content	Revision Date	Remarks
1	V1.0	- Completed SDK Calling Conventions - Concepts and Terminology Related to Motion Control - Motion Control Modes - Programming Workflow - Single Controller Programming Guide	May 4, 2015	MT_API.dll V3.11 and Later Versions
2	V2.0	- Convention for Adding Serial Numbers - Added description of development interface for multi-controller devices - Added measurement unit conversion description - Added coordinate system description - Added descriptions for standard zero-point search mode and zero-point priority search mode	April 28, 2021	MT_API.dll V4.3 and later versions

		6. Added description of synchronous trigger function		
--	--	--	--	--